

Project Code Compilation

Unified Benchmark of Grokking Predictors

Generated: 2026-09-25

Project root: /Users/jonathanjohn/Documents/grokking-benchmark

Total code files: **50**

Contents

- 06_Code/configs/nanda_unified.yaml
- 06_Code/requirements.txt
- 06_Code/scripts/analyze_nanda_unified.py
- 06_Code/scripts/analyze_threshold.py
- 06_Code/scripts/collage_images.py
- 06_Code/scripts/compile_context_bundle.py
- 06_Code/scripts/compile_python_files.py
- 06_Code/scripts/debug_detection.py
- 06_Code/scripts/generate_file.py
- 06_Code/scripts/inherticance_example.py
- 06_Code/scripts/markPhase1Task1Complete.gs
- 06_Code/scripts/md_to_image.py
- 06_Code/scripts/run_nanda_benchmark.py
- 06_Code/scripts/scaffold.py
- 06_Code/src/data/modular_arithmetic.py
- 06_Code/src/models/model_shadow_with_layernorm.py
- 06_Code/src/models/transformer_four_head.py
- 06_Code/src/plot_results_four_head.py
- 06_Code/src/plot_shadow_layernorm.py
- 06_Code/src/predictors/age.py
- 06_Code/src/predictors/dropout.py
- 06_Code/src/predictors/l2_norm.py
- 06_Code/src/predictors/spectral.py
- 06_Code/src/train_four_head.py
- 06_Code/src/train_shadow_layernorm.py
- 06_Code/src/unified_measurements.py
- 08_Experiments/nanda_l2_p113/__init__.py
- 08_Experiments/nanda_l2_p113/data/__init__.py
- 08_Experiments/nanda_l2_p113/data/modular_arithmetic.py
- 08_Experiments/nanda_l2_p113/measurements.py
- 08_Experiments/nanda_l2_p113/models/__init__.py
- 08_Experiments/nanda_l2_p113/models/transformer_four_head.py
- 08_Experiments/nanda_l2_p113/predictors/__init__.py
- 08_Experiments/nanda_l2_p113/predictors/l2_norm.py
- 08_Experiments/nanda_l2_p113/run_benchmark.py
- 08_Experiments/nanda_l2_p113/train.py
- archive/Deprecated_Code/old_benchmarks/run_full_benchmark.py.deprecated
- archive/Deprecated_Code/single_head/models/transformer.py
- archive/Deprecated_Code/single_head/plot_results.py
- archive/Deprecated_Code/single_head/train.py
- archive/Deprecated_Code/src_p97_mainstream_old/data/modular_arithmetic.py
- archive/Deprecated_Code/src_p97_mainstream_old/models/model_shadow_with_layernorm.py
- archive/Deprecated_Code/src_p97_mainstream_old/models/transformer_four_head.py
- archive/Deprecated_Code/src_p97_mainstream_old/plot_results_four_head.py
- archive/Deprecated_Code/src_p97_mainstream_old/plot_shadow_layernorm.py
- archive/Deprecated_Code/src_p97_mainstream_old/predictors/dropout.py

- 47. `archive/Deprecated_Code/src_p97_mainstream_old/predictors/l2_norm.py`
- 48. `archive/Deprecated_Code/src_p97_mainstream_old/train_four_head.py`
- 49. `archive/Deprecated_Code/src_p97_mainstream_old/train_shadow_layernorm.py`
- 50. `archive/Deprecated_Code/src_p97_mainstream_old/unified_measurements.py`

1. 06_Code/configs/nanda_unified.yaml

```
# =====
# Nanda-Unified – the SINGLE common benchmark protocol.
#
# Every grokking predictor in this thesis (L2-Norm, Dropout, Spectral,
# HTSR Alpha, AGE, Weight-PCA, Higher-MI, Commutator Defect, ...) is to
# be measured on a model trained with EXACTLY these settings. No
# predictor gets its own substrate.
#
# This file is the source of truth. src/train_four_head.py loads it at
# startup and asserts its own constants against it, so the two can never
# drift silently.
#
# Why this exact configuration (see context.md, Sept 3–4 2026 sessions):
# - It is a faithful replication of Nanda et al. (2023) "Progress
#   Measures for Grokking via Mechanistic Interpretability", whose
#   1-layer / full-batch / no-warmup modular-addition transformer is
#   the substrate the grokking-predictor literature standardised on.
# - The old default substrate (p=97, PyTorch-default init, betas
#   0.9/0.999) carried two artefacts that pollute every weight-space
#   predictor: a token-embedding init transient crushed by weight
#   decay in the first ~2000 epochs, and a beta2=0.999 slingshot
#   spike at the grok transition. Both are gone here.
# - The aligned setup reproduces Nanda Figure 7 (weight norm rises
#   during memorisation, declines through circuit formation, drops
#   through grokking, then flat floor) cleanly across seeds.
# =====

task:
  operation: "(a + b) mod p"
  modulus: 113                # Nanda mainline prime. NOT 97.
  equals_token_id: 113        # number tokens are 0..p-1; "=" token id is p
  vocab_size: 114              # p + 1
  sequence: "a b ="          # 3 positions; logits read at the "=" position
  read_position: 2
  train_fraction: 0.3         # 30 / 70 random split
  split: random

architecture:
  layers: 1
  num_heads: 4
  d_model: 128
  head_dim: 32                # d_model / num_heads
  d_mlp: 512                  # 4 * d_model
  mlp_activation: relu
  layernorm: false            # Nanda uses NO LayerNorm (paper Section 3)
  tied_embeddings: false
  positional_embedding: learned
  attention_bias: false       # no bias in query / key / value
  embedding_bias: false
  unembedding_bias: false     # no bias in output_head
  mlp_bias: true              # Nanda keeps b_in / b_out in the MLP

init:
  scheme: transformerlens_small
  init_std_formula: "0.8 / sqrt(d_model)"
  init_std: null              # null –> computed from the formula (~0.0707 for d_model=128)
  applies_to:                 # every weight matrix, embeddings included
    - token_embedding
    - position_embedding
    - query
    - key
    - value
    - mlp_in
    - mlp_out
    - output_head
  bias_init: zeros            # MLP biases start at 0 (TransformerLens convention)

optimizer:
  name: adamw
  lr: 0.001
  weight_decay: 1.0           # decoupled (torch.optim.AdamW)
  betas: [0.9, 0.98]         # NOT the PyTorch default (0.9, 0.999)

training:
  epochs: 40000
  full_batch: true            # batch_size = floor(train_fraction * p * p)
  warmup_steps: 0
  loss: cross_entropy
  device_preference: [mps, cpu]

seeds:
  per_predictor_min_runs: 3    # fresh torch.seed() per run, recorded to seed.npy
  # (The post-grok limit-cycle check runs on every seed of the benchmark,
  # i.e. --seeds; it has no separate run count. The old fixed value 5 was removed.)
```

```

# -----
# Evaluation constants read by 06_Code/scripts/run_nanda_benchmark.py.
# Each one carries its source, so none of them is an unexplained magic number.
# -----
evaluation:
# Grok epoch = first epoch at which test accuracy exceeds the threshold.
# All three are computed and reported for every seed. The FIRST entry is the
# primary one (used for every predictor's signal-epoch / grok-epoch ratio).
# Source: 0.9 is the literature standard (grokking time = delay until
# accuracy surpasses 0.9); 0.60/0.75/0.90/0.95 are used in other papers.
# 0.95 and 0.99 are added so the conclusions do not rest on one arbitrary cut.
grok_thresholds: [0.9, 0.95, 0.99]

# Console cadence: number of progress lines printed per run.
# Source: own engineering choice; log_every = epochs // log_lines_per_run.
log_lines_per_run: 40

# Post-grok limit-cycle diagnostic.
# settle_fraction: the window starts this fraction of the run AFTER grok, so
# the immediate post-grok transient is excluded (settle = fraction * epochs).
# dip_below: primary grok threshold is used (a "dip" = test acc falls back
# below the grokked level), so no separate number is needed.
# std_above: a tail with min &lt; primary threshold AND std > std_above is called
# a limit cycle. Source: own diagnostic, set from the p=113 run_1 artefact
# (test acc swinging 0.5-1.0); not a literature standard.
limit_cycle:
    settle_fraction: 0.05
    std_above: 0.05

# Dropout predictor.
dropout:
# Multi-rate sweep. Source: own choice, same five rates as the earlier sweeps.
rates: [0.1, 0.3, 0.5, 0.7, 0.9]
# Dropout-Variance signal (Salah & Yevick, arXiv:2507.11645).
# n_samples: 100 stochastic passes per checkpoint = the paper's own value
# (earlier 30 was a compute saving, no longer used).
# variance_rate: 0.5, where the paper's Dropout Robustness Curve (their
# Fig. 2) separates pre- and post-grok best. (Their Fig. 1 sweep used 0.3.)
n_samples: 100
variance_rate: 0.5

# L2-Norm predictor windows. Source: own choices from the 2026-08-07 sessions
# (see context.md), identical to 06_Code/src/train_four_head.py. Not from a paper.
l2_norm:
    fast_window: 50
    slow_window: 200
    ma_of_ma_fast_window: 20
    skip_epochs: 100
    quiet_epoch_cutoff: 90

logging:
# Weight-norm quantities recorded every epoch for the weight-space predictors.
weight_norm_total_l2: true          # sqrt(sum w^2) over all parameters
weight_norm_total_sum_w2: true      # sum w^2 over all parameters (Nanda Fig 7 quantity)
weight_norm_per_module_sum_w2: true # groups: token_embedding, position_embedding,
                                     #          attention_qkv, mlp, output_head

```

2. 06_Code/requirements.txt

```
# — Core ML —————
torch>=2.1.0      # MPS (Apple Silicon) support solid from 2.x
torchvision>=0.16.0
numpy>=1.24.0
scipy>=1.11.0

# — Transformer / grokking reproduction (Nanda et al.) —————
einops>=0.7.0      # tensor rearrangements used in transformer implementations
transformer_lens>=1.14.0 # Nanda et al. mechanistic interpretability library

# — Predictor-specific —————
scikit-learn>=1.3.0 # Weight-PCA, Correlation Traps, mutual information utilities
weightwatcher>=0.7.0 # HTSR Alpha predictor (heavy-tailed self-regularisation)

# — Experiment tracking & logging —————
wandb>=0.16.0      # experiment tracking for ~80 training runs (Phase 4 sweep)
tensorboard>=2.14.0 # optional local alternative to wandb
tqdm>=4.66.0       # progress bars for training loops

# — Data & analysis —————
pandas>=2.1.0      # results tables, leaderboard
matplotlib>=3.8.0  # Plots 1–4 (canonical thesis figures)
seaborn>=0.13.0    # statistical plots, heatmaps

# — Config & utilities —————
pyyaml>=6.0.1      # experiment config files
jsonlines>=4.0.0   # structured run logging
```

3. 06_Code/scripts/analyze_nanda_unified.py

```
#!/usr/bin/env python3
"""
analyze_nanda_unified.py
=====
```

Post-hoc analysis + plots for the 5-seed Nanda-Unified run (L2 Norm, Dropout, Spectral, AGE). Reads only saved results, trains nothing.

For every predictor signal it asks one question: does the signal move BEFORE the model groks (test acc first > 0.9), and does the moment it moves follow the grok epoch from seed to seed?

Event epochs (all computed from the saved per-checkpoint arrays):

- * Implemented predictor outputs, taken straight from summary.json (L2 MA-crossover, L2 MA-of-MA zero crossing, dropout-variance peak, spectral k90-min / alignment-max, AGE NC1-min).
- * "Fraction-of-change" epochs: first epoch at which a signal has covered 10% / 50% / 90% of the way from its starting value to its final extreme. NC1 is measured in log10 because it falls by three orders of magnitude.

lead = grok_epoch - event_epoch (positive = signal fired before grok)

Usage (from the repo root):

```
python 06_Code/scripts/analyze_nanda_unified.py
Outputs go to 08_Experiments/results/nanda_unified/analysis/.
"""
```

```
import json
import os
```

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
REPO_ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
RESULTS = os.path.join(REPO_ROOT, "08_Experiments", "results", "nanda_unified")
OUT = os.path.join(RESULTS, "analysis")
FRACTIONS = (0.1, 0.5, 0.9)
```

```
def load_seed(seed):
    d = os.path.join(RESULTS, f"seed_{seed}")
    with open(os.path.join(d, "summary.json")) as handle:
        summary = json.load(handle)
    ld = lambda sub, name: np.load(os.path.join(d, sub, name))
    return {
        "summary": summary,
        "grok": summary["grok_epoch"],
        "test_acc": ld("training", "test_acc_history.npy"),
        "train_acc": ld("training", "train_acc_history.npy"),
        "sum_w2": ld("l2_norm", "sum_w2_history.npy"),
        "var_ep": ld("dropout", "dropout_variance_checkpoints.npy"),
        "var": ld("dropout", "dropout_variance_history.npy"),
        "sp_ep": ld("spectral", "spectral_checkpoints.npy"),
        "align": ld("spectral", "spectral_alignment.npy"),
        "k90": ld("spectral", "spectral_k90.npy"),
        "age_ep": ld("age", "age_checkpoints.npy"),
        "nc1": ld("age", "age_nc1.npy"),
    }
```

```
def smooth(y, window):
    kernel = np.ones(window) / window
    padded = np.pad(y, (window // 2, window - 1 - window // 2), mode="edge")
    return np.convolve(padded, kernel, mode="valid")
```

```
def fraction_epochs(epochs, y, fractions=FRACTIONS):
    """First epoch at which y has covered `f` of the way from its start to
    its farthest value. Returns {f: epoch}."""
    y0 = np.median(y[:5])
    extreme = y[np.argmax(np.abs(y - y0))]
    sign = np.sign(extreme - y0)
    out = {}
    for f in fractions:
        target = y0 + f * (extreme - y0)
        idx = np.nonzero(sign * (y - target) >= 0)[0]
        out[f] = int(epochs[idx[0]])
    return out
```

```
def l2_decline_epoch(sum_w2, fraction=0.5):
    """First epoch after the (smoothed) sum-of-squares peak at which it has
```

```

fallen `fraction` of the way from that peak to its final value."""
y = smooth(sum_w2, 200)
peak = int(np.argmax(y))
target = y[peak] - fraction * (y[peak] - y[-1])
idx = np.nonzero(y[peak:] <= target)[0]
return peak, int(peak + idx[0])

def build_events(runs):
    """events[predictor_signal][seed] = epoch."""
    ev = {}

    def put(name, seed, epoch):
        ev.setdefault(name, {})[seed] = float(epoch)

    for s, r in enumerate(runs):
        sm = r["summary"]
        put("L2: MA crossover (implemented)", s, sm["l2_predictor"]["ma_crossover_epoch"])
        put("L2: MA-of-MA zero crossing (implemented)", s, sm["l2_predictor"]["ma_of_ma_zero_crossing_epoch"])
        _, half = l2_decline_epoch(r["sum_w2"], 0.5)
        put("L2: sum(w^2) half-way down from peak", s, half)

        put("Dropout: variance peak (implemented)", s, sm["dropout_variance_predictor"]["variance_peak_epoch"])
        fr = fraction_epochs(r["var_ep"], r["var"])
        put("Dropout: variance 10% of rise", s, fr[0.1])
        put("Dropout: variance 50% of rise", s, fr[0.5])

        put("Spectral: k90 minimum (implemented)", s, sm["spectral_predictor"]["k_90_min_epoch"])
        put("Spectral: alignment maximum (implemented)", s, sm["spectral_predictor"]["alignment_max_epoch"])
        fr = fraction_epochs(r["sp_ep"], r["align"])
        put("Spectral: alignment 10% of rise", s, fr[0.1])
        put("Spectral: alignment 50% of rise", s, fr[0.5])
        fr = fraction_epochs(r["sp_ep"], r["k90"])
        put("Spectral: k90 50% of fall", s, fr[0.5])

        put("AGE: NC1 minimum (implemented)", s, sm["age_predictor"]["nc1_min_epoch"])
        fr = fraction_epochs(r["age_ep"], np.log10(r["nc1"]))
        put("AGE: log10 NC1 10% of fall", s, fr[0.1])
        put("AGE: log10 NC1 50% of fall", s, fr[0.5])
    return ev

def rank_corr(a, b):
    ra = np.argsort(np.argsort(a))
    rb = np.argsort(np.argsort(b))
    return float(np.corrcoef(ra, rb)[0, 1])

def summarise(ev, grok):
    rows = []
    g = np.array(grok, dtype=float)
    for name, per_seed in ev.items():
        e = np.array([per_seed[s] for s in range(len(grok))])
        lead = g - e
        pear = float(np.corrcoef(e, g)[0, 1]) if e.std() > 0 else float("nan")
        rows.append({
            "signal": name,
            "epochs": e.tolist(),
            "lead": lead.tolist(),
            "seeds_before_grok": int((lead > 0).sum()),
            "spearman": rank_corr(e, g),
            "pearson": pear,
            "epoch_cv": float(e.std() / e.mean()),
        })
    return rows

def print_table(rows, grok):
    print(f"\ngrok epochs: {grok}")
    print(f"{'signal':<46}{'event epoch per seed':<38}{'before':<7}{'rho':<7}{'r':<7}")
    for r in rows:
        ep = " ".join(f"{int(x):<6}" for x in r["epochs"])
    print(f"{'r':<46}{'signal':<46}{'ep':<38}{'seeds_before_grok':<5}{'spearman':<7.2f}{'r':<7.2f}{'pearson':<7.2f}")

# ----- plots -----

def norm01(y):
    lo, hi = float(np.min(y)), float(np.max(y))
    return (y - lo) / (hi - lo) if hi > lo else np.zeros_like(y)

def plot_curves(runs, path):
    fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
    cols = plt.cm.tab10(np.arange(len(runs)))
    for s, r in enumerate(runs):

```

```

    ax[0].plot(r["test_acc"], color=cols[s], lw=1.2, label=f"seed {s}")
    ax[0].axvline(r["grok"], color=cols[s], ls=":", lw=0.9)
    ax[1].plot(r["train_acc"], color=cols[s], lw=1.2)
ax[0].set(title="Test accuracy (dotted = grok epoch, test acc > 0.9)", xlabel="epoch", ylabel="accuracy")
ax[1].set(title="Train accuracy", xlabel="epoch", ylabel="accuracy")
ax[0].legend()
fig.tight_layout()
fig.savefig(path, dpi=140)
plt.close(fig)

def plot_signals(runs, ev, path):
    cols = [
        ("Weight norm sum(w^2)", lambda r: (np.arange(len(r["sum_w2"])), r["sum_w2"])),
        [("L2: MA crossover (implemented)", "tab:blue"),
         ("L2: MA-of-MA zero crossing (implemented)", "tab:cyan")],
        ("Dropout variance", lambda r: (r["var_ep"], r["var"])),
        [("Dropout: variance peak (implemented)", "tab:orange")],
        ("Spectral alignment", lambda r: (r["sp_ep"], r["align"])),
        [("Spectral: alignment maximum (implemented)", "tab:green"),
         ("Spectral: alignment 50% of rise", "tab:olive")],
        ("AGE NC1 (log scale)", lambda r: (r["age_ep"], r["nc1"])),
        [("AGE: NC1 minimum (implemented)", "tab:red"),
         ("AGE: log10 NC1 50% of fall", "tab:pink")],
    ]
    n = len(runs)
    fig, axes = plt.subplots(n, len(cols), figsize=(4.4 * len(cols), 2.4 * n), sharex=True)
    for s, r in enumerate(runs):
        for c, (title, getter, marks) in enumerate(cols):
            ax = axes[s, c]
            x, y = getter(r)
            ax.plot(x, y, color="black", lw=0.9)
            if c == 3:
                ax.set_yscale("log")
            ax.axvline(r["grok"], color="crimson", ls="--", lw=1.2, label="grok")
            for name, colour in marks:
                ax.axvline(ev[name][s], color=colour, lw=1.1, label=name.split(":")[1].strip()[26])
            if s == 0:
                ax.set_title(title, fontsize=10)
            if c == 0:
                ax.set_ylabel(f"seed {s}")
            if s == n - 1:
                ax.set_xlabel("epoch")
            if s == 0:
                ax.legend(fontsize=6, loc="upper right")
    fig.suptitle("Predictor signals vs grok epoch (red dashed = grok, coloured lines = predictor event)", y=1.0)
    fig.tight_layout()
    fig.savefig(path, dpi=130)
    plt.close(fig)

def plot_scatter(ev, grok, path):
    show = [
        "L2: MA crossover (implemented)",
        "L2: MA-of-MA zero crossing (implemented)",
        "Dropout: variance peak (implemented)",
        "Spectral: alignment maximum (implemented)",
        "Spectral: alignment 10% of rise",
        "AGE: NC1 minimum (implemented)",
        "AGE: log10 NC1 10% of fall",
    ]
    fig, ax = plt.subplots(figsize=(7.5, 6.5))
    lim = 42000
    ax.plot([0, lim], [0, lim], color="grey", ls="--", lw=1)
    ax.text(28000, 24500, "fires at grok", color="grey", rotation=33, fontsize=8)
    ax.fill_between([0, lim], [0, lim], 0, color="tab:green", alpha=0.06)
    ax.text(1500, 900, "below the line = fires BEFORE grok", color="tab:green", fontsize=8)
    for name, mk in zip(show, "os^Dv<&gt;"):
        e = [ev[name][s] for s in range(len(grok))]
        ax.scatter(grok, e, marker=mk, s=55, label=name, alpha=0.85)
    ax.set(xlim=(0, lim), ylim=(0, lim), xlabel="grok epoch of the seed", ylabel="predictor event epoch")
    ax.set_title("Does the predictor fire before grok, and does it follow grok?")
    ax.legend(fontsize=7, loc="upper left")
    fig.tight_layout()
    fig.savefig(path, dpi=140)
    plt.close(fig)

def plot_leads(ev, grok, path):
    names = [
        "L2: MA crossover (implemented)",
        "L2: MA-of-MA zero crossing (implemented)",
        "Dropout: variance peak (implemented)",
        "Spectral: alignment maximum (implemented)",
        "Spectral: alignment 10% of rise",
        "AGE: NC1 minimum (implemented)",
    ]

```

```

    "AGE: log10 NC1 10% of fall",
]
n = len(grok)
fig, ax = plt.subplots(figsize=(11, 4.8))
width = 0.8 / n
for s in range(n):
    leads = [grok[s] - ev[nm][s] for nm in names]
    ax.bar(np.arange(len(names)) + s * width, leads, width, label=f"seed {s}")
ax.axhline(0, color="black", lw=1)
ax.set_xticks(np.arange(len(names)) + 0.4 - width / 2)
ax.set_xticklabels([nm.replace(" (implemented)", "") for nm in names], rotation=25, ha="right", fontsize=8)
ax.set_ylabel="lead time = grok epoch - event epoch (positive = early warning)",
    title="Lead time per predictor and seed")
ax.legend(fontsize=7, ncol=n)
fig.tight_layout()
fig.savefig(path, dpi=140)
plt.close(fig)

def main():
    os.makedirs(OUT, exist_ok=True)
    seeds = sorted(int(n.split("_")[1]) for n in os.listdir(RESULTS) if n.startswith("seed_"))
    runs = [load_seed(s) for s in seeds]
    grok = [r["grok"] for r in runs]
    ev = build_events(runs)
    rows = summarise(ev, grok)
    print_table(rows, grok)

    with open(os.path.join(OUT, "predictor_events.json"), "w") as handle:
        json.dump({"grok_epochs": grok, "signals": rows}, handle, indent=2)

    plot_curves(runs, os.path.join(OUT, "01_grokking_curves.png"))
    plot_signals(runs, ev, os.path.join(OUT, "02_signals_vs_grok.png"))
    plot_scatter(ev, grok, os.path.join(OUT, "03_event_vs_grok_scatter.png"))
    plot_leads(ev, grok, os.path.join(OUT, "04_lead_times.png"))
    print(f"\nWrote plots and predictor_events.json to {OUT}")

if __name__ == "__main__":
    main()

```

4. 06_Code/scripts/analyze_threshold.py

```
import numpy as np

# Load the data
l2_norm_history = np.load("l2_norm_history.npy")
test_acc_history = np.load("test_acc_history.npy")

# Compute rate of decline (how much L2 norm drops each epoch)
rate_of_decline = np.diff(l2_norm_history) * -1

# Find when test accuracy crosses 90% (grok event)
grok_epoch = np.where(np.array(test_acc_history) > 0.9)[0][0]
print(f"Grok epoch (test acc > 90%): {grok_epoch}")
print(f"L2 norm at grok: {l2_norm_history[grok_epoch]:.4f}")

# Analyze rate of decline
print(f"\nRate of decline statistics:")
print(f"  Min: {rate_of_decline.min():.6f}")
print(f"  Max: {rate_of_decline.max():.6f}")
print(f"  Mean: {rate_of_decline.mean():.6f}")
print(f"  Median: {np.median(rate_of_decline):.6f}")
print(f"  Std: {rate_of_decline.std():.6f}")

# Look at rate of decline around the grok region (500 epochs before and after)
grok_window_start = max(0, grok_epoch - 500)
grok_window_end = min(len(rate_of_decline), grok_epoch + 500)
rates_around_grok = rate_of_decline[grok_window_start:grok_window_end]

print(f"\nRate of decline in the grok window (epoch {grok_window_start}-{grok_window_end}):")
print(f"  Mean: {rates_around_grok.mean():.6f}")
print(f"  Max: {rates_around_grok.max():.6f}")
print(f"  Percentile 75: {np.percentile(rates_around_grok, 75):.6f}")
print(f"  Percentile 90: {np.percentile(rates_around_grok, 90):.6f}")

# Look at rates BEFORE the grok (epoch 500-grok_epoch)
if grok_epoch > 500:
    pre_grok_start = 500
    rates_before_grok = rate_of_decline[pre_grok_start:grok_epoch]
    print(f"\nRate of decline BEFORE grok (epoch {pre_grok_start}-{grok_epoch}):")
    print(f"  Mean: {rates_before_grok.mean():.6f}")
    print(f"  Max: {rates_before_grok.max():.6f}")
    print(f"  Percentile 75: {np.percentile(rates_before_grok, 75):.6f}")
    print(f"  Percentile 90: {np.percentile(rates_before_grok, 90):.6f}")

# Recommended thresholds
print(f"\n=== THRESHOLD RECOMMENDATIONS ===")
pre_grok_mean = rates_before_grok.mean()
pre_grok_p90 = np.percentile(rates_before_grok, 90)
grok_window_mean = rates_around_grok.mean()
grok_window_max = rates_around_grok.max()

print(f"Pre-grok mean rate: {pre_grok_mean:.6f}")
print(f"Pre-grok 90th percentile: {pre_grok_p90:.6f}")
print(f"Grok window mean rate: {grok_window_mean:.6f}")
print(f"Grok window max rate: {grok_window_max:.6f}")

print(f"\nSuggested thresholds (aim for detection ~500-1000 epochs before grok):")
print(f"  Conservative (low false positives): {pre_grok_p90:.6f}")
print(f"  Balanced: {grok_window_mean:.6f}")
print(f"  Aggressive (catch early): {pre_grok_mean:.6f}")
```

5. 06_Code/scripts/collage_images.py

```
#!/usr/bin/env python3

"""
image_collage.py

Combine all images in a folder into a single collage either
horizontally or vertically.

Requirements:
    pip install pillow

Usage:
    python image_collage.py
"""

from pathlib import Path
from PIL import Image

SUPPORTED_EXTENSIONS = {".png", ".jpg", ".jpeg", ".bmp", ".webp"}

def get_images(folder):
    images = [
        f for f in sorted(folder.iterdir())
        if f.suffix.lower() in SUPPORTED_EXTENSIONS
    ]
    return images

def horizontal_collage(image_paths, output_path):
    images = [Image.open(p).convert("RGB") for p in image_paths]

    total_width = sum(img.width for img in images)
    max_height = max(img.height for img in images)

    collage = Image.new("RGB", (total_width, max_height), "white")

    x = 0
    for img in images:
        collage.paste(img, (x, 0))
        x += img.width

    collage.save(output_path)
    print(f"\n Saved: {output_path}")

def vertical_collage(image_paths, output_path):
    images = [Image.open(p).convert("RGB") for p in image_paths]

    max_width = max(img.width for img in images)
    total_height = sum(img.height for img in images)

    collage = Image.new("RGB", (max_width, total_height), "white")

    y = 0
    for img in images:
        collage.paste(img, (0, y))
        y += img.height

    collage.save(output_path)
    print(f"\n Saved: {output_path}")

def main():
    print("=" * 60)
    print("Image Collage Generator")
    print("=" * 60)

    while True:
        folder = Path(
            input("\nEnter folder containing images:\n> ").strip().strip("'")
        )

        if folder.exists() and folder.is_dir():
            break

        print("\n Invalid folder.")

    image_paths = get_images(folder)

    if not image_paths:
        print("\n No images found.")
        return
```

```

print(f"\nFound {len(image_paths)} images.")

while True:
    mode = input(
        "\nChoose layout:\n"
        "H = Horizontal\n"
        "V = Vertical\n"
        "> "
    ).strip().lower()

    if mode in ("h", "v"):
        break

    output = input(
        "\nOutput file name (leave blank for collage.png):\n> "
    ).strip()

    if not output:
        output = "collage.png"

    output_path = folder / output

    if mode == "h":
        horizontal_collage(image_paths, output_path)
    else:
        vertical_collage(image_paths, output_path)

if __name__ == "__main__":
    main()

```

6. 06_Code/scripts/compile_context_bundle.py

```
#!/usr/bin/env python3
"""
compile_context_bundle.py
=====

Builds `full_context_profile.pdf` in the repo root – a curated, single-file
context bundle meant to be handed to a NEW chat session (Meta AI or any
other assistant) that has no memory of this project.

This is NOT a "compile everything" tool like the other tools/compile_*.py
scripts. It deliberately picks the smallest set of files/sections that let a
fresh session pick up the thesis work correctly: project identity, the last
two session summaries from context.md, the benchmark config, latest results,
the master runner's key parts, the two CLOSED predictors' full code, and the
shared model/data files.

Usage:
python tools/compile_context_bundle.py
"""

import json
import os
import subprocess
import sys
from datetime import date

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_PDF = os.path.join(REPO_ROOT, "full_context_profile.pdf")

TODAY = date.today().isoformat()

EXCLUDED_NOTES = [
    ("08_Experiments/results/nanda_unified/ (legacy: was runs/four_head/)", "old 3-run pipeline, superseded by
    results/nanda_unified/ "
    "(run_full_benchmark.py itself is deprecated/archived)"),
    ("archive/", "historical pre-Nanda-Unified data, not needed to continue current work"),
    ("08_Experiments/results/nanda_unified/seed_*/checkpoints/*.pt", "large binary model weights, "
    "not readable context, and not needed to explain the benchmark"),
    ("08_Experiments/results/nanda_unified/seed_*/{l2_norm,dropout}/*.npy", "large raw numeric arrays; "
    "the per-seed summary.json already carries the derived predictor signals"),
    ("graphify-out/", "generated knowledge-graph cache, rebuildable from source anytime"),
    ("Literature/*.pdf", "source papers themselves – cited by name in context.md, "
    "not needed verbatim for a new chat to continue engineering work"),
    ("context.md sessions before the last 2", "older history superseded by the L2-Norm "
    "and Dropout closure decisions already summarized in the last 2 sessions"),
]

def run_git(args):
    try:
        return subprocess.run(
            ["git"] + args, cwd=REPO_ROOT, capture_output=True, text=True, check=True
        ).stdout.strip()
    except Exception as exc:
        return f"(git command failed: {exc})"

def read_text(path, max_lines=None):
    full_path = os.path.join(REPO_ROOT, path)
    if not os.path.exists(full_path):
        return None
    with open(full_path, "r", encoding="utf-8", errors="replace") as handle:
        lines = handle.readlines()
        truncated = False
        if max_lines and len(lines) > max_lines:
            lines = lines[:max_lines]
            truncated = True
        text = "".join(lines)
        if truncated:
            text += "\n... [truncated] ...\n"
    return text

def read_text_head_tail(path, head=350, tail=50):
    full_path = os.path.join(REPO_ROOT, path)
    if not os.path.exists(full_path):
        return None
    with open(full_path, "r", encoding="utf-8", errors="replace") as handle:
        lines = handle.readlines()
        if len(lines) <= head + tail:
            return "".join(lines)
        gap = len(lines) - head - tail
        return (
            "".join(lines[:head])

```

```

        + f"\n... [gap: {gap} lines omitted] ...\n\n"
        + "".join(lines[-tail:])
    )

def extract_last_n_sessions(context_md_text, n=2):
    """Split context.md on '## Session Summary' / '### Session Summary'
    headers and return the last n session blocks, full text."""
    lines = context_md_text.splitlines()
    header_idx = [
        i for i, line in enumerate(lines)
        if line.strip().startswith("## Session Summary")
        or line.strip().startswith("### Session Summary")
    ]
    if not header_idx:
        return context_md_text[-8000:]
    chosen_starts = header_idx[-n:]
    start = chosen_starts[0]
    return "\n".join(lines[start:])

def extract_runner_constants_and_registry(runner_text, max_lines=400):
    """Pull out the constants block, CHECKPOINT_PREDICTOR_FUNCS registry, and
    the _checkpoint_predictor_dropout_variance docstring from
    run_nanda_benchmark.py, capped at max_lines total."""
    lines = runner_text.splitlines()
    chunks = []

    def grab(start_marker, end_marker=None, max_span=200):
        for i, line in enumerate(lines):
            if start_marker in line:
                j = i
                count = 0
                block = []
                while j < len(lines) and count < max_span:
                    block.append(lines[j])
                    if end_marker and end_marker in lines[j] and j > i:
                        break
                    j += 1
                    count += 1
                return "\n".join(block)
        return None

    constants = grab("GROK_ACC_THRESHOLD = ", end_marker="DROPOUT_VARIANCE_NUM_CHECKPOINTS", max_span=40)
    if constants:
        chunks.append("--- Constants block ---\n" + constants)

    docstring = grab('def _checkpoint_predictor_dropout_variance', end_marker='"""', max_span=25)
    if docstring:
        # grab up to the closing triple-quote (second occurrence)
        for i, line in enumerate(lines):
            if 'def _checkpoint_predictor_dropout_variance' in line:
                block = []
                quote_count = 0
                j = i
                while j < len(lines) and quote_count < 2:
                    block.append(lines[j])
                    quote_count += lines[j].count('"""')
                    j += 1
                docstring = "\n".join(block)
                break
        chunks.append("\n--- _checkpoint_predictor_dropout_variance docstring "
            "(reference shape for future checkpoint-only predictors "
            "like Spectral) ---\n" + docstring)

    registry = grab("CHECKPOINT_PREDICTOR_FUNCS = {", end_marker="CHECKPOINT_ONLY_PREDICTORS = ", max_span=15)
    if registry:
        chunks.append("\n--- CHECKPOINT_PREDICTOR_FUNCS registry ---\n" + registry)

    text = "\n".join(chunks)
    text_lines = text.splitlines()
    if len(text_lines) > max_lines:
        text = "\n".join(text_lines[:max_lines]) + "\n... truncated ..."
    return text

def build_seed_table():
    rows = [{"seed", "grok_epoch", "l2_ma_crossover_epoch", "dropout_variance_peak_epoch", "dropout_gap@0.5"}]
    seed_dir = os.path.join(REPO_ROOT, "results", "nanda_unified")
    if not os.path.isdir(seed_dir):
        return rows
    for name in sorted(os.listdir(seed_dir)):
        if not name.startswith("seed_"):
            continue
        summary_path = os.path.join(seed_dir, name, "summary.json")
        if not os.path.exists(summary_path):

```

```

        continue
    with open(summary_path, "r", encoding="utf-8") as handle:
        summary = json.load(handle)
        grok_epoch = summary.get("grok_epoch", "-")
        l2_epoch = summary.get("l2_predictor", {}).get("ma_crossover_epoch", "-")
        dv_epoch = summary.get("dropout_variance_predictor", {}).get("variance_peak_epoch", "-")
        gap05 = summary.get("dropout_final_gap_by_rate", {}).get("0.5", "-")
        rows.append((
            name,
            str(grok_epoch),
            f"{l2_epoch:.1f}" if isinstance(l2_epoch, float) else str(l2_epoch),
            str(dv_epoch),
            f"{gap05:.4f}" if isinstance(gap05, float) else str(gap05),
        ))
    return rows

def pretty_json(path, max_lines=300):
    full_path = os.path.join(REPO_ROOT, path)
    if not os.path.exists(full_path):
        return f"(not found: {path})"
    with open(full_path, "r", encoding="utf-8") as handle:
        data = json.load(handle)
        text = json.dumps(data, indent=2)
        lines = text.splitlines()
        if len(lines) > max_lines:
            text = "\n".join(lines[:max_lines]) + "\n... [truncated] ..."
        return text

def build_with_reportlab():
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.units import inch
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.enums import TA_LEFT
    from reportlab.platypus import (
        SimpleDocTemplate, Paragraph, Preformatted, Spacer, PageBreak,
    )
    from reportlab.lib import colors

    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        "TitleBig", parent=styles["Title"], fontSize=20, spaceAfter=14)
    h1 = ParagraphStyle(
        "H1", parent=styles["Heading1"], fontSize=15, spaceBefore=6, spaceAfter=8,
        textColor=colors.HexColor("#1a3d6d"))
    h2 = ParagraphStyle(
        "H2", parent=styles["Heading2"], fontSize=12, spaceBefore=10, spaceAfter=6)
    body = ParagraphStyle(
        "BodyBig", parent=styles["BodyText"], fontSize=10, leading=13, alignment=TA_LEFT)
    mono = ParagraphStyle(
        "Mono", fontName="Courier", fontSize=7.2, leading=8.6)
    filehdr = ParagraphStyle(
        "FileHdr", fontName="Courier-Bold", fontSize=9, spaceBefore=8, spaceAfter=3,
        textColor=colors.HexColor("#7a1f1f"))

    current_section = {"name": ""}

    def footer(canvas_obj, doc):
        canvas_obj.saveState()
        canvas_obj.setFont("Helvetica", 7)
        text = f"Context Bundle - for new chat - {current_section['name']} - {TODAY}"
        canvas_obj.drawString(0.6 * inch, 0.4 * inch, text)
        canvas_obj.drawRightString(A4[0] - 0.6 * inch, 0.4 * inch, f"p.{doc.page}")
        canvas_obj.restoreState()

    def set_section(name):
        current_section["name"] = name

    def esc(text):
        return (text.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;"))

    story = []

    # ---- Page 1: Title ----
    set_section("title")
    story.append(Spacer(1, 1.5 * inch))
    story.append(Paragraph(
        "Thesis Full Context Profile<br/>Unified Benchmark of Grokking Predictors",
        title_style))
    story.append(Spacer(1, 0.2 * inch))
    story.append(Paragraph(f"Generated: {TODAY}", body))
    story.append(Paragraph("Generated for new chat session", body))
    commit_hash = run_git(["rev-parse", "--short", "HEAD"])
    story.append(Paragraph(f"Git commit: {esc(commit_hash)}", body))
    story.append(Spacer(1, 0.15 * inch))

```

```

story.append(Paragraph("Last 10 commits:", h2))
log = run_git(["log", "--oneline", "-10"])
story.append(Preformatted(esc(log), mono))
story.append(PageBreak())

# ---- Section 1: Project Identity ----
set_section("context.md (project identity)")
story.append(Paragraph("Section 1 – Project Identity", h1))
ctx_text = read_text("context.md") or ""
top120 = "\n".join(ctx_text.splitlines()[:120])
story.append(Preformatted(esc(top120), mono))
story.append(PageBreak())

# ---- Section 2: Current Progress Snapshot ----
set_section("context.md (last 2 sessions)")
story.append(Paragraph("Section 2 – Current Progress Snapshot (last 2 sessions)", h1))
story.append(Paragraph(
    "<b>HIGHLIGHTS:</b> Predictor 1 (L2-Norm) = CLOSED, NEGATIVE. "
    "Predictor 2 (Dropout) = CLOSED, NEGATIVE (k=30, rate=0.5 deviation "
    "from Salah &amp; Yevick justified on compute grounds). "
    "Predictor 3 (Spectral) = CLOSED, NEGATIVE (k_90 3448-&gt;~3350-3400, "
    "no rank collapse; Canatar task-model alignment). "
    "Predictor 4 (AGE) = CLOSED, NEGATIVE (NC1 collapses ~45-&gt;~0.05 but "
    "after grok in 5/5 seeds). "
    "Predictor 5 (HTSR Alpha) = NEXT, not started.",
    body))
story.append(Spacer(1, 0.1 * inch))
last2 = extract_last_n_sessions(ctx_text, n=2)
for chunk_start in range(0, len(last2), 6000):
    story.append(Preformatted(esc(last2[chunk_start:chunk_start + 6000]), mono))
story.append(PageBreak())

# ---- Section 3: Benchmark Config ----
set_section("06_Code/configs/nanda_unified.yaml")
story.append(Paragraph("Section 3 – Benchmark Config", h1))
story.append(Paragraph("=== configs/nanda_unified.yaml ===", filehdr))
cfg_text = read_text("06_Code/configs/nanda_unified.yaml") or "(not found)"
story.append(Preformatted(esc(cfg_text), mono))
story.append(PageBreak())

# ---- Section 4: Latest Results ----
set_section("08_Experiments/results/nanda_unified")
story.append(Paragraph("Section 4 – Latest Results", h1))
story.append(Paragraph("=== results/nanda_unified/aggregate.json ===", filehdr))
story.append(Preformatted(esc(pretty_json("08_Experiments/results/nanda_unified/aggregate.json")), mono))
story.append(Spacer(1, 0.15 * inch))
story.append(Paragraph("Per-seed summary extract:", h2))
rows = build_seed_table()
row_text = "\n".join(
    f"{r[0]:<10} {r[1]:<12} {r[2]:<24} {r[3]:<28} {r[4]:<12}" for r in rows)
story.append(Preformatted(esc(row_text), mono))
story.append(PageBreak())

# ---- Section 5: Master Runner Key Parts ----
set_section("run_nanda_benchmark.py")
story.append(Paragraph("Section 5 – Master Runner Key Parts", h1))
story.append(Paragraph("=== run_nanda_benchmark.py (extract) ===", filehdr))
runner_text = read_text("run_nanda_benchmark.py") or ""
extract = extract_runner_constants_and_registry(runner_text, max_lines=400)
story.append(Preformatted(esc(extract), mono))
story.append(PageBreak())

# ---- Section 6: Predictor Implementations ----
set_section("src/predictors")
story.append(Paragraph("Section 6 – Predictor Implementations (closed predictors)", h1))
for pfile in ("src/predictors/l2_norm.py", "src/predictors/dropout.py"):
    story.append(Paragraph(f"=== {pfile} ===", filehdr))
    text = read_text(pfile) or "(not found)"
    story.append(Preformatted(esc(text), mono))
    story.append(Spacer(1, 0.1 * inch))
story.append(Paragraph("=== src/predictors/spectral.py ===", filehdr))
spectral_text = read_text("src/predictors/spectral.py")
if spectral_text is None:
    story.append(Paragraph("Spectral not yet implemented – stub expected", body))
else:
    story.append(Preformatted(esc(spectral_text), mono))
story.append(PageBreak())

# ---- Section 7: Measurements & Model ----
set_section("measurements/model")
story.append(Paragraph("Section 7 – Measurements & Model", h1))
story.append(Paragraph("=== src/unified_measurements.py (first 200 lines) ===", filehdr))
story.append(Preformatted(esc(read_text("src/unified_measurements.py", max_lines=200) or ""), mono))
story.append(Spacer(1, 0.1 * inch))
story.append(Paragraph("=== src/models/transformer_four_head.py (first 150 lines) ===", filehdr))
story.append(Preformatted(esc(read_text("src/models/transformer_four_head.py", max_lines=150) or ""), mono))

```

```

story.append(PageBreak())

# ---- Section 8: Data Pipeline ----
set_section("src/data/modular_arithmetic.py")
story.append(Paragraph("Section 8 – Data Pipeline", h1))
story.append(Paragraph("=== src/data/modular_arithmetic.py ===", filehdr))
story.append(Preformatted(esc(read_text("src/data/modular_arithmetic.py") or ""), mono))
story.append(PageBreak())

# ---- Section 9: Predictor Evaluation Order & Open Questions ----
set_section("predictor order / open questions")
story.append(Paragraph("Section 9 – Predictor Evaluation Order & Open Questions", h1))
order = [
    ("1. L2 Norm", "CLOSED – negative"),
    ("2. Dropout", "CLOSED – negative (k=30, rate=0.5 deviation justified)"),
    ("3. Spectral", "CLOSED – negative (v4; k_90 3448->~3350-3400, no collapse)"),
    ("4. AGE", "CLOSED – negative (v4; NC1 collapses after grok in 5/5)"),
    ("5. HTSR Alpha", "NEXT – not started"),
    ("6. Correlation Traps", "not started"),
    ("7. Weight-PCA", "not started"),
    ("8. Higher-MI", "not started"),
    ("9. Commutator Defect", "not started"),
]
order_text = "\n".join(f"{{name:<24}} {{status}}" for name, status in order)
story.append(Preformatted(esc(order_text), mono))
story.append(Spacer(1, 0.15 * inch))
story.append(Paragraph(
    "Predictors 1–4 are all CLOSED negative under the same two-criterion "
    "falsification test (signal extremum does not lead grok in 5/5 seeds; "
    "failure direction not seed-consistent). Next open work item: "
    "Predictor 5, HTSR Alpha (Martin & Mahoney weightwatcher heavy-tailed "
    "self-regularisation, per-weight-matrix ESD power-law exponent alpha) – "
    "checkpoint-only plugin pattern, not yet started.", body))

doc = SimpleDocTemplate(
    OUTPUT_PDF, pagesize=A4,
    leftMargin=0.6 * inch, rightMargin=0.6 * inch,
    topMargin=0.6 * inch, bottomMargin=0.6 * inch,
    title="Thesis Full Context Profile",
)
doc.build(story, onFirstPage=footer, onLaterPages=footer)

def build_with_matplotlib_fallback():
    import matplotlib
    matplotlib.use("Agg")
    from matplotlib.backends.backend_pdf import PdfPages
    import matplotlib.pyplot as plt

    sections = []

    commit_hash = run_git(["rev-parse", "--short", "HEAD"])
    log = run_git(["log", "--oneline", "-10"])
    sections.append(("title", "Thesis Full Context Profile – Unified Benchmark of "
        f"Grokking Predictors\nGenerated: {TODAY}\n"
        "Generated for new chat session\n"
        f"Git commit: {commit_hash}\n\nLast 10 commits:\n{log}"))

    ctx_text = read_text("context.md") or ""
    top120 = "\n".join(ctx_text.splitlines()[:120])
    sections.append(("context.md (identity)", "SECTION 1 – PROJECT IDENTITY\n\n" + top120))

    last2 = extract_last_n_sessions(ctx_text, n=2)
    sections.append(("context.md (last 2 sessions)",
        "SECTION 2 – CURRENT PROGRESS SNAPSHOT\n\n"
        "HIGHLIGHTS: Predictor 1 (L2-Norm) = CLOSED, NEGATIVE. "
        "Predictor 2 (Dropout) = CLOSED, NEGATIVE. "
        "Predictor 3 (Spectral) = CLOSED, NEGATIVE (k_90 3448->3500, "
        "no collapse). Predictor 4 (AGE) = CLOSED, NEGATIVE (NC1 "
        "collapses after grok). Predictor 5 (HTSR Alpha) = NEXT.\n\n"
        + last2))

    sections.append(("06_Code/configs/nanda_unified.yaml",
        "SECTION 3 – BENCHMARK CONFIG\n\n=== configs/nanda_unified.yaml ===\n"
        + (read_text("06_Code/configs/nanda_unified.yaml") or "")))

    rows = build_seed_table()
    row_text = "\n".join(
        f"{{r[0]:<10}} {{r[1]:<12}} {{r[2]:<24}} {{r[3]:<28}} {{r[4]:<12}}" for r in rows)
    sections.append(("08_Experiments/results/nanda_unified",
        "SECTION 4 – LATEST RESULTS\n\n=== aggregate.json ===\n"
        + pretty_json("08_Experiments/results/nanda_unified/aggregate.json")
        + "\n\nPer-seed summary extract:\n" + row_text))

    runner_text = read_text("run_nanda_benchmark.py") or ""
    extract = extract_runner_constants_and_registry(runner_text, max_lines=400)

```

```

sections.append(("run_nanda_benchmark.py",
                "SECTION 5 – MASTER RUNNER KEY PARTS\n\n" + extract))

pred_text = "SECTION 6 – PREDICTOR IMPLEMENTATIONS\n\n"
for pfile in ("src/predictors/l2_norm.py", "src/predictors/dropout.py"):
    pred_text += f"=== {pfile} ===\n" + (read_text(pfile) or "(not found)") + "\n\n"
spectral_text = read_text("src/predictors/spectral.py")
pred_text += "=== src/predictors/spectral.py ===\n"
pred_text += spectral_text if spectral_text is not None else "Spectral not yet implemented – stub expected"
sections.append(("src/predictors", pred_text))

meas_text = ("SECTION 7 – MEASUREMENTS & MODEL\n\n"
            "=== src/unified_measurements.py (first 200 lines) ===\n"
            + (read_text("src/unified_measurements.py", max_lines=200) or "")
            + "\n\n=== src/models/transformer_four_head.py (first 150 lines) ===\n"
            + (read_text("src/models/transformer_four_head.py", max_lines=150) or ""))
sections.append(("measurements/model", meas_text))

sections.append(("src/data/modular_arithmetic.py",
                "SECTION 8 – DATA PIPELINE\n\n=== src/data/modular_arithmetic.py ===\n"
                + (read_text("src/data/modular_arithmetic.py") or "")))

order = [
    ("1. L2 Norm", "CLOSED – negative"),
    ("2. Dropout", "CLOSED – negative (k=30, rate=0.5 deviation justified)"),
    ("3. Spectral", "CLOSED – negative (v4; k_90 3448->~3350-3400, no collapse)"),
    ("4. AGE", "CLOSED – negative (v4; NC1 collapses after grok in 5/5)"),
    ("5. HTSR Alpha", "NEXT – not started"),
    ("6. Correlation Traps", "not started"),
    ("7. Weight-PCA", "not started"),
    ("8. Higher-MI", "not started"),
    ("9. Commutator Defect", "not started"),
]
order_text = "\n".join(f"{name}<24} {status}" for name, status in order)
sections.append(("predictor order",
                "SECTION 9 – PREDICTOR EVALUATION ORDER & OPEN QUESTIONS\n\n"
                + order_text
                + "\n\nPredictors 1-4 all CLOSED negative under the same "
                + "two-criterion falsification test. Next open work item: "
                + "Predictor 5, HTSR Alpha (checkpoint-only plugin pattern), "
                + "not yet started."))

with PdfPages(OUTPUT_PDF) as pdf:
    for name, text in sections:
        lines = text.splitlines()
        lines_per_page = 80
        for start in range(0, max(1, len(lines)), lines_per_page):
            chunk = "\n".join(lines[start:start + lines_per_page])
            fig = plt.figure(figsize=(8.27, 11.69)) # A4
            fig.text(0.05, 0.97, chunk, family="monospace", fontsize=6,
                    va="top", ha="left", wrap=True)
            fig.text(0.05, 0.02, f"Context Bundle – for new chat – {name} – {TODAY}",
                    fontsize=6)
            pdf.savefig(fig)
            plt.close(fig)

def main():
    try:
        build_with_reportlab()
        engine = "reportlab"
    except ImportError:
        build_with_matplotlib_fallback()
        engine = "matplotlib fallback"

    size_kb = os.path.getsize(OUTPUT_PDF) / 1024.0

    page_count = "?"
    try:
        from pypdf import PdfReader
        page_count = len(PdfReader(OUTPUT_PDF).pages)
    except Exception:
        try:
            import subprocess as sp
            out = sp.run(["mdls", "-name", "KMDItemNumberOfPages", OUTPUT_PDF],
                        capture_output=True, text=True).stdout
            page_count = out.strip().split("=")[-1].strip()
        except Exception:
            pass

    print(f"Saved: full_context_profile.pdf ({page_count} pages, {size_kb:.1f} KB) [{engine}]")
    print("Includes: context.md (2 latest sessions), aggregate.json, 5 seed summaries, "
          "nanda_unified.yaml, 2 predictors (l2_norm, dropout) + spectral if present, "
          "model, measurements")
    print("Excluded (deliberately, with reason):")
    for path, reason in EXCLUDED_NOTES:

```

```
        print(f" - {path}: {reason}")

if __name__ == "__main__":
    sys.exit(main())
```

7. 06_Code/scripts/compile_python_files.py

```
"""
compile_python_files.py
=====

Compiles EVERY codeable file under the project root into one single PDF
('project_compilation.pdf', written in the project root).

The project root is found from this file's location
(06_Code/scripts/ -> two levels up), so the script works on any machine.

Usage:
python 06_Code/scripts/compile_python_files.py
"""

import textwrap
from datetime import date
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]
OUTPUT_FILE = ROOT / "project_compilation.pdf"

# Codeable = source code, scripts and code-like config. Data, images,
# weights (.npy/.png/.pt), papers (.pdf) and prose (.md) are not code.
CODE_EXTENSIONS = {
    ".py", ".gs", ".js", ".ts", ".sh", ".html", ".css",
    ".yaml", ".yml", ".toml", ".cfg", ".ini", ".ipynb",
}
# Code files with no useful suffix, or a suffix added on top (e.g. *.py.deprecated).
CODE_NAMES = {"requirements.txt"}
CODE_SUFFIX_PAIRS = {"deprecated"}

SKIP_DIRS = {
    "__pycache__", ".git", ".idea", ".vs", ".vscode", ".claude",
    ".venv", ".env", "build", "dist", "node_modules",
    ".pytest_cache", ".mypy_cache", "graphify-out",
}

WRAP_WIDTH = 112
FONT_SIZE = 7

def should_skip(path: Path):
    return any(part in SKIP_DIRS for part in path.relative_to(ROOT).parts)

def is_code_file(path: Path):
    if path == OUTPUT_FILE:
        return False
    if path.name in CODE_NAMES:
        return True
    suffixes = [s.lower() for s in path.suffixes]
    if not suffixes:
        return False
    if suffixes[-1] in CODE_SUFFIX_PAIRS and len(suffixes) > 1:
        return suffixes[-2] in CODE_EXTENSIONS
    return suffixes[-1] in CODE_EXTENSIONS

def collect_files():
    return sorted(
        f for f in ROOT.rglob("*")
        if f.is_file() and not should_skip(f) and is_code_file(f)
    )

def read_code(path: Path):
    try:
        return path.read_text(encoding="utf-8")
    except UnicodeDecodeError:
        return path.read_text(encoding="latin-1")

def wrap_code(code: str):
    """Hard-wrap long lines (PDF has no horizontal scroll), keeping indent."""
    out = []
    for line in code.expandtabs(4).splitlines():
        if len(line) <= WRAP_WIDTH:
            out.append(line)
            continue
        indent = len(line) - len(line.lstrip())
        out.extend(textwrap.wrap(
            line, WRAP_WIDTH, subsequent_indent=" " * (indent + 4),
            break_long_words=True, break_on_hyphens=False,
        ) or [""])
    return out
```

```

return "\n".join(out)

def register_mono_font():
    """Prefer a Unicode monospace font (arrows, Greek letters in comments);
    fall back to built-in Courier, which cannot draw those characters."""
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont

    candidates = [
        "/System/Library/Fonts/Menlo.ttc",
        "/Library/Fonts/DejaVuSansMono.ttf",
        "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
    ]
    for candidate in candidates:
        if Path(candidate).exists():
            try:
                pdfmetrics.registerFont(TTFont("ProjMono", candidate))
                return "ProjMono", True
            except Exception:
                continue
    return "Courier", False

def build_pdf(files):
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import inch
    from reportlab.platypus import (
        PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer,
    )

    mono_name, unicode_ok = register_mono_font()
    styles = getSampleStyleSheet()
    title = ParagraphStyle("T", parent=styles["Title"], fontSize=20)
    body = styles["BodyText"]
    mono = ParagraphStyle("Mono", fontName=mono_name, fontSize=FONT_SIZE,
                          leading=FONT_SIZE + 1.6)
    listing = ParagraphStyle("List", fontName=mono_name, fontSize=8,
                             leading=10.5)
    file_header = ParagraphStyle(
        "FH", fontName="Helvetica-Bold", fontSize=10, spaceBefore=4,
        spaceAfter=6, textColor=colors.HexColor("#7a1f1f"))

    def esc(text):
        text = text.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
        if not unicode_ok:
            text = text.encode("latin-1", "replace").decode("latin-1")
        return text

    def footer(canvas_obj, doc):
        canvas_obj.saveState()
        canvas_obj.setFont("Helvetica", 7)
        canvas_obj.drawString(0.6 * inch, 0.4 * inch,
                              f"Project code compilation - {date.today().isoformat()}")
        canvas_obj.drawRightString(A4[0] - 0.6 * inch, 0.4 * inch, f"p.{doc.page}")
        canvas_obj.restoreState()

    relative = [f.relative_to(ROOT).as_posix() for f in files]

    story = [
        Spacer(1, 1.2 * inch),
        Paragraph("Project Code Compilation<br>Unified Benchmark of Grokking Predictors", title),
        Spacer(1, 0.2 * inch),
        Paragraph(f"Generated: {date.today().isoformat()}", body),
        Paragraph(f"Project root: {esc(str(ROOT))}", body),
        Paragraph(f"Total code files: <b>{len(files)}</b>", body),
        Spacer(1, 0.2 * inch),
        Paragraph("Contents", styles["Heading2"]),
        Preformatted(esc("\n".join(f"{i:>3}. {name}" for i, name in enumerate(relative, 1))), listing),
        PageBreak(),
    ]

    for index, (path, rel) in enumerate(zip(files, relative), 1):
        print(f"Processing: {rel}")
        code = wrap_code(read_code(path)) or "(empty file)"
        story.append(Paragraph(f"{index}. {esc(rel)}", file_header))
        story.append(Preformatted(esc(code), mono))
        story.append(PageBreak())

    doc = SimpleDocTemplate(
        str(OUTPUT_FILE), pagesize=A4,
        leftMargin=0.6 * inch, rightMargin=0.6 * inch,
        topMargin=0.6 * inch, bottomMargin=0.6 * inch,
        title="Project Code Compilation",
    )

```

```

    )
    doc.build(story, onFirstPage=footer, onLaterPages=footer)

def main():
    files = collect_files()
    if not files:
        print("No code files found.")
        return

    try:
        build_pdf(files)
    except ImportError:
        print("reportlab is not installed. Run: pip install reportlab")
        return

    print(f"\nDone! {len(files)} files compiled.")
    print(f"Output saved to:\n{OUTPUT_FILE}")

if __name__ == "__main__":
    main()

```

8. 06_Code/scripts/debug_detection.py

```
import numpy as np

# Load the data
l2_norm_history = np.load("l2_norm_history.npy")
test_acc_history = np.load("test_acc_history.npy")

# Compute rate of decline
rate_of_decline = np.diff(l2_norm_history) * -1

# Find grok
grok_epoch = np.where(np.array(test_acc_history) > 0.9)[0][0]
print(f"Grok epoch: {grok_epoch}\n")

# Test different thresholds with skip_epochs=200
print("Testing thresholds (skip_epochs=200):")
print("Threshold | Detection Epoch | Lead Time")
print("-" * 45)

for threshold in [0.008, 0.010, 0.012, 0.014, 0.016, 0.020]:
    # Find first epoch after skip where rate > threshold
    detection_epoch = None
    for i in range(200, len(rate_of_decline)):
        if rate_of_decline[i] > threshold:
            detection_epoch = i
            break

    if detection_epoch is not None:
        lead_time = grok_epoch - detection_epoch
        print(f"{threshold:.3f} | {detection_epoch:14d} | {lead_time:9d}")
    else:
        print(f"{threshold:.3f} | {'None':14s} | {'N/A':>9s}")

# Show rates around skip boundary and grok
print(f"\nRates around skip boundary (epoch 195-210):")
for i in range(195, min(210, len(rate_of_decline))):
    print(f" rate[{i:4d}] = {rate_of_decline[i]:.6f}")

print(f"\nRates around grok (epoch {grok_epoch-5} to {grok_epoch+5}):")
for i in range(max(0, grok_epoch-5), min(grok_epoch+5, len(rate_of_decline))):
    print(f" rate[{i:4d}] = {rate_of_decline[i]:.6f}")
```

9. 06_Code/scripts/generate_file.py

```
from pathlib import Path

# Path to the file you want to create
file_path = Path("src/predictors/l2_norm.py")

# Create parent directories if they don't exist
file_path.parent.mkdir(parents=True, exist_ok=True)

# Initial file contents
content = '''
L2 Norm Predictor

This module implements the L2 norm predictor.
'''

# Add your implementation here
'''

# Write the file only if it doesn't already exist
if not file_path.exists():
    file_path.write_text(content, encoding="utf-8")
    print(f"Created: {file_path}")
else:
    print(f"File already exists: {file_path}")
```

10. 06_Code/scripts/inherticance_example.py

```
class Parent:
    def __init__(parent, name):
        parent.name = name
        print("Parent class initialized")
        print(f"Name: {parent.name}")

class Child(Parent):
    def __init__(child, parent_name, child_name):
        super().__init__(parent_name)
        child.name = child_name
        print("Child class initialized")
        print(f"Name: {child.name}")

if __name__ == "__main__":
    child_instance1 = Child("Alice", "Bob")
    child_instance2 = Child("Charlie", "David")
```

11. 06_Code/scripts/markPhase1Task1Complete.gs

```
/**
 * markPhase1Task1Complete
 *
 * Marks row 4 ("Git repo + environment setup") as complete:
 * - fills columns A-C green with white text
 * - fills the Gantt bar cells (D onward) with a darker done-green
 * - prepends "✓ " to the task name in column B
 *
 * HOW TO RUN:
 * Extensions → Apps Script → paste → Run → markPhase1Task1Complete
 */

function markPhase1Task1Complete() {
  const ss      = SpreadsheetApp.getActiveSpreadsheet();
  const sheet   = ss.getActiveSheet();
  const numCols = sheet.getLastColumn();

  const TASK_ROW   = 4;           // Row 4 = "Git repo + environment setup"
  const LABEL_COL  = 2;           // Column B = task name
  const GANTT_START = 4;          // Column D = first week column

  const DONE_BG_LABEL = '#2e7d32'; // dark green for label columns (A-C)
  const DONE_BG_BAR   = '#4caf50'; // mid green for Gantt bar cells
  const DONE_FONT     = 'ffffff';

  // — Colour label columns A-C —————
  sheet.getRange(TASK_ROW, 1, 1, 3)
    .setBackground(DONE_BG_LABEL)
    .setFontColor(DONE_FONT);

  // — Colour Gantt bar columns D onward —————
  sheet.getRange(TASK_ROW, GANTT_START, 1, numCols - GANTT_START + 1)
    .setBackground(DONE_BG_BAR)
    .setFontColor(DONE_FONT);

  // — Prepend ✓ to the task label in column B —————
  const labelCell = sheet.getRange(TASK_ROW, LABEL_COL);
  const existing  = String(labelCell.getValue());
  if (!existing.startsWith('✓')) {
    labelCell.setValue('✓ ' + existing);
  }

  SpreadsheetApp.getUi().alert('☐ Phase 1 – Task 1 marked as complete.');
```

12. 06_Code/scripts/md_to_image.py

```
#!/usr/bin/env python3

"""
markdown_to_images.py

Usage:
    python markdown_to_images.py

The script will ask for:
1. Absolute path to the Markdown file
2. Output directory (optional)
"""

import textwrap
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

# -----
# Configuration
# -----

PAGE_WIDTH = 1928
PAGE_HEIGHT = 1928

MARGIN_X = 40
MARGIN_Y = 40

FONT_SIZE = 20
LINE_SPACING = 4

BACKGROUND = "white"
FOREGROUND = "black"

FONT_CANDIDATES = [
    "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
    "/System/Library/Fonts/Menlo.ttc",
    "/System/Library/Fonts/SFNSMono.ttf",
    "C:\\Windows\\Fonts\\consola.ttf",
]

# -----

def load_font():
    for path in FONT_CANDIDATES:
        try:
            return ImageFont.truetype(path, FONT_SIZE)
        except Exception:
            pass
    return ImageFont.load_default()

font = load_font()

def chars_per_line():
    dummy = Image.new("RGB", (100, 100))
    draw = ImageDraw.Draw(dummy)

    bbox = draw.textbbox((0, 0), "M", font=font)
    char_width = bbox[2] - bbox[0]

    usable = PAGE_WIDTH - (2 * MARGIN_X)
    return max(20, usable // char_width)

MAX_CHARS = chars_per_line()

def wrap_markdown(md):
    wrapped = []

    for line in md.splitlines():
        if line.strip() == "":
            wrapped.append("")
            continue

        if line.startswith(" ") or line.startswith("\t"):
            wrapped.append(line)
            continue
        wrapped.extend(
```

```

        textwrap.wrap(
            line,
            width=MAX_CHARS,
            replace_whitespace=False,
            drop_whitespace=False,
        ) or [""]
    )

    return wrapped

def render_page(lines, page_number, outdir):
    image = Image.new("RGB", (PAGE_WIDTH, PAGE_HEIGHT), BACKGROUND)
    draw = ImageDraw.Draw(image)

    bbox = draw.textbbox((0, 0), "Ag", font=font)
    line_height = bbox[3] - bbox[1] + LINE_SPACING

    y = MARGIN_Y

    for line in lines:
        draw.text((MARGIN_X, y), line, fill=FOREGROUND, font=font)
        y += line_height

    image.save(outdir / f"page_{page_number:03}.png")

def main():
    print("=" * 60)
    print(" Markdown → PNG Converter")
    print("=" * 60)

    while True:
        md_path = input("\nEnter absolute path of the Markdown file:\n> ").strip().strip('\'')

        md_file = Path(md_path)

        if md_file.exists() and md_file.is_file():
            break

        print("\n❌ File not found. Please try again.")

    output = input(
        "\nEnter output folder (leave blank for automatic):\n> "
    ).strip().strip('\'')

    if output:
        outdir = Path(output)
    else:
        outdir = md_file.parent / f"{md_file.stem}_images"

    outdir.mkdir(parents=True, exist_ok=True)

    text = md_file.read_text(encoding="utf-8")

    wrapped = wrap_markdown(text)

    dummy = Image.new("RGB", (100, 100))
    draw = ImageDraw.Draw(dummy)

    bbox = draw.textbbox((0, 0), "Ag", font=font)
    line_height = bbox[3] - bbox[1] + LINE_SPACING

    lines_per_page = (PAGE_HEIGHT - (2 * MARGIN_Y)) // line_height

    page = 1

    for i in range(0, len(wrapped), lines_per_page):
        render_page(
            wrapped[i:i + lines_per_page],
            page,
            outdir,
        )
        page += 1

    print(f"\n✅ Done!")
    print(f"Generated {page - 1} image(s).")
    print(f"Output folder:\n{outdir.resolve()}")

if __name__ == "__main__":
    main()

```

13.06_Code/scripts/run_nanda_benchmark.py

```
#!/usr/bin/env python3
"""
run_nanda_benchmark.py
=====
```

Master runner for the Nanda-Unified grokking-predictor benchmark, for the predictors that actually exist so far: ONLY L2-Norm and Dropout.

Every constant comes from configs/nanda_unified.yaml (the single source of truth). Nothing is hardcoded here about the task or the optimiser. The "p=" token id is the modulus from the config (p = 113), never a fixed value. Predictors 3-9 (Spectral, HTSR Alpha, AGE, Weight-PCA, Higher-MI, Commutator Defect, ...) are NOT built yet and are NOT called.

Per seed it:

- sets torch.manual_seed(seed)
- builds the dataloaders for p from the config (full-batch)
- builds TransformerFourHead with the small init (N(0, 0.8/sqrt(d)))
- trains full-batch with AdamW(lr, betas, weight_decay) from the config
- logs every epoch via unified_measurements.PredictorMeasurements: train_acc, test_acc, loss, l2_norm, sum_w2, per-module sum_w2 (5 groups: token_embedding, position_embedding, attention_qkv, mlp, output_head)
- after training: computes the L2-Norm predictor signals (MA crossover, MA-of-MA zero crossing, noise floor) and the Dropout gap sweep, and saves them under output_dir/seed_{seed}/

After all seeds it aggregates: mean grok epoch, per-seed predictor signals, and a post-grok limit-cycle check across the seeds.

Resume: a seed whose output_dir/seed_{seed}/summary.json exists is skipped and its summary reloaded.

Usage:

```
python 06_Code/scripts/run_nanda_benchmark.py --seeds 5 --epochs 40000 \
--output_dir 08_Experiments/results/nanda_unified --config 06_Code/configs/nanda_unified.yaml
```

quick smoke:

```
python 06_Code/scripts/run_nanda_benchmark.py --seeds 1 --epochs 100 \
--output_dir 08_Experiments/results/test_smoke
```

.....

```
import argparse
import json
import math
import os
import sys
import time
```

```
import numpy as np
import torch
import yaml
from torch import nn
from torch.optim import AdamW
```

```
# This file lives at <repo>/06_Code/scripts/, so the repo root is two folders
# up. (Before the 2026-09-17 reorganisation the script sat at the repo root,
# and REPO_ROOT was simply its own folder; that broke after the move.) The
# relative --output_dir / --config defaults below are resolved from here.
REPO_ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
SRC = os.path.join(REPO_ROOT, "06_Code", "src")
if SRC not in sys.path:
    sys.path.insert(0, SRC)
```

```
from data.modular_arithmetic import get_dataloaders # noqa: E402
from models.transformer_four_head import TransformerFourHead # noqa: E402
from predictors.l2_norm import ( # noqa: E402
    compute_l2_norm,
    compute_sum_of_squared_weights,
    compute_per_module_sum_of_squared_weights,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from predictors.dropout import ( # noqa: E402
    compute_dropout_gap_multi_rate,
    compute_dropout_variance,
)
from predictors.spectral import compute_spectral_metrics_for_checkpoint # noqa: E402
from predictors.age import compute_age_metrics_for_checkpoint # noqa: E402
from unified_measurements import PredictorMeasurements # noqa: E402
# -----
```

```

# Science constants live in configs/nanda_unified.yaml (the single source of
# truth; every value has its source written next to it there). They are copied
# into the module-level names below ONCE, by apply_config_constants(cfg) at the
# start of main(), so every helper (including the checkpoint-predictor plugins,
# whose signature is shared) can read them without a cfg argument.
GROK_THRESHOLDS = None           # e.g. [0.9, 0.95, 0.99]; grok epoch reported for each
GROK_ACC_THRESHOLD = None        # primary threshold = GROK_THRESHOLDS[0]
LOG_LINES_PER_RUN = None         # console lines per run; log_every = epochs // this
LIMIT_CYCLE_SETTLE_FRACTION = None
LIMIT_CYCLE_STD_ABOVE = None
DROPOUT_RATES = None             # multi-rate sweep
DROPOUT_VARIANCE_RATE = None
DROPOUT_VARIANCE_N_SAMPLES = None
L2_FAST_WINDOW = None
L2_SLOW_WINDOW = None
L2_MA_OF_MA_FAST_WINDOW = None
L2_SKIP_EPOCHS = None
L2_QUIET_EPOCH_CUTOFF = None

# Dropout-Variance predictor (Salah & Yevick, arXiv:2507.11645): at a
# checkpoint, run n_samples stochastic forward passes with dropout active
# and look at the variance of test accuracy across those passes (see
# src/predictors/dropout.py's compute_dropout_variance docstring).
#
# Now paper-faithful on the number of passes: n_samples = 100 (the paper's
# own value; the earlier 30 was a compute saving). Remaining deviations:
# - rate = 0.5 here, the paper's Figure 1 sweep used 0.3. Their own
# Figure 2 (Dropout Robustness Curve) shows 0.5 separates pre- and
# post-grok best, so the primary signal uses 0.5.
# - the paper measures at (implicitly) every logged epoch; here every
# predictor, this one included, is evaluated on the same EVAL_EVERY grid
# (see the checkpoint-schedule block below). 40000 epochs x 100 passes at
# every epoch is not affordable on MPS.
# Cost: about 3x the old n_samples=30 setting per checkpoint (was ~1.8 s).

# -----
# Checkpoint schedule: SAVING is decoupled from EVALUATING.
# (Updates 2026-09-25; the full derivations, the measured transition widths and
# the design criteria used to compare spacings are in context.md, appended
# sections "Switch from 24 log-spaced to save-every-100 (derived)" and
# "Shift from 100 to 50 to support 250 grid (GCD justification)". They are
# thesis analysis criteria, so they are deliberately NOT repeated as code
# constants or checks here.)
#
# WHY SAVE EVERY 50: the evaluation grids we want to test are 100, 200 and
# 250, and gcd(100, 200, 250) = 50. A grid can only be evaluated if every one
# of its epochs was saved, so SAVE_EVERY must divide all of them. 50 does; 100
# does not divide 250 (saved multiples of 100 that are also multiples of 250
# are only multiples of 500, i.e. a "250 grid" would silently alias to a 500
# grid). Save densely once, evaluate on any coarser grid later without
# retraining.
#
# SAVE   : a checkpoint every SAVE_EVERY = 50 epochs, PLUS the final epoch:
#           range(0, 40000, 50) = 800 points (0..39950) + epoch 39999 = 801.
#           Storage: ~0.85 MB per checkpoint & ~680 MB per seed, ~3.4 GB for 5 seeds.
# EVAL   : every predictor evaluates only the saved epochs with
#           epoch % EVAL_EVERY == 0, PLUS the final saved epoch (the fully
#           trained model; kept so results stay comparable with the earlier
#           runs, whose grids always ended on epoch 39999). Saving is nearly
#           free, evaluating is not: Spectral ~9.3 s and Dropout-Variance
#           ~1.8 s per checkpoint on this machine (~11 s together), so the
#           default 100 grid (401 points) costs about 1.2 h per seed for those
#           two, the 200 grid (201 points) ~0.6 h, the 250 grid (161 points)
#           ~0.5 h, and the full 50 grid (801 points) ~2.5 h.
# --eval_every accepts 50, 100, 200 or 250 (any multiple of SAVE_EVERY);
# select_eval_epochs() raises if the requested grid is not on disk.
# EVAL_EVERY = 0 means "use every saved checkpoint as-is" (needed to
# re-run predictors on the legacy 24-point directories).
TOTAL_STEPS = 40000              # default --epochs
DESIRED_EVAL_GRIDS = [100, 200, 250] # first = default, the rest = fallbacks
# SAVE_EVERY derived as GCD of desired eval grids, so every eval grid is subset of saved checkpoints
SAVE_EVERY = math.gcd(*DESIRED_EVAL_GRIDS) # gives 50
EVAL_EVERY = DESIRED_EVAL_GRIDS[0]
EVAL_EVERY_FALLBACK = DESIRED_EVAL_GRIDS[1]
EVAL_EVERY_FALLBACK_COARSE = DESIRED_EVAL_GRIDS[2]

# LEGACY – for Archive comparison only, not used in unified benchmark.
# Schedule used up to benchmark v4 (tag benchmark-v4-4predictors): 24
# log-spaced epochs, chosen as a compute budget for Dropout-Variance, with no
# scientific justification for the count (see context.md, 2026-09-25). Kept,
# NOT deleted, so the old grid can be reproduced for ablation comparison, e.g.
# via --eval_every 0 on a legacy results directory. select_eval_epochs() and
# the main flow never read it.
OLD_24_LOG_SPACED_CHECKPOINTS = [
    0, 1, 2, 3, 5, 9, 15, 24, 39, 62, 99, 158, 251, 398, 632, 1002, 1589,
    2520, 3995, 6333, 10040, 15917, 25232, 39999,

```

```

]

# Names understood by --predictors / --overwrite, and the summary.json key
# each one's presence is checked against (see get_done_predictors below).
PREDICTOR_SUMMARY_KEY = {
    "l2": "l2_predictor",
    "dropout_gap": "dropout_final_gap_by_rate",
    "dropout_variance": "dropout_variance_predictor",
    "spectral": "spectral_predictor",
    "age": "age_predictor",
}
ALL_PREDICTORS = list(PREDICTOR_SUMMARY_KEY.keys())

def save_checkpoint_schedule(total_epochs, save_every=SAVE_EVERY):
    """Epoch indices (0-based, matching the training loop's `epoch`) at which
    a model checkpoint is SAVED: 0, save_every, 2*save_every, ... below
    total_epochs, plus the final epoch total_epochs - 1. For 40000 epochs and
    save_every=50 that is 800 + 1 = 801 points."""
    if total_epochs <= 1:
        return [0]
    schedule = list(range(0, total_epochs, save_every))
    if schedule[-1] != total_epochs - 1:
        schedule.append(total_epochs - 1)
    return schedule

SAVE_CHECKPOINTS = save_checkpoint_schedule(TOTAL_STEPS)

def select_eval_epochs(saved_epochs, eval_every=EVAL_EVERY):
    """Subsample the evaluation grid from the SAVED checkpoint epochs.
    Keeps saved epochs with epoch % eval_every == 0, plus the final saved
    epoch. eval_every <= 0 returns every saved epoch unchanged (legacy
    directories). Raises if the saved files cannot supply the requested
    grid, instead of silently evaluating on a coarser one."""
    saved = sorted(saved_epochs)
    if not saved:
        return []
    if eval_every <= 0:
        return saved
    saved_set = set(saved)
    missing = [e for e in range(0, saved[-1] + 1, eval_every) if e not in saved_set]
    if missing:
        raise ValueError(
            f"eval_every={eval_every} needs saved checkpoints at epochs such as "
            f"{missing[:5]}, which are not on disk (saved grid spacing is not a "
            f"divisor of {eval_every}). Use a multiple of the save spacing "
            f"({SAVE_EVERY}; e.g. 100, {EVAL_EVERY_FALLBACK} or {EVAL_EVERY_FALLBACK_COARSE}), "
            f"or --eval_every 0 to use "
            f"all saved checkpoints as they are.")
    chosen = [e for e in saved if e % eval_every == 0]
    if chosen[-1] != saved[-1]:
        chosen.append(saved[-1])
    return chosen

def dropout_variance_checkpoint_schedule(total_epochs, num_points=len(OLD_24_LOG_SPACED_CHECKPOINTS)):
    """
    LEGACY – for Archive comparison only, not used in unified benchmark
    (24 log-spaced points; see OLD_24_LOG_SPACED_CHECKPOINTS and the
    checkpoint-schedule block above). Nothing in the main flow calls it.
    Log-uniform-spaced epoch indices for the dropout-variance checkpoints,
    same spirit as l2_norm.resample_to_log_uniform_grid's log-epoch grid –
    equal VISUAL spacing on a log-x plot, so early training (where things
    change fast) gets proportionally more checkpoints than the long flat
    tail. Returns a sorted list of 0-based epoch indices (matching the
    training loop's `epoch` variable, range(total_epochs)), always
    including epoch 0 and the final epoch (total_epochs - 1).
    """
    if total_epochs <= 1:
        return [0]
    log_grid = np.linspace(0, np.log10(total_epochs), num_points)
    raw = np.round(10 ** log_grid).astype(int) - 1 # 1..total_epochs -> 0..total_epochs-1
    raw = np.clip(raw, 0, total_epochs - 1)
    checkpoints = sorted(set(int(v) for v in raw))
    if checkpoints[0] != 0:
        checkpoints.insert(0, 0)
    if checkpoints[-1] != total_epochs - 1:
        checkpoints.append(total_epochs - 1)
    return checkpoints

def _num_or_none(value):
    return None if value is None else float(value)
def load_config(path):

```

```

"""Read configs/nanda_unified.yaml -> flat dict of the constants used here."""
with open(path) as handle:
    cfg = yaml.safe_load(handle)

d_model = cfg["architecture"]["d_model"]
init_std = cfg["init"]["init_std"]
if init_std is None:
    init_std = 0.8 / (d_model ** 0.5)

parsed = {
    "modulus": cfg["task"]["modulus"],
    "vocab_size": cfg["task"]["vocab_size"],
    "train_fraction": cfg["task"]["train_fraction"],
    "d_model": d_model,
    "num_heads": cfg["architecture"]["num_heads"],
    "init_std": float(init_std),
    "lr": float(cfg["optimizer"]["lr"]),
    "weight_decay": float(cfg["optimizer"]["weight_decay"]),
    "betas": tuple(cfg["optimizer"]["betas"]),
    "epochs_default": int(cfg["training"]["epochs"]),
    # evaluation / predictor constants (each has its source in the yaml)
    "grok_thresholds": [float(t) for t in cfg["evaluation"]["grok_thresholds"]],
    "log_lines_per_run": int(cfg["evaluation"]["log_lines_per_run"]),
    "limit_cycle_settle_fraction": float(cfg["evaluation"]["limit_cycle"]["settle_fraction"]),
    "limit_cycle_std_above": float(cfg["evaluation"]["limit_cycle"]["std_above"]),
    "dropout_rates": [float(r) for r in cfg["dropout"]["rates"]],
    "dropout_variance_rate": float(cfg["dropout"]["variance_rate"]),
    "dropout_variance_n_samples": int(cfg["dropout"]["n_samples"]),
    "l2_fast_window": int(cfg["l2_norm"]["fast_window"]),
    "l2_slow_window": int(cfg["l2_norm"]["slow_window"]),
    "l2_ma_of_ma_fast_window": int(cfg["l2_norm"]["ma_of_ma_fast_window"]),
    "l2_skip_epochs": int(cfg["l2_norm"]["skip_epochs"]),
    "l2_quiet_epoch_cutoff": int(cfg["l2_norm"]["quiet_epoch_cutoff"]),
}
# Fail loud if the config is internally inconsistent.
assert parsed["vocab_size"] == parsed["modulus"] + 1, \
    "config: vocab_size must be modulus + 1"
assert parsed["d_model"] % parsed["num_heads"] == 0, \
    "config: d_model must be divisible by num_heads"
assert parsed["grok_thresholds"], "config: evaluation.grok_thresholds must not be empty"
return parsed

def apply_config_constants(cfg):
    """Copy the yaml-derived constants into the module-level names declared
    near the top of this file (see the comment there for why). Called once,
    at the start of main(), before any training or recompute."""
    global GROK_THRESHOLDS, GROK_ACC_THRESHOLD, LOG_LINES_PER_RUN
    global LIMIT_CYCLE_SETTLE_FRACTION, LIMIT_CYCLE_STD_ABOVE
    global DROPOUT_RATES, DROPOUT_VARIANCE_RATE, DROPOUT_VARIANCE_N_SAMPLES
    global L2_FAST_WINDOW, L2_SLOW_WINDOW, L2_MA_OF_MA_FAST_WINDOW
    global L2_SKIP_EPOCHS, L2_QUIET_EPOCH_CUTOFF
    GROK_THRESHOLDS = list(cfg["grok_thresholds"])
    GROK_ACC_THRESHOLD = GROK_THRESHOLDS[0] # primary
    LOG_LINES_PER_RUN = cfg["log_lines_per_run"]
    LIMIT_CYCLE_SETTLE_FRACTION = cfg["limit_cycle_settle_fraction"]
    LIMIT_CYCLE_STD_ABOVE = cfg["limit_cycle_std_above"]
    DROPOUT_RATES = list(cfg["dropout_rates"])
    DROPOUT_VARIANCE_RATE = cfg["dropout_variance_rate"]
    DROPOUT_VARIANCE_N_SAMPLES = cfg["dropout_variance_n_samples"]
    L2_FAST_WINDOW = cfg["l2_fast_window"]
    L2_SLOW_WINDOW = cfg["l2_slow_window"]
    L2_MA_OF_MA_FAST_WINDOW = cfg["l2_ma_of_ma_fast_window"]
    L2_SKIP_EPOCHS = cfg["l2_skip_epochs"]
    L2_QUIET_EPOCH_CUTOFF = cfg["l2_quiet_epoch_cutoff"]

def pick_device():
    return torch.device("mps" if torch.backends.mps.is_available() else "cpu")

def seed_dir_for(output_dir, seed):
    return os.path.join(output_dir, f"seed_{seed}")

def checkpoints_dir_for(output_dir, seed):
    return os.path.join(seed_dir_for(output_dir, seed), "checkpoints")

def summary_path_for(output_dir, seed):
    return os.path.join(seed_dir_for(output_dir, seed), "summary.json")

def load_summary(output_dir, seed):
    path = summary_path_for(output_dir, seed)
    if not os.path.isfile(path):

```

```

        return None
    with open(path) as handle:
        return json.load(handle)

def get_done_predictors(output_dir, seed):
    """Which predictors already have a real result in this seed's
    summary.json, keyed by PREDICTOR_SUMMARY_KEY -> returns a set of
    predictor names (e.g. {"l2", "dropout_gap"}). Empty set if
    summary.json does not exist yet, or exists but is missing a key
    entirely (e.g. an old run from before dropout_variance existed)."""
    summary = load_summary(output_dir, seed)
    if summary is None:
        return set()
    done = set()
    for name, key in PREDICTOR_SUMMARY_KEY.items():
        if summary.get(key) is not None:
            done.add(name)
    return done

def is_predictor_done(output_dir, seed, predictor):
    return predictor in get_done_predictors(output_dir, seed)

def has_saved_checkpoints(output_dir, seed):
    ckpt_dir = checkpoints_dir_for(output_dir, seed)
    if not os.path.isdir(ckpt_dir):
        return False
    return any(name.endswith(".pt") for name in os.listdir(ckpt_dir))

def grok_epoch_from(test_acc_history, threshold=None):
    """First epoch at which test accuracy exceeds `threshold` (default: the
    primary threshold, GROK_THRESHOLDS[0]); None if it never does."""
    if threshold is None:
        threshold = GROK_ACC_THRESHOLD
    arr = np.asarray(test_acc_history, dtype=float)
    hits = np.where(arr > threshold)[0]
    return int(hits[0]) if len(hits) else None

def grok_epochs_by_threshold(test_acc_history):
    """{"0.9": epoch, "0.95": epoch, "0.99": epoch} for every threshold in
    GROK_THRESHOLDS (None where a threshold is never reached)."""
    return {str(t): grok_epoch_from(test_acc_history, t) for t in GROK_THRESHOLDS}

def limit_cycle_check(test_acc_history, grok_epoch):
    """Detect the post-grok test-accuracy oscillation seen in the p=113
    run_1 (test acc swinging ~0.5-1.0 for the rest of training). Looks at
    the test-acc tail starting settle_epochs after grok, where settle_epochs
    is LIMIT_CYCLE_SETTLE_FRACTION of the run length (5% = 2000 of 40000
    epochs), so it scales with the run instead of being a fixed number. A
    "dip" is a fall below the primary grok threshold."""
    arr = np.asarray(test_acc_history, dtype=float)
    if grok_epoch is None:
        return {"applicable": False, "limit_cycle": False,
                "reason": "no grok"}
    settle_epochs = int(LIMIT_CYCLE_SETTLE_FRACTION * len(arr))
    start = grok_epoch + settle_epochs
    if start >= len(arr) - 10:
        return {"applicable": False, "limit_cycle": False,
                "reason": "not enough epochs after grok"}
    tail = arr[start:]
    post_min = float(tail.min())
    post_std = float(tail.std())
    n_dips = int(np.sum(tail < GROK_ACC_THRESHOLD))
    is_limit_cycle = bool((post_min < GROK_ACC_THRESHOLD) and (post_std > LIMIT_CYCLE_STD_ABOVE))
    return {
        "applicable": True,
        "window_start_epoch": int(start),
        "post_grok_min": post_min,
        "post_grok_std": post_std,
        "post_grok_final": float(tail[-1]),
        "dips_key": n_dips,
        "limit_cycle": is_limit_cycle,
    }

def dips_key():
    """Key under which limit_cycle_check stores its dip count. With the
    primary grok threshold 0.9 this is 'epochs_below_0.9_post_grok', the same
    key earlier summaries used."""
    return f"epochs_below_{GROK_ACC_THRESHOLD}_post_grok"

def train_one_seed(seed, args, cfg, device, predictors_to_compute, old_summary,
```

```

        save_checkpoints=True):
    """Train seed from scratch. predictors_to_compute (a set, subset of
    ALL_PREDICTORS) controls which expensive per-predictor work actually
    runs this call; a predictor NOT in that set falls back to its block
    in old_summary (if any) in the final summary.json, so a predictor
    that was already done is never silently dropped or recomputed just
    because a different predictor triggered this retrain. l2_norm_history
    itself is always collected during training regardless (needed for the
    training-data plots either way, and is cheap), but the L2-predictor
    *signal* block still only overwrites old_summary's when "l2" is
    actually requested."""
    out_dir = seed_dir_for(args.output_dir, seed)
    os.makedirs(out_dir, exist_ok=True)
    ckpt_dir = checkpoints_dir_for(args.output_dir, seed)
    if save_checkpoints:
        os.makedirs(ckpt_dir, exist_ok=True)
    compute_dropout_gap = "dropout_gap" in predictors_to_compute
    compute_dv = "dropout_variance" in predictors_to_compute
    compute_l2 = "l2" in predictors_to_compute

    # (a) deterministic per-seed RNG
    torch.manual_seed(seed)
    np.random.seed(seed)

    p = cfg["modulus"]
    batch_size = int(cfg["train_fraction"] * p * p) # full-batch

    # (b) data - "=" token id must equal the config modulus p, not any
    # value carried over from the old p=<old prime> code path
    train_loader, test_loader = get_data_loaders(number=p, batch_size=batch_size)
    probe_x, _ = next(iter(train_loader))
    eq_token_ids = {int(v) for v in probe_x[:, 2].tolist()}
    assert eq_token_ids == {p}, \
        f"=' token ids {sorted(eq_token_ids)} != {{p}} (config modulus)"

    # (c) model - small-init _init_weights (init_std = 0.8/sqrt(d_model))
    model = TransformerFourHead(
        vocab_size=p + 1,
        d_model=cfg["d_model"],
        num_heads=cfg["num_heads"],
        init_std=cfg["init_std"],
    ).to(device)

    # (d) optimiser - AdamW(lr, betas, weight_decay) from the config
    optimizer = AdamW(
        model.parameters(),
        lr=cfg["lr"],
        betas=cfg["betas"],
        weight_decay=cfg["weight_decay"],
    )
    loss_fn = nn.CrossEntropyLoss()

    measurements = PredictorMeasurements(out_dir, model_type="four_head")
    np.save(os.path.join(out_dir, "seed.npy"), np.array([seed]))

    train_acc_history, test_acc_history, loss_history = [], [], []
    l2_norm_history, sum_w2_history, per_module_sum_w2_history = [], [], []

    # Checkpoint SAVE grid (every SAVE_EVERY epochs + final epoch) and the
    # EVAL grid (subsamped from it, every args.eval_every epochs + final
    # epoch) are fixed up front. Saving and evaluating are decoupled: the
    # Dropout-Variance measurement (which needs live weights) runs only on
    # the eval grid, but every save-grid checkpoint is written to disk so
    # any other predictor can later be evaluated on any coarser grid
    # without retraining.
    save_checkpoint_set = set(save_checkpoint_schedule(args.epochs))
    dv_checkpoint_schedule = select_eval_epochs(
        sorted(save_checkpoint_set), args.eval_every)
    dv_checkpoint_set = set(dv_checkpoint_schedule)
    # Console cadence derived from the run length: LOG_LINES_PER_RUN lines
    # per run (40 lines = every 1000 epochs for the default 40000).
    log_every = max(1, args.epochs // LOG_LINES_PER_RUN)
    dropout_variance_checkpoints, dropout_variance_history = [], []
    dropout_variance_mean_acc_history = []
    dropout_drc_snapshots = {}

    started = time.time()
    last_log = started
    for epoch in range(args.epochs):
        # ---- train pass (full-batch: one iteration) ----
        model.train()
        train_correct = train_total = 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            logits = model.forward(x)[:, 2, :]
            loss = loss_fn(logits, y)

```

```

optimizer.zero_grad()
loss.backward()
optimizer.step()
train_correct += (logits.argmax(dim=1) == y).sum().item()
train_total += len(y)

# ---- test pass ----
model.eval()
test_correct = test_total = 0
with torch.no_grad():
    for x_t, y_t in test_loader:
        x_t, y_t = x_t.to(device), y_t.to(device)
        pred_t = model.forward(x_t)[: , 2, :].argmax(dim=1)
        test_correct += (pred_t == y_t).sum().item()
        test_total += len(y_t)

# (e) per-epoch logging quantities
train_acc_history.append(train_correct / train_total)
test_acc_history.append(test_correct / test_total)
loss_history.append(loss.item())
l2_norm_history.append(compute_l2_norm(model))
sum_w2_history.append(compute_sum_of_squared_weights(model))
per_module_sum_w2_history.append(
    compute_per_module_sum_of_squared_weights(model))

# ---- checkpoint save + Dropout-Variance predictor (post-epoch, model frozen) ----
# Model-weight checkpointing (SAVE grid) is independent of whether the
# Dropout-Variance predictor is being computed THIS run – other
# predictors (Spectral, AGE, HTSR, ...) reuse these .pt files
# without retraining, as long as save_checkpoints is on.
if save_checkpoints and epoch in save_checkpoint_set:
    torch.save(model.state_dict(),
               os.path.join(ckpt_dir, f"model_epoch_{epoch}.pt"))

# Dropout-Variance evaluation (EVAL grid, a subset of the SAVE grid).
if compute_dv and epoch in dv_checkpoint_set:
    mean_acc, variance = compute_dropout_variance(
        model, test_loader, n_samples=DROPOUT_VARIANCE_N_SAMPLES,
        dropout_rate=DROPOUT_VARIANCE_RATE, device=device)
    dropout_variance_checkpoints.append(epoch)
    dropout_variance_history.append(variance)
    dropout_variance_mean_acc_history.append(mean_acc)

# 5-rate Dropout Robustness Curve snapshot on the SAME eval grid
# (no separate, smaller subset).
drc_results = compute_dropout_gap_multi_rate(model, test_loader, DROPOUT_RATES)
dropout_drc_snapshots[epoch] = {
    str(r): drc_results[r]["train_accuracy"] for r in DROPOUT_RATES}

if epoch % log_every == 0 or epoch == args.epochs - 1:
    now = time.time()
    elapsed = now - started          # total wall time this seed
    since_last = now - last_log      # wall time for the last log_every block
    last_log = now
    print(f"[seed {seed}] epoch {epoch:>6}/{args.epochs} "
          f"train_acc={train_acc_history[-1]:.4f} "
          f"test_acc={test_acc_history[-1]:.4f} "
          f"sum_w2={sum_w2_history[-1]:.1f} "
          f"elapsed={elapsed:10.3f}s "
          f"d[log_every]={since_last:8.3f}s", flush=True)

# (f) save the per-epoch histories
measurements.save_training_data(train_acc_history, test_acc_history, loss_history)

# grok epochs only need test_acc_history – compute them once here so both
# the Dropout-Variance signal below and the final summary can use them.
# grok_epoch is the primary threshold (GROK_THRESHOLDS[0]); the epochs for
# every threshold are also stored, so no conclusion rests on one cut.
grok_epochs_all = grok_epochs_by_threshold(test_acc_history)
grok_epoch = grok_epoch_from(test_acc_history)

# (g) L2-Norm predictor signals (post-training)
epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(
    l2_norm_history, fast_window=L2_FAST_WINDOW, slow_window=L2_SLOW_WINDOW)
ma_crossover_epoch = detect_ma_crossover(
    epoch_grid, fast_ma, slow_ma, skip_epochs=L2_SKIP_EPOCHS)
fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(
    slow_ma, fast_window=L2_MA_OF_MA_FAST_WINDOW)
noise_floor = compute_noise_floor(
    ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=L2_QUIET_EPOCH_CUTOFF)
ma_of_ma_zero_crossing_epoch = detect_ma_of_ma_zero_crossing(
    epoch_grid, ma_of_ma_diff, skip_epochs=L2_SKIP_EPOCHS)

measurements.save_l2_norm_data(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, ma_crossover_epoch,

```

```

        sum_w2_history=sum_w2_history,
        per_module_sum_w2_history=per_module_sum_w2_history,
    )

l2_signals = {
    "ma_crossover_epoch": _num_or_none(ma_crossover_epoch),
    "ma_of_ma_zero_crossing_epoch": _num_or_none(ma_of_ma_zero_crossing_epoch),
    "noise_floor": float(noise_floor),
    "windows": {
        "fast": L2_FAST_WINDOW, "slow": L2_SLOW_WINDOW,
        "ma_of_ma_fast": L2_MA_OF_MA_FAST_WINDOW,
        "skip_epochs": L2_SKIP_EPOCHS,
        "quiet_epoch_cutoff": L2_QUIET_EPOCH_CUTOFF,
    },
}
with open(os.path.join(measurements.l2_norm_dir,
                       "l2_predictor_signals.json"), "w") as handle:
    json.dump(l2_signals, handle, indent=2)

# (g) Dropout predictor – one multi-rate sweep on the final model
old_dropout_gap_block = (old_summary or {}).get("dropout_final_gap_by_rate")
if compute_dropout_gap:
    dropout_results = compute_dropout_gap_multi_rate(model, test_loader, DROPOUT_RATES)
    model.train() # compute_dropout_gap_multi_rate leaves the model in eval()

    final_epoch = args.epochs
    measurements.save_dropout_data(
        dropout_gap_epochs=[final_epoch],
        dropout_gap_history_by_rate={
            r: [dropout_results[r]["dropout_gap"]] for r in DROPOUT_RATES},
        dropout_train_acc_by_rate={
            r: [dropout_results[r]["train_accuracy"]] for r in DROPOUT_RATES},
        dropout_eval_acc_by_rate={
            r: [dropout_results[r]["eval_accuracy"]] for r in DROPOUT_RATES},
        dropout_rates=DROPOUT_RATES,
    )
    dropout_json = {
        str(r): {k: float(v) for k, v in dropout_results[r].items()}
        for r in DROPOUT_RATES
    }
    with open(os.path.join(measurements.dropout_dir,
                           "dropout_gap_final.json"), "w") as handle:
        json.dump(dropout_json, handle, indent=2)
    dropout_gap_block = {
        str(r): float(dropout_results[r]["dropout_gap"]) for r in DROPOUT_RATES}
else:
    # not requested this call –> carry the old result forward unchanged
    dropout_gap_block = old_dropout_gap_block

# (h) Dropout-Variance predictor – per-checkpoint arrays + signal
old_dv_block = (old_summary or {}).get("dropout_variance_predictor")
if compute_dv:
    np.save(os.path.join(measurements.dropout_dir, "dropout_variance_checkpoints.npy"),
            np.array(dropout_variance_checkpoints, dtype=int))
    np.save(os.path.join(measurements.dropout_dir, "dropout_variance_history.npy"),
            np.array(dropout_variance_history, dtype=float))
    np.save(os.path.join(measurements.dropout_dir, "dropout_variance_mean_acc_history.npy"),
            np.array(dropout_variance_mean_acc_history, dtype=float))
    with open(os.path.join(measurements.dropout_dir,
                           "dropout_drc_snapshots.json"), "w") as handle:
        json.dump({str(e): v for e, v in dropout_drc_snapshots.items()}, handle, indent=2)

    peak_idx = int(np.argmax(dropout_variance_history))
    variance_peak_epoch = dropout_variance_checkpoints[peak_idx]
    dropout_variance_block = {
        "variance_peak_epoch": variance_peak_epoch,
        "variance_peak_value": float(dropout_variance_history[peak_idx]),
        "grok_epoch": grok_epoch,
        "peak_to_grok_ratio": (
            float(variance_peak_epoch) / grok_epoch if grok_epoch else None),
        "rate": DROPOUT_VARIANCE_RATE,
        "n_samples": DROPOUT_VARIANCE_N_SAMPLES,
        "num_checkpoints": len(dropout_variance_checkpoints),
    }
    with open(os.path.join(measurements.dropout_dir,
                           "dropout_variance_signal.json"), "w") as handle:
        json.dump(dropout_variance_block, handle, indent=2)
else:
    dropout_variance_block = old_dv_block

# L2-predictor block: reuse the old one unless "l2" was requested.
old_l2_block = (old_summary or {}).get("l2_predictor")
l2_block = l2_signals if compute_l2 else (old_l2_block or l2_signals)

# Spectral is checkpoint-only (see CHECKPOINT_PREDICTOR_FUNCS below) –
# train_one_seed never computes it live, it only carries forward

```

```

# whatever recompute_from_checkpoints already wrote for this seed.
spectral_block = (old_summary or {}).get("spectral_predictor")

# AGE is checkpoint-only too (CHECKPOINT_PREDICTOR_FUNCS["age"]) -
# train_one_seed never computes it live, it only carries forward
# whatever recompute_from_checkpoints already wrote for this seed.
age_block = (old_summary or {}).get("age_predictor")

# per-seed summary + resume sentinel. Old blocks for predictors NOT
# recomputed this call are carried forward, never dropped.
summary = {
    "seed": seed,
    "epochs": args.epochs,
    "modulus": p,
    "grok_epoch": grok_epoch,
    "grok_epochs_by_threshold": grok_epochs_all,
    "final_train_acc": float(train_acc_history[-1]),
    "final_test_acc": float(test_acc_history[-1]),
    "l2_norm_init": float(l2_norm_history[0]),
    "l2_norm_final": float(l2_norm_history[-1]),
    "sum_w2_init": float(sum_w2_history[0]),
    "sum_w2_final": float(sum_w2_history[-1]),
    "token_embedding_share_init": float(
        per_module_sum_w2_history[0]["token_embedding"] / sum_w2_history[0]),
    "l2_predictor": l2_block,
    "dropout_final_gap_by_rate": dropout_gap_block,
    "dropout_variance_predictor": dropout_variance_block,
    "spectral_predictor": spectral_block,
    "age_predictor": age_block,
    "limit_cycle_check": limit_cycle_check(test_acc_history, grok_epoch),
    "wall_time_sec": round(time.time() - started, 1),
}
with open(os.path.join(out_dir, "summary.json"), "w") as handle:
    json.dump(summary, handle, indent=2)

print(f"[seed {seed}] done in {summary['wall_time_sec']}s "
      f"grok_epoch={grok_epoch} "
      f"final_test_acc={summary['final_test_acc']:.4f}", flush=True)
return summary

```

```

def _checkpoint_predictor_dropout_variance(model, test_loader, ckpt_dir, ckpt_epochs,
                                          measurements, grok_epoch, device):
    """CHECKPOINT_PREDICTOR_FUNCS["dropout_variance"] - loads each saved
    checkpoint into `model` in turn and rebuilds the same variance-peak
    signal that train_one_seed computes live during training. Returns the
    dropout_variance_predictor block; also re-writes the same .npz/.json
    files train_one_seed writes, so downstream (plot_nanda_results.py)
    cannot tell a live-trained result from a checkpoint-recomputed one.

```

This is the reference shape for any FUTURE checkpoint-only predictor (e.g. Spectral): take (model, test_loader, ckpt_dir, ckpt_epochs, measurements, grok_epoch, device), load each checkpoint's state_dict() into `model` yourself, compute your signal, save your own files under measurements.<predictor>_dir, and return one block dict - then register the function in CHECKPOINT_PREDICTOR_FUNCS below and add the predictor's name to PREDICTOR_SUMMARY_KEY / ALL_PREDICTORS / CHECKPOINT_ONLY_PREDICTORS. No other part of this file needs to change.

```

dropout_variance_checkpoints, dropout_variance_history = [], []
dropout_variance_mean_acc_history = []
dropout_drc_snapshots = {}

for epoch in ckpt_epochs:
    state = torch.load(os.path.join(ckpt_dir, f"model_epoch_{epoch}.pt"), map_location=device)
    model.load_state_dict(state)
    mean_acc, variance = compute_dropout_variance(
        model, test_loader, n_samples=DROPOUT_VARIANCE_N_SAMPLES,
        dropout_rate=DROPOUT_VARIANCE_RATE, device=device)
    dropout_variance_checkpoints.append(epoch)
    dropout_variance_history.append(variance)
    dropout_variance_mean_acc_history.append(mean_acc)
    # DRC snapshot on every eval-grid checkpoint, same grid as the variance.
    drc_results = compute_dropout_gap_multi_rate(model, test_loader, DROPOUT_RATES)
    dropout_drc_snapshots[epoch] = {
        str(r): drc_results[r]["train_accuracy"] for r in DROPOUT_RATES}

np.save(os.path.join(measurements.dropout_dir, "dropout_variance_checkpoints.npz"),
        np.array(dropout_variance_checkpoints, dtype=int))
np.save(os.path.join(measurements.dropout_dir, "dropout_variance_history.npz"),
        np.array(dropout_variance_history, dtype=float))
np.save(os.path.join(measurements.dropout_dir, "dropout_variance_mean_acc_history.npz"),
        np.array(dropout_variance_mean_acc_history, dtype=float))
with open(os.path.join(measurements.dropout_dir,
    "dropout_drc_snapshots.json"), "w") as handle:
    json.dump({str(e): v for e, v in dropout_drc_snapshots.items()}, handle, indent=2)

```

```

peak_idx = int(np.argmax(dropout_variance_history))
variance_peak_epoch = dropout_variance_checkpoints[peak_idx]
block = {
    "variance_peak_epoch": variance_peak_epoch,
    "variance_peak_value": float(dropout_variance_history[peak_idx]),
    "grok_epoch": grok_epoch,
    "peak_to_grok_ratio": (
        float(variance_peak_epoch) / grok_epoch if grok_epoch else None),
    "rate": DROPOUT_VARIANCE_RATE,
    "n_samples": DROPOUT_VARIANCE_N_SAMPLES,
    "num_checkpoints": len(dropout_variance_checkpoints),
}
with open(os.path.join(measurements.dropout_dir,
    "dropout_variance_signal.json"), "w") as handle:
    json.dump(block, handle, indent=2)
return block

```

```

def _checkpoint_predictor_spectral(model, test_loader, ckpt_dir, ckpt_epochs,
    measurements, grok_epoch, device):
    """CHECKPOINT_PREDICTOR_FUNCS["spectral"] - Canatar et al. 2021
    task-model alignment (see src/predictors/spectral.py). Loads each
    saved checkpoint into `model` in turn, computes
    compute_spectral_metrics_for_checkpoint(model, train_loader, device)
    - kernel eigenvalues eta_k, task power w_k^2, cumulative power C(k),
    k_90 / k_95, alignment_score, entropy - collects the per-checkpoint
    history, saves it via measurements.save_spectral_data, and returns the
    spectral_predictor block for summary.json.

    test_loader is IGNORED - Canatar's metrics are built on the TRAINING
    representations and training labels, not the test split. This function
    rebuilds the exact training split this seed used (same per-seed RNG
    seeding as train_one_seed step (a)) so the sampled kernel matches the
    one the model was actually trained on. The signature keeps
    `test_loader` only to match the shared CHECKPOINT_PREDICTOR_FUNCS
    shape every registered function follows."""
    # Recover this seed and rebuild its training split deterministically.
    seed = int(np.load(os.path.join(measurements.output_dir, "seed.npy"))[0])
    p = model.output_head.out_features - 1
    batch_size = int(0.3 * p * p) # full-batch training split (matches get_data loaders)
    torch.manual_seed(seed)
    np.random.seed(seed)
    train_loader, _ = get_data_loaders(number=p, batch_size=batch_size)

    spectral_checkpoints = []
    hist = {
        "k_90": [], "k_95": [], "alignment_score": [], "entropy": [],
        "eigenvalues_history": [], "cumulative_power_history": [],
    }

    for epoch in ckpt_epochs:
        state = torch.load(os.path.join(ckpt_dir, f"model_epoch_{epoch}.pt"), map_location=device)
        model.load_state_dict(state)
        m = compute_spectral_metrics_for_checkpoint(model, train_loader, device)
        spectral_checkpoints.append(epoch)
        hist["k_90"].append(m["k_90"])
        hist["k_95"].append(m["k_95"])
        hist["alignment_score"].append(m["alignment_score"])
        hist["entropy"].append(m["entropy"])
        hist["eigenvalues_history"].append(m["eigenvalues"])
        hist["cumulative_power_history"].append(m["cumulative_power"])

    measurements.save_spectral_data(spectral_checkpoints, hist)

    # Aggregate signal (Canatar): task-model alignment should form at or
    # before grokking, i.e. k_90 DROPS and alignment_score RISES around
    # grok_epoch. Record the checkpoint epoch of minimum k_90 / maximum
    # alignment and compare against grok_epoch.
    k90 = hist["k_90"]
    align = hist["alignment_score"]
    k_90_min_idx = int(np.argmin(k90))
    k_95_min_idx = int(np.argmin(hist["k_95"]))
    alignment_max_idx = int(np.argmax(align))
    k_90_min_epoch = spectral_checkpoints[k_90_min_idx]
    alignment_max_epoch = spectral_checkpoints[alignment_max_idx]

    block = {
        "spectral_checkpoints": spectral_checkpoints,
        "k_90_history": [int(v) for v in k90],
        "k_95_history": [int(v) for v in hist["k_95"]],
        "alignment_history": [float(v) for v in align],
        "entropy_history": [float(v) for v in hist["entropy"]],
        "grok_epoch": grok_epoch,
        "k_90_min_epoch": k_90_min_epoch,
        "k_95_min_epoch": spectral_checkpoints[k_95_min_idx],
        "alignment_max_epoch": alignment_max_epoch,
    }

```

```

        "k_90_first_last": {"first": int(k90[0]), "last": int(k90[-1])},
        "k_95_first_last": {"first": int(hist["k_95"][0]), "last": int(hist["k_95"][-1])},
        "alignment_first_last": {"first": float(aligned[0]), "last": float(aligned[-1])},
        "k_90_min_to_grok_ratio": (
            float(k_90_min_epoch) / grok_epoch if grok_epoch else None),
        "alignment_max_to_grok_ratio": (
            float(alignment_max_epoch) / grok_epoch if grok_epoch else None),
        "num_checkpoints": len(spectral_checkpoints),
    }
    with open(os.path.join(measurements.spectral_dir,
                          "spectral_signal.json"), "w") as handle:
        json.dump(block, handle, indent=2)
    return block

def _checkpoint_predictor_age(model, test_loader, ckpt_dir, ckpt_epochs,
                             measurements, grok_epoch, device):
    """CHECKPOINT_PREDICTOR_FUNCS["age"] - AGE, Adaptive Grokking Epoch via
    Neural Collapse (Papayan et al. 2020 NC1; see src/predictors/age.py).
    Loads each saved checkpoint into `model` in turn, computes
    compute_age_metrics_for_checkpoint(model, train_loader, device) - the
    NC1 variability-collapse ratio  $\text{Tr}(\Sigma_W)/\text{Tr}(\Sigma_B)$  and the mean
    feature norm - collects the per-checkpoint history, saves it via
    measurements.save_age_data, and returns the age_predictor block for
    summary.json.

    test_loader is IGNORED - NC1 is a property of the TRAINING
    representations (within-/between-class scatter of the penultimate
    features on the train split). This function rebuilds the exact
    training split this seed used (same per-seed RNG seeding as
    train_one_seed step (a) and _checkpoint_predictor_spectral) so the
    class means match the data the model was trained on. The signature
    keeps `test_loader` only to match the shared
    CHECKPOINT_PREDICTOR_FUNCS shape.

    AGE prediction under test: NC1 falls sharply at or BEFORE grok, so the
    checkpoint epoch of minimum NC1 is a leading/coincident grokking
    marker. nc1_min_to_grok_ratio = nc1_min_epoch / grok_epoch is the
    head-to-head number the benchmark records."""
    # Recover this seed and rebuild its training split deterministically.
    seed = int(np.load(os.path.join(measurements.output_dir, "seed.npy"))[0])
    p = model.output_head.out_features - 1
    batch_size = int(0.3 * p * p) # full-batch training split (matches get_data loaders)
    torch.manual_seed(seed)
    np.random.seed(seed)
    train_loader, _ = get_data_loaders(number=p, batch_size=batch_size)

    age_checkpoints = []
    nc1_history, fn_history = [], []

    for epoch in ckpt_epochs:
        state = torch.load(os.path.join(ckpt_dir, f"model_epoch_{epoch}.pt"), map_location=device)
        model.load_state_dict(state)
        m = compute_age_metrics_for_checkpoint(model, train_loader, device)
        age_checkpoints.append(epoch)
        nc1_history.append(m["nc1"])
        fn_history.append(m["fn"])

    measurements.save_age_data(age_checkpoints, {"nc1": nc1_history, "fn": fn_history})

    nc1_min_idx = int(np.argmin(nc1_history))
    nc1_min_epoch = age_checkpoints[nc1_min_idx]
    fn_arr = np.asarray(fn_history, dtype=float)

    block = {
        "age_checkpoints": age_checkpoints,
        "nc1_history": [float(v) for v in nc1_history],
        "fn_history": [float(v) for v in fn_history],
        "grok_epoch": grok_epoch,
        "nc1_min_epoch": nc1_min_epoch,
        "nc1_min_value": float(nc1_history[nc1_min_idx]),
        "nc1_first_last": {"first": float(nc1_history[0]), "last": float(nc1_history[-1])},
        "fn_first_last": {"first": float(fn_history[0]), "last": float(fn_history[-1])},
        "fn_min": float(fn_arr.min()),
        "fn_max": float(fn_arr.max()),
        "nc1_min_to_grok_ratio": (
            float(nc1_min_epoch) / grok_epoch if grok_epoch else None),
        "num_checkpoints": len(age_checkpoints),
    }
    with open(os.path.join(measurements.age_dir, "age_signal.json"), "w") as handle:
        json.dump(block, handle, indent=2)
    return block

# Predictors this runner knows how to (re)compute from already-saved
# checkpoints/model_epoch_*.pt alone, with NO retraining. Adding a future

```

```

# predictor here (once it exists) is the ONLY change needed for it to gain
# the same "compute later, no retrain" resume behaviour dropout_variance
# already has - see _checkpoint_predictor_dropout_variance's docstring for
# the exact function shape expected.
CHECKPOINT_PREDICTOR_FUNCS = {
    "dropout_variance": _checkpoint_predictor_dropout_variance,
    "spectral": _checkpoint_predictor_spectral,
    "age": _checkpoint_predictor_age,
}
CHECKPOINT_ONLY_PREDICTORS = set(CHECKPOINT_PREDICTOR_FUNCS.keys())

def recompute_from_checkpoints(seed, args, cfg, device, predictors_to_compute, old_summary):
    """Compute predictor(s) that only need frozen model weights (currently
    just "dropout_variance", via CHECKPOINT_PREDICTOR_FUNCS) from this
    seed's already-saved checkpoints/model_epoch*.pt, with NO retraining.
    Every other block in old_summary is carried forward unchanged. Only
    called when predictors_to_compute is a subset of
    CHECKPOINT_ONLY_PREDICTORS and has_saved_checkpoints() is true for
    this seed."""
    assert predictors_to_compute &lt;= CHECKPOINT_ONLY_PREDICTORS, (
        f"recompute_from_checkpoints can only rebuild "
        f"{sorted(CHECKPOINT_ONLY_PREDICTORS)} from saved weights; the rest "
        f"need the full per-epoch training history")

    out_dir = seed_dir_for(args.output_dir, seed)
    ckpt_dir = checkpoints_dir_for(args.output_dir, seed)
    p = cfg["modulus"]
    batch_size = int(cfg["train_fraction"] * p * p)
    _, test_loader = get_data loaders(number=p, batch_size=batch_size)

    model = TransformerFourHead(
        vocab_size=p + 1, d_model=cfg["d_model"], num_heads=cfg["num_heads"],
        init_std=cfg["init_std"],
    ).to(device)

    saved_epochs = sorted(
        int(name[len("model_epoch"):-len(".pt")])
        for name in os.listdir(ckpt_dir) if name.startswith("model_epoch") and name.endswith(".pt"))
    # Evaluate on the EVAL grid, subsampled from what is saved on disk.
    ckpt_epochs = select_eval_epochs(saved_epochs, args.eval_every)
    measurements = PredictorMeasurements(out_dir, model_type="four_head")
    grok_epoch = old_summary.get("grok_epoch")

    summary = dict(old_summary) # carry every other block forward unchanged
    for predictor in sorted(predictors_to_compute):
        func = CHECKPOINT_PREDICTOR_FUNCS[predictor]
        block = func(model, test_loader, ckpt_dir, ckpt_epochs, measurements, grok_epoch, device)
        summary[PREDICTOR_SUMMARY_KEY[predictor]] = block

    with open(summary_path_for(args.output_dir, seed), "w") as handle:
        json.dump(summary, handle, indent=2)

    print(f"[seed {seed}] {' ', '.join(sorted(predictors_to_compute))} recomputed from "
          f"{len(ckpt_epochs)} of {len(saved_epochs)} saved checkpoints "
          f"{eval_every={args.eval_every}, no retrain}", flush=True)
    return summary

def aggregate(summaries, args):
    print("\n" + "=" * 72)
    print(f"NANDA-UNIFIED BENCHMARK - AGGREGATE "
          f"{len(summaries)} seeds x {args.epochs} epochs")
    print("=" * 72)

    groks = [s["grok_epoch"] for s in summaries if s["grok_epoch"] is not None]
    if groks:
        print(f"Grok epoch (test acc &gt; {GROK_ACC_THRESHOLD}): "
              f"mean={np.mean(groks):.1f} std={np.std(groks):.1f} "
              f"min={min(groks)} max={max(groks)} "
              f"{len(groks)}/{len(summaries)} seeds grokked")
    else:
        print("Grok epoch: no seed reached the grok threshold")

    # Grok epoch for EVERY threshold (primary first). Older summaries written
    # before the multi-threshold change have no such block; they are skipped.
    grok_by_threshold = {}
    for t in GROK_THRESHOLDS:
        per_seed = [(s.get("grok_epochs_by_threshold") or {}).get(str(t)) for s in summaries]
        reached = [e for e in per_seed if e is not None]
        grok_by_threshold[str(t)] = {
            "per_seed": per_seed,
            "mean": float(np.mean(reached)) if reached else None,
            "std": float(np.std(reached)) if reached else None,
            "n_grokked": len(reached),
        }

```

```

print("\nGrok epoch by threshold (first epoch test acc > threshold):")
for t, info in grok_by_threshold.items():
    mean_str = f"mean={info['mean']:.1f} std={info['std']:.1f}" if info['mean'] is not None else "mean=n/a"
    print(f" > {t:<5}: {mean_str} ({info['n_grokked']}/{len(summaries)} seeds) "
          f"per seed={info['per_seed']}")

print("\nL2-Norm predictor (per seed):")
for s in summaries:
    lp = s.get("l2_predictor")
    if lp is None:
        print(f" seed {s['seed']}: n/a (not computed for this seed)")
        continue
    print(f" seed {s['seed']}: MA-crossover={lp['ma_crossover_epoch']} "
          f"MA-of-MA zero-cross={lp['ma_of_ma_zero_crossing_epoch']} "
          f"(grok={s['grok_epoch']})")

print("\nDropout-Variance predictor (per seed):")
for s in summaries:
    dv = s.get("dropout_variance_predictor")
    if dv is None:
        print(f" seed {s['seed']}: n/a (not computed for this seed)")
        continue
    ratio = dv["peak_to_grok_ratio"]
    ratio_str = f"{ratio:.4f}" if ratio is not None else "None"
    print(f" seed {s['seed']}: variance_peak_epoch={dv['variance_peak_epoch']} "
          f"peak/grok={ratio_str} (grok={s['grok_epoch']})")

print("\nSpectral predictor (Canatar task-model alignment, per seed):")
for s in summaries:
    sp = s.get("spectral_predictor")
    if sp is None:
        print(f" seed {s['seed']}: n/a (not computed for this seed)")
        continue
    k90 = sp["k_90_first_last"]
    al = sp["alignment_first_last"]
    print(f" seed {s['seed']}: k_90 {k90['first']}->{k90['last']} "
          f"k_90_min_epoch={sp['k_90_min_epoch']} "
          f"alignment {al['first']:.4f}->{al['last']:.4f} "
          f"alignment_max_epoch={sp['alignment_max_epoch']} "
          f"(grok={s['grok_epoch']})")

print("\nAGE predictor (NC1 variability collapse, per seed):")
for s in summaries:
    ap = s.get("age_predictor")
    if ap is None:
        print(f" seed {s['seed']}: n/a (not computed for this seed)")
        continue
    nc1 = ap["nc1_first_last"]
    ratio = ap["nc1_min_to_grok_ratio"]
    ratio_str = f"{ratio:.4f}" if ratio is not None else "None"
    print(f" seed {s['seed']}: NC1 {nc1['first']:.4f}->{nc1['last']:.4f} "
          f"nc1_min_epoch={ap['nc1_min_epoch']} "
          f"nc1_min/grok={ratio_str} (grok={s['grok_epoch']})")

print("\nDropout gap at final epoch (per seed, by rate):")
for s in summaries:
    gaps = s.get("dropout_final_gap_by_rate")
    if gaps is None:
        print(f" seed {s['seed']}: n/a (not computed for this seed)")
        continue
    print(f" seed {s['seed']}: " +
          " ".join(f"p{r}={gaps[r]:+.3f}" for r in gaps))

print("\nLimit-cycle check (post-grok test-acc oscillation):")
n_limit_cycle = 0
for s in summaries:
    lc = s["limit_cycle_check"]
    if not lc.get("applicable"):
        print(f" seed {s['seed']}: n/a ({lc.get('reason', 'n/a')})")
        continue
    n_limit_cycle += int(lc["limit_cycle"])
    label = "LIMIT CYCLE" if lc["limit_cycle"] else "stable"
    print(f" seed {s['seed']}: {label} "
          f"post-grok min={lc['post_grok_min']:.3f} "
          f"std={lc['post_grok_std']:.3f} "
          f"final={lc['post_grok_final']:.3f} "
          f"dips<{GROK_ACC_THRESHOLD}={lc.get('dips_key', '')}")
print(f"\n => {n_limit_cycle}/{len(summaries)} seeds show a post-grok limit cycle")

agg_path = os.path.join(args.output_dir, "aggregate.json")
with open(agg_path, "w") as handle:
    json.dump({
        "n_seeds": len(summaries),
        "epochs": args.epochs,
        "grok_epoch_mean": float(np.mean(groks)) if groks else None,
        "grok_epoch_std": float(np.std(groks)) if groks else None,
    }, handle)

```

```

        "grok_epochs": groks,
        "grok_thresholds": GROK_THRESHOLDS,
        "grok_epochs_by_threshold": grok_by_threshold,
        "n_limit_cycle": n_limit_cycle,
        "seeds": summaries,
    }, handle, indent=2)
print(f"\nWrote {agg_path}")
print("=" * 72)

def main():
    parser = argparse.ArgumentParser(
        description="Nanda-Unified benchmark runner (L2-Norm + Dropout only).",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )
    parser.add_argument("--seeds", type=int, default=5,
                        help="number of seeds; seed i uses torch.manual_seed(i)")
    parser.add_argument("--epochs", type=int, default=TOTAL_STEPS,
                        help="full-batch epochs per seed")
    parser.add_argument("--output_dir", type=str, default="08_Experiments/results/nanda_unified")
    parser.add_argument("--config", type=str, default="06_Code/configs/nanda_unified.yaml")
    parser.add_argument("--predictors", type=str, default="l2,dropout_gap,dropout_variance",
                        help="comma-separated subset of " + ",".join(ALL_PREDICTORS))
    parser.add_argument("--overwrite", type=str, default="",
                        help="comma-separated predictors to force-recompute even if "
                             "already present in summary.json")
    parser.add_argument("--no_save_checkpoints", action="store_true",
                        help="disable saving checkpoints/model_epoch_*.pt during training")
    parser.add_argument("--eval_every", type=int, default=EVAL EVERY,
                        help=f"evaluate predictors on saved checkpoints with epoch %% eval_every == 0 "
                             f"(plus the final epoch). Must be a multiple of the save spacing "
                             f"({SAVE EVERY}); fallbacks if too slow: {EVAL EVERY_FALLBACK}, "
                             f"{EVAL EVERY_FALLBACK_COARSE}. "
                             f"0 = use every saved checkpoint (legacy 24-point directories).")
    args = parser.parse_args()

    if not os.path.isabs(args.output_dir):
        args.output_dir = os.path.join(REPO_ROOT, args.output_dir)
    if not os.path.isabs(args.config):
        args.config = os.path.join(REPO_ROOT, args.config)
    os.makedirs(args.output_dir, exist_ok=True)

    requested_predictors = {p.strip() for p in args.predictors.split(",") if p.strip()}
    unknown = requested_predictors - set(ALL_PREDICTORS)
    assert not unknown, f"--predictors: unknown predictor(s) {unknown}, valid: {ALL_PREDICTORS}"
    overwrite_predictors = {p.strip() for p in args.overwrite.split(",") if p.strip()}
    unknown_ow = overwrite_predictors - set(ALL_PREDICTORS)
    assert not unknown_ow, f"--overwrite: unknown predictor(s) {unknown_ow}, valid: {ALL_PREDICTORS}"

    cfg = load_config(args.config)
    apply_config_constants(cfg)
    device = pick_device()

    print("=" * 72)
    print("NANDA-UNIFIED BENCHMARK RUNNER (predictors built so far: L2-Norm, Dropout)")
    print("=" * 72)
    print(f"config      : {args.config}")
    print(f"output_dir  : {args.output_dir}")
    print(f"device      : {device}")
    print(f"task        : (a + b) mod {cfg['modulus']} "
          f"{'=' token id {cfg['modulus']} vocab {cfg['vocab_size']}"")
    print(f"model       : 4-head, d_model={cfg['d_model']}, "
          f"{'init_std={cfg['init_std']:.5f} (small init)"}")
    print(f"optimiser   : AdamW lr={cfg['lr']} betas={cfg['betas']} "
          f"{'weight_decay={cfg['weight_decay']}"")
    print(f"run         : seeds={args.seeds} epochs={args.epochs} "
          f"{'full-batch={int(cfg['train_fraction'] * cfg['modulus'] * cfg['modulus'])}"")
    print(f"grok thr.   : {GROK_THRESHOLDS} (primary {GROK_ACC_THRESHOLD}) "
          f"{'dropout-variance: n_samples={DROPOUT_VARIANCE_N_SAMPLES} rate={DROPOUT_VARIANCE_RATE}"")
    print(f"checkpoints: save every {SAVE EVERY} (gcd of {DESIRED_EVAL_GRIDS}), "
          f"{'evaluate every {args.eval_every}"")
    print("=" * 72)

    summaries = []
    for seed in range(args.seeds):
        old_summary = load_summary(args.output_dir, seed)
        done = get_done_predictors(args.output_dir, seed)
        effective_done = done - overwrite_predictors

        missing = set()
        for predictor in sorted(requested_predictors):
            if predictor in effective_done:
                print(f"[seed {seed}] {predictor} complete - skipping")
            else:
                missing.add(predictor)
        if not missing:

```

```

        print(f"[seed {seed}] all predictors complete - skipping")
        summaries.append(old_summary)
        continue

    can_skip_retrain = (
        missing &lt;= CHECKPOINT_ONLY_PREDICTORS
        and old_summary is not None
        and has_saved_checkpoints(args.output_dir, seed)
    )
    if can_skip_retrain:
        summaries.append(
            recompute_from_checkpoints(seed, args, cfg, device, missing, old_summary))
    else:
        summary = train_one_seed(
            seed, args, cfg, device, predictors_to_compute=missing,
            old_summary=old_summary, save_checkpoints=not args.no_save_checkpoints)
        # Some CHECKPOINT_ONLY_PREDICTORS (e.g. spectral) are never
        # computed live inside train_one_seed's epoch loop - they only
        # need the checkpoints that fresh training just saved. Run
        # those now, straight off this seed's brand-new checkpoints,
        # so a first-time "--predictors spectral" run does not silently
        # finish with spectral_predictor still None.
        leftover = {p for p in (missing & CHECKPOINT_ONLY_PREDICTORS)
                     if summary.get(PREDICTOR_SUMMARY_KEY[p]) is None}
        if leftover and not args.no_save_checkpoints and has_saved_checkpoints(args.output_dir, seed):
            summary = recompute_from_checkpoints(seed, args, cfg, device, leftover, summary)
        summaries.append(summary)

    aggregate(summaries, args)

if __name__ == "__main__":
    main()

```

14. 06_Code/scripts/scaffold.py

```
"""
```

Scaffold script – creates the src/ folder structure for the grokking benchmark rebuild.
Run once from the project root: python scaffold.py

```
"""
```

```
import os
```

```
FILES = [  
    "src/data/modular_arithmetic.py",  
    "src/models/transformer.py",  
    "src/train.py",  
]
```

```
for path in FILES:  
    folder = os.path.dirname(path)  
    if folder:  
        os.makedirs(folder, exist_ok=True)  
    if not os.path.exists(path):  
        open(path, "w").close()  
        print(f"created: {path}")  
    else:  
        print(f"skipped (already exists): {path}")
```

15. 06_Code/src/data/modular_arithmetic.py

```
from torch import long, tensor
from torch.utils.data import Dataset, DataLoader, random_split

def generate_pairs(number):
    pairs = []
    for i in range(number):
        for j in range(number):
            pairs.append((i, j, (i + j) % number))
    return pairs

def get_dataloaders(number, batch_size):
    modular_arithmetic_dataset = ModularArithmeticDataset(number)
    train_size = int(0.3 * len(modular_arithmetic_dataset))
    test_size = len(modular_arithmetic_dataset) - train_size

    train_dataset, test_dataset = random_split(modular_arithmetic_dataset, [train_size, test_size])
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_dataloader = DataLoader(test_dataset, batch_size=batch_size)

    return train_dataloader, test_dataloader

class ModularArithmeticDataset(Dataset):
    def __init__(self, number):
        self.number = number
        self.pairs = generate_pairs(number)

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        return (self.get_tensor(self.pairs[idx]), self.pairs[idx][2])

    def get_tensor(self, item):
        # item is (a, b, answer). The third sequence position is the "="
        # token, whose id is the modulus itself (number tokens are 0..p-1,
        # so the "=" token id is p). Previously hardcoded to 97; now
        # parametrised so the Nanda-Unified protocol (p = 113) gives the
        # "=" token id 113. See configs/nanda_unified.yaml.
        sequence = [item[0], item[1], self.number]
        return tensor(sequence)

if __name__ == "__main__":
    print(tensor([5, 3, 8, 113]))
```

16. 06_Code/src/models/model_shadow_with_layernorm.py

```
from torch import arange, nn
import torch

# SHADOW - LayerNorm added, original had no norm
USE_FINAL_LN = False

class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

        self.ln1 = nn.LayerNorm(d_model) # SHADOW - LayerNorm added, original had no norm
        self.ln2 = nn.LayerNorm(d_model) # SHADOW - LayerNorm added, original had no norm
        self.ln_final = nn.LayerNorm(d_model) # SHADOW - LayerNorm added, original had no norm

    def forward(self, x):
        token_vectors = self.token_embedding(x)
        position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors
        query_vector = self.query(combined_vector)
        key_vector = self.key(combined_vector)
        value_vector = self.value(combined_vector)

        scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (query_vector.size(-1) ** 0.5)
        attention_weights = torch.softmax(scores, dim=-1)
        attention_values = combined_vector + self.dropout1(torch.matmul(attention_weights, value_vector))
        attention_values = self.ln1(attention_values) # SHADOW - LayerNorm added, original had no norm
        mlp_output = attention_values + self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))
        mlp_output = self.ln2(mlp_output) # SHADOW - LayerNorm added, original had no norm

        if USE_FINAL_LN:
            mlp_output = self.ln_final(mlp_output) # SHADOW - LayerNorm added, original had no norm

        logits = self.output_head(mlp_output)

        return logits

if __name__ == "__main__":
    model = Transformer(vocab_size=98, d_model=128)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

    # Quick 100 epoch sanity check - should not crash
    # Should reach >90% test acc and ideally break the 99.5446% plateau
    pass
```

17. 06_Code/src/models/transformer_four_head.py

```
from torch import arange, nn
import torch

# =====
# 4-HEAD VARIANT – matches Nanda et al.'s actual architecture
# (d_model=128, 4 attention heads, head dimension = 128/4 = 32 each).
#
# The original src/models/transformer.py is UNCHANGED and still uses
# single-head attention (one Query/Key/Value block covering the full
# d_model=128 at once). That file is left exactly as it is – this is a
# separate, new model class, following the same "shadow copy" pattern
# already used in this project for the LayerNorm side-experiment
# (see src/models/model_shadow_with_layernorm.py).
#
# This is the SHARED model architecture for the Nanda-Unified benchmark
# protocol (configs/nanda_unified.yaml). It is used by training, every
# predictor, and evaluation – it is not owned by any one predictor.
#
# Design: 1-layer, 4-head, same token + position embedding, MLP (4x
# expansion + ReLU), NO LayerNorm, learned positional embeddings,
# untied embed/unembed, same dropout1/dropout2 hooks (used by
# predictors/dropout.py's stress-test), same output head.
#
# Two Nanda-alignment details baked in here:
# 1. WEIGHT INIT. Every weight matrix – embeddings included – is drawn
#    from N(0, init_std) with init_std = 0.8 / sqrt(d_model) (~0.0707
#    for d_model=128). This is the TransformerLens / Nanda scheme.
#    PyTorch defaults (nn.Embedding = N(0,1), nn.Linear =
#    Kaiming-uniform) would otherwise make the token-embedding table
#    ~94% of the starting weight norm and force a large early collapse
#    under weight decay that has nothing to do with grokking.
# 2. MLP BIASES. query/key/value and the output head have NO bias
#    (Nanda: no biases in attention or unembedding), but the MLP keeps
#    b_in / b_out (Nanda keeps them). MLP biases are zero-initialised.
# =====

class TransformerFourHead(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads=4, init_std=None):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        # TransformerLens / Nanda init scale: N(0, 0.8 / sqrt(d_model)).
        if init_std is None:
            init_std = 0.8 / (d_model ** 0.5)
        self.init_std = init_std

        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=True)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=True)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

        self._init_weights()

    def _init_weights(self):
        # Match Nanda et al. / TransformerLens: every weight matrix,
        # embeddings included, drawn from N(0, init_std). MLP biases start
        # at zero.
        for module in [self.token_embedding, self.position_embedding,
                        self.query, self.key, self.value,
                        self.mlp_in, self.mlp_out, self.output_head]:
            nn.init.normal_(module.weight, mean=0.0, std=self.init_std)
        for module in [self.mlp_in, self.mlp_out]:
            if module.bias is not None:
                nn.init.zeros_(module.bias)

    def split_into_heads(self, x):
        # x: (batch_size, seq_len, d_model) –> (batch_size, num_heads, seq_len, head_dim)
        # This is the step that turns one big d_model-wide attention into
        # num_heads smaller, independent attention computations running
        # side by side.
```

```

batch_size, seq_len, d_model = x.shape
x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
return x.permute(0, 2, 1, 3)

def combine_heads(self, x):
    # x: (batch_size, num_heads, seq_len, head_dim) -> (batch_size, seq_len, d_model)
    # Reverses split_into_heads - puts all heads' outputs back side by
    # side into one d_model-wide vector, exactly as Nanda et al.
    # describe combining the 4 heads' outputs before the MLP.
    batch_size, num_heads, seq_len, head_dim = x.shape
    x = x.permute(0, 2, 1, 3)
    return x.reshape(batch_size, seq_len, num_heads * head_dim)

def forward(self, x):
    token_vectors = self.token_embedding(x)
    position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
    combined_vector = token_vectors + position_vectors

    query_vector = self.split_into_heads(self.query(combined_vector))
    key_vector = self.split_into_heads(self.key(combined_vector))
    value_vector = self.split_into_heads(self.value(combined_vector))

    # Scaling uses head_dim (32), not the full d_model (128) - each
    # head does its own scaled dot-product attention independently.
    scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (self.head_dim ** 0.5)
    attention_weights = torch.softmax(scores, dim=-1)
    attention_output = torch.matmul(attention_weights, value_vector)
    attention_output = self.combine_heads(attention_output)

    attention_values = combined_vector + self.dropout1(attention_output)
    mlp_output = attention_values +
        self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))

    logits = self.output_head(mlp_output)

    return logits

if __name__ == "__main__":
    # Nanda-Unified: p = 113 -> vocab_size = 113 + 1 = 114
    model = TransformerFourHead(vocab_size=114, d_model=128, num_heads=4)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

```

18.06_Code/src/plot_results_four_head.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os
import re

from predictors.l2_norm import compute_noise_floor, detect_ma_of_ma_zero_crossing, detect_ma_crossover

# =====
# 4-HEAD VARIANT of plot_results.py – now MULTI-RUN aware.
#
# train_four_head.py saves every run into its own numbered subfolder,
# results/four_head/run_1/, results/four_head/run_2/, and so on (see that
# file's RUN NUMBERING comment for why – grokking is stochastic, so a
# real comparison needs several independent runs kept side by side, not
# one run's numbers overwritten by the next).
#
# This script does two things:
# 1. For every run <N> folder it finds, regenerates that run's own 8
#    plots (same as before), saved inside that run's own folder.
# 2. Builds 4 NEW comparison plots at the results/four_head/ level,
#    overlaying every run's grokking curve / loss curve / L2 norm
#    curve / Dropout Gap curve together, so all previous runs can be
#    read on one chart instead of opening each run's folder one by one.
# =====

project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
four_head_dir = os.path.join(project_root, "runs", "four_head")

SKIP_EPOCHS = 100
QUIET_EPOCH_CUTOFF = 90

LEGACY_FILENAMES = [
    "train_acc_history.npy", "test_acc_history.npy", "loss_history.npy",
    "l2_norm_history.npy", "dropout_gap_epochs.npy", "dropout_gap_history.npy",
    "dropout_train_acc_history.npy", "dropout_eval_acc_history.npy",
    "epoch_grid.npy", "fast_ma.npy", "slow_ma.npy", "fast_ma_of_slow_ma.npy",
    "ma_of_ma_diff.npy", "training_report.pdf",
    "ma_of_slow_ma_crossover.png", "ma_of_slow_ma_diff.png",
    "ma_of_slow_ma_diff_linear.png", "ma_of_ma_diff_vs_grokking_linear.png",
    "grokking_curve.png", "loss_curve.png", "l2_norm_curve.png", "dropout_gap_curve.png",
]

# Distinct colors cycled across runs, so Run 1 / Run 2 / Run 3 ... always
# look different from each other on the comparison plots.
RUN_COLORS = ["steelblue", "darkorange", "seagreen", "crimson", "purple",
              "goldenrod", "teal", "indianred", "slategray", "mediumvioletred"]

def migrate_legacy_flat_run(base_dir):
    """Same migration as train_four_head.py – kept here too so this
    script can be re-run on its own and still pick up an old flat run
    even if train_four_head.py has not been re-run since the update."""
    run_1_dir = os.path.join(base_dir, "run_1")
    if os.path.isdir(run_1_dir):
        return
    legacy_present = any(
        os.path.isfile(os.path.join(base_dir, name)) for name in LEGACY_FILENAMES
    )
    if not legacy_present:
        return
    os.makedirs(run_1_dir, exist_ok=True)
    for name in LEGACY_FILENAMES:
        src = os.path.join(base_dir, name)
        if os.path.isfile(src):
            os.rename(src, os.path.join(run_1_dir, name))
    print("[MIGRATION] Found results saved directly in results/four_head/ by an earlier "
          "version of these scripts – moved them into results/four_head/run_1/ so they "
          "are counted as Run 1.")

def discover_run_dirs(base_dir):
    """Finds every run <N> folder inside base_dir, sorted by run number
    (not alphabetically, so run_2 comes before run_10)."""
    if not os.path.isdir(base_dir):
        return []
    found = []
    for name in os.listdir(base_dir):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(base_dir, name)):
            found.append((int(match.group(1)), os.path.join(base_dir, name)))
    found.sort(key=lambda pair: pair[0])
    return found
```

```

def load_run_data(run_dir):
    # unified_measurements.PredictorMeasurements writes into per-predictor
    # subfolders: training/, l2_norm/, dropout/. Each entry below is
    # (key, relative_path).
    required_files = [
        ("train_acc", "training/train_acc_history.npy"), ("test_acc", "training/test_acc_history.npy"),
        ("loss", "training/loss_history.npy"), ("l2_norm", "l2_norm/l2_norm_history.npy"),
        ("epoch_grid", "l2_norm/epoch_grid.npy"), ("fast_ma", "l2_norm/fast_ma.npy"),
        ("slow_ma", "l2_norm/slow_ma.npy"), ("fast_ma_of_slow_ma", "l2_norm/fast_ma_of_slow_ma.npy"),
        ("ma_of_ma_diff", "l2_norm/ma_of_ma_diff.npy"),
        ("dropout_gap_epochs", "dropout/dropout_gap_epochs.npy"),
        ("dropout_gap_by_rate", "dropout/dropout_gap_by_rate.npy"),
        ("dropout_rates", "dropout/dropout_rates.npy"),
    ]

    for _, rel in required_files:
        if not os.path.exists(os.path.join(run_dir, rel)):
            return None

    data = {}
    for key, rel in required_files:
        data[key] = np.load(os.path.join(run_dir, rel))
    data["num_epochs"] = len(data["train_acc"])
    return data

def plot_single_run(run_number, run_dir, data):
    """Recreates this run's own 8 plots, saved inside results/four_head/run_&N&gt;/ itself."""
    epoch_grid = data["epoch_grid"]
    fast_ma = data["fast_ma"]
    slow_ma = data["slow_ma"]
    fast_ma_of_slow_ma = data["fast_ma_of_slow_ma"]
    diff = data["ma_of_ma_diff"]
    train_acc = data["train_acc"]
    test_acc = data["test_acc"]
    loss_history = data["loss"]
    l2_norm_history = data["l2_norm"]
    dropout_gap_epochs = data["dropout_gap_epochs"]
    dropout_gap_by_rate = data["dropout_gap_by_rate"] # shape [rate, epoch]
    dropout_rates = data["dropout_rates"]
    num_epochs = data["num_epochs"]
    epochs_axis = range(1, num_epochs + 1)

    noise_floor = compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=QUIET_EPOCH_CUTOFF)
    trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=SKIP_EPOCHS)
    detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=SKIP_EPOCHS)
    grokked = bool((test_acc > 0.9).any())
    grok_epoch = int(np.argmax(test_acc > 0.9)) if grokked else None

    def save(fig, filename):
        path = os.path.join(run_dir, filename)
        fig.savefig(path, dpi=150, bbox_inches="tight")
        plt.close(fig)
        print(f"[OK] Run {run_number}: plot saved to results/four_head/run_{run_number}/{filename}")

    # Plot 1: Slow MA vs. Fast MA of Slow MA
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epoch_grid, slow_ma, color="purple", linewidth=3.5, label="Slow MA (w=200)")
    ax.plot(epoch_grid, fast_ma_of_slow_ma, color="orange", linewidth=3.5, label="Fast MA of Slow MA (w=20)")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
        ax.scatter(trigger_epoch, slow_ma[trigger_idx], color="red", zorder=5, s=150, marker="*",
            label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("L2 Norm", fontsize=12)
    ax.set_title(f"4-head model, Run {run_number}: Slow MA vs. Fast MA of Slow MA", fontsize=13)
    ax.legend(loc="upper right", fontsize=11)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig, "ma_of_slow_ma_crossover.png")

    # Plot 2: Difference curve, log-x
    fig2, ax2 = plt.subplots(figsize=(12, 7))
    ax2.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
    ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
    ax2.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
    ax2.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax2.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
        ax2.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150, marker="*",
            label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax2.set_xscale("log")
    ax2.set_xlabel("Epoch (log scale)", fontsize=12)

```

```

ax2.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax2.set_title(f"4-head model, Run {run_number}: Difference - Fast MA of Slow MA minus Slow MA", fontsize=13)
ax2.legend(loc="upper right", fontsize=11)
ax2.grid(True, alpha=0.3)
plt.tight_layout()
save(fig2, "ma_of_slow_ma_diff.png")

# Plot 3: Same difference curve, linear x-axis
fig3, ax3 = plt.subplots(figsize=(12, 7))
ax3.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax3.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax3.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax3.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")
if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax3.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax3.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150, marker="*",
               label=f"Trigger (epoch {trigger_epoch:.0f})")
ax3.set_xlabel("Epoch (linear scale)", fontsize=12)
ax3.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax3.set_title(f"4-head model, Run {run_number}: Difference - Linear Scale", fontsize=13)
ax3.legend(loc="upper right", fontsize=11)
ax3.grid(True, alpha=0.3)
plt.tight_layout()
save(fig3, "ma_of_slow_ma_diff_linear.png")

# Plot 4: Difference curve overlaid on the grokking curve
fig4, ax4 = plt.subplots(figsize=(12, 7))
ax4.plot(range(1, num_epochs + 1), train_acc, color="steelblue", linewidth=2.5, label="Train Accuracy")
ax4.plot(range(1, num_epochs + 1), test_acc, color="seagreen", linewidth=2.5, label="Test Accuracy")
ax4.set_xlabel("Epoch (linear scale)", fontsize=12)
ax4.set_ylabel("Accuracy", fontsize=12)
ax4.set_ylim(-0.05, 1.05)
ax4b = ax4.twinx()
ax4b.plot(epoch_grid, diff, color="darkred", linewidth=3, label="Fast MA of Slow MA - Slow MA", alpha=0.85)
ax4b.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax4b.set_ylabel("MA-of-MA Difference", fontsize=12)
if grokked:
    ax4.axvline(x=grok_epoch, color="seagreen", linestyle=":", linewidth=2, alpha=0.7)
if trigger_epoch is not None:
    ax4.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax4b.scatter(trigger_epoch, diff[np.argmin(np.abs(epoch_grid - trigger_epoch))],
               color="red", zorder=5, s=150, marker="*", label=f"Trigger (epoch {trigger_epoch:.0f})")
title_suffix = f"Grok epoch {grok_epoch}" if grokked else "did not grok in this run"
ax4.set_title(f"4-head model, Run {run_number}: MA-of-MA Difference vs. Grokking Curve - {title_suffix}",
             fontsize=13)
lines4, labels4 = ax4.get_legend_handles_labels()
lines4b, labels4b = ax4b.get_legend_handles_labels()
ax4.legend(lines4 + lines4b, labels4 + labels4b, loc="center right", fontsize=10)
ax4.grid(True, alpha=0.3)
plt.tight_layout()
save(fig4, "ma_of_ma_diff_vs_grokking_linear.png")

# Plot 5: Grokking curve
fig5, ax5 = plt.subplots(figsize=(12, 7))
ax5.plot(epochs_axis, train_acc, label="Train Accuracy", color="steelblue", linewidth=2)
ax5.plot(epochs_axis, test_acc, label="Test Accuracy", color="seagreen", linewidth=2)
ax5.set_xscale("log")
ax5.set_xlabel("Epoch (log scale)")
ax5.set_ylabel("Accuracy")
ax5.set_title(f"4-head model, Run {run_number}: Grokking Curve - Train vs. Test Accuracy")
ax5.legend(loc="center right")
ax5.grid(True, alpha=0.3)
plt.tight_layout()
save(fig5, "grokking_curve.png")

# Plot 6: Loss curve
fig6, ax6 = plt.subplots(figsize=(12, 7))
ax6.plot(epochs_axis, loss_history, color="darkorange", linewidth=2)
ax6.set_xscale("log")
ax6.set_xlabel("Epoch (log scale)")
ax6.set_ylabel("Loss")
ax6.set_title(f"4-head model, Run {run_number}: Training Loss")
ax6.grid(True, alpha=0.3)
plt.tight_layout()
save(fig6, "loss_curve.png")

# Plot 7: L2 Norm curve, with MA Crossover marked
fig7, ax7 = plt.subplots(figsize=(12, 7))
ax7.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
if detection_epoch is not None:
    ax7.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
               label=f"MA Crossover (epoch {detection_epoch:.0f})")
ax7.set_xscale("log")
ax7.set_xlabel("Epoch (log scale)")
ax7.set_ylabel("L2 Norm")

```

```

ax7.set_title(f"4-head model, Run {run_number}: L2 Norm of Model Weights")
ax7.legend(loc="upper right")
ax7.grid(True, alpha=0.3)
plt.tight_layout()
save(fig7, "l2_norm_curve.png")

# Plot 8: Dropout Gap – full multi-rate sweep
fig8, ax8 = plt.subplots(figsize=(12, 7))
for r_idx, rate in enumerate(dropout_rates):
    ax8.plot(dropout_gap_epochs, dropout_gap_by_rate[r_idx], linewidth=2,
             label=f"p={rate:g}")
ax8.axhline(y=0, color="black", linewidth=0.6, alpha=0.5)
ax8.set_xscale("log")
ax8.set_xlabel("Epoch (log scale)")
ax8.set_ylabel("Dropout Gap")
ax8.set_title(f"4-head model, Run {run_number}: Dropout Gap – Multi-Rate Sweep")
ax8.legend()
ax8.grid(True, alpha=0.3)
plt.tight_layout()
save(fig8, "dropout_gap_curve.png")

final_gap_by_rate = {float(rate): float(dropout_gap_by_rate[r_idx][-1])
                     for r_idx, rate in enumerate(dropout_rates)}

return {
    "run_number": run_number, "num_epochs": num_epochs, "grokked": grokked,
    "grok_epoch": grok_epoch, "final_test_acc": float(test_acc[-1]),
    "final_l2_norm": float(l2_norm_history[-1]),
    "final_dropout_gap_by_rate": final_gap_by_rate,
}

def plot_comparison(all_runs_data, out_dir):
    """Builds 4 plots overlaying every discovered run together, saved
    directly in results/four_head/ (not inside any single run's folder),
    so all previous runs can be read on one chart."""

    if not all_runs_data:
        print("[SKIP] No complete runs to compare – comparison plots not generated.")
        return

    def color_for(i):
        return RUN_COLORS[i % len(RUN_COLORS)]

    # Comparison Plot A: Test Accuracy (grokking curve) across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
        color = color_for(i)
        ax.plot(epochs_axis, data["test_acc"], color=color, linewidth=2.2,
              label=f"Run {run_number} ({data['num_epochs']} epochs)")
        if (data["test_acc"] > 0.9).any():
            grok_epoch = int(np.argmax(data["test_acc"] > 0.9))
            ax.axvline(x=grok_epoch, color=color, linestyle=":", linewidth=1.5, alpha=0.6)
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Test Accuracy", fontsize=12)
    ax.set_ylim(-0.05, 1.05)
    ax.set_title("4-head model: Test Accuracy Across All Runs", fontsize=13)
    ax.legend(loc="center right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "comparison_grokking_curve.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to results/four_head/comparison_grokking_curve.png")

    # Comparison Plot B: Loss across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
        ax.plot(epochs_axis, data["loss"], color=color_for(i), linewidth=2,
              label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Loss", fontsize=12)
    ax.set_title("4-head model: Training Loss Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "comparison_loss_curve.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to results/four_head/comparison_loss_curve.png")

    # Comparison Plot C: L2 Norm across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)

```

```

        ax.plot(epochs_axis, data["l2_norm"], color=color_for(i), linewidth=2,
                label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("L2 Norm", fontsize=12)
    ax.set_title("4-head model: L2 Norm of Model Weights Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "comparison_l2_norm_curve.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to results/four_head/comparison_l2_norm_curve.png")

# Comparison Plot D: Dropout Gap sweep – one panel per run, all rates
n = len(all_runs_data)
fig, axes = plt.subplots(1, n, figsize=(6 * n, 5), squeeze=False)
for i, (run_number, data) in enumerate(all_runs_data):
    ax = axes[0][i]
    rates = data["dropout_rates"]
    gap_by_rate = data["dropout_gap_by_rate"]
    for r_idx, rate in enumerate(rates):
        ax.plot(data["dropout_gap_epochs"], gap_by_rate[r_idx], linewidth=1.8,
                alpha=0.9, label=f"p={rate:g}")
    ax.axhline(y=0, color="black", linewidth=0.6, alpha=0.5)
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=11)
    ax.set_ylabel("Dropout Gap", fontsize=11)
    ax.set_title(f"Run {run_number} ({data['num_epochs']} epochs)", fontsize=12)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)
fig.suptitle("4-head model: Dropout Gap Multi-Rate Sweep Across All Runs", fontsize=13)
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "comparison_dropout_gap_curve.png"), dpi=150, bbox_inches="tight")
plt.close(fig)
print("[OK] Comparison plot saved to results/four_head/comparison_dropout_gap_curve.png")

migrate_legacy_flat_run(four_head_dir)
run_dirs = discover_run_dirs(four_head_dir)

if not run_dirs:
    print(f"No runs found under results/four_head/. Run train_four_head.py at least once first.")
else:
    print(f"Found {len(run_dirs)} run(s): {' '.join(f'Run {n}' for n, _ in run_dirs)}\n")

    all_runs_data = []
    summaries = []
    for run_number, run_dir in run_dirs:
        data = load_run_data(run_dir)
        if data is None:
            print(f"Skipping Run {run_number} (incomplete or empty)")
            continue
        all_runs_data.append((run_number, data))
        summaries.append(plot_single_run(run_number, run_dir, data))

    plot_comparison(all_runs_data, four_head_dir)

    print("\n" + "=" * 60)
    print("SUMMARY ACROSS ALL RUNS")
    print("=" * 60)
    for s in summaries:
        grok_text = f"grokked at epoch {s['grok_epoch']}" if s["grokked"] else "did NOT grok"
        gap_text = ", ".join(f"p{rate:g}={gap:+.4f}"
                             for rate, gap in sorted(s["final_dropout_gap_by_rate"].items()))
        print(f"Run {s['run_number']} ({s['num_epochs']} epochs): {grok_text}, "
              f"final test acc = {s['final_test_acc']:.4f}, "
              f"final L2 norm = {s['final_l2_norm']:.4f}")
        print(f"final dropout gap sweep: {gap_text}")

    print(f"\n[OK] Per-run plots saved inside each results/four_head/run_&N& folder.")
    print(f"[OK] 4 comparison plots saved directly in results/four_head/:")
    print(" - comparison_grokking_curve.png")
    print(" - comparison_loss_curve.png")
    print(" - comparison_l2_norm_curve.png")
    print(" - comparison_dropout_gap_curve.png")

```

19.06_Code/src/plot_shadow_layernorm.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os

# Look for files in project root (parent of src/), same pattern as plot_results.py
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Known plateau value from the original (no-LayerNorm) model, recorded in
# context.md (Aug 7, 2026 plateau-investigation session): 6557/6587 correct.
ORIGINAL_PLATEAU_ACC = 0.995446

shadow_train_acc = np.load("shadow_ln_train_acc_history.npy")
shadow_test_acc = np.load("shadow_ln_test_acc_history.npy")
num_epochs = len(shadow_train_acc)
epochs_axis = range(1, num_epochs + 1)

fig, ax = plt.subplots(figsize=(12, 7))

ax.plot(epochs_axis, shadow_train_acc, color="steelblue", linewidth=2.5, label="Shadow (LayerNorm) Train Accuracy")
ax.plot(epochs_axis, shadow_test_acc, color="seagreen", linewidth=2.5, label="Shadow (LayerNorm) Test Accuracy")

# Reference line: the original no-LayerNorm model's known plateau value.
ax.axhline(y=ORIGINAL_PLATEAU_ACC, color="darkred", linestyle="--", linewidth=2,
           label=f"Original plateau (no LayerNorm): {ORIGINAL_PLATEAU_ACC:.4%}")

# If the original run's test_acc_history.npy is present in the project root,
# overlay it too, so both curves can be compared directly on one graph.
# This is optional – the shadow curve above still plots fine without it.
try:
    original_test_acc = np.load("test_acc_history.npy")
    original_epochs_axis = range(1, len(original_test_acc) + 1)
    ax.plot(original_epochs_axis, original_test_acc, color="darkorange", linewidth=2,
            linestyle=":", label="Original (no LayerNorm) Test Accuracy", alpha=0.85)
except FileNotFoundError:
    print("Note: test_acc_history.npy (original run) not found – skipping overlay, "
          "shadow curve plotted alone.")

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)", fontsize=12)
ax.set_ylabel("Accuracy", fontsize=12)
ax.set_ylim(-0.05, 1.05)
ax.set_title("Shadow Model (with LayerNorm) – Grokking Curve vs. Original Plateau", fontsize=13)
ax.legend(loc="center right", fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("shadow_ln_grokking_curve.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to shadow_ln_grokking_curve.png")

final_shadow_test_acc = shadow_test_acc[-1]
print(f"\nFinal shadow (LayerNorm) test accuracy: {final_shadow_test_acc:.6f}")
print(f"Original (no LayerNorm) plateau: {ORIGINAL_PLATEAU_ACC:.6f}")
if final_shadow_test_acc > ORIGINAL_PLATEAU_ACC:
    print("Shadow model beat the original plateau.")
elif final_shadow_test_acc == ORIGINAL_PLATEAU_ACC:
    print("Shadow model matched the original plateau exactly.")
else:
    print("Shadow model did not reach the original plateau.")
```

20.06_Code/src/predictors/age.py

""""
AGE predictor (Predictor 4 of 9) – Adaptive Grokking Epoch via Neural Collapse.

DEFINITION (locked)

AGE = the epoch at which NC1 "within-class variability collapse" occurs.
NC1 is the neural-collapse quantity from Papayan, Han & Donoho 2020:

Papayan, Vardan; Han, X. Y.; Donoho, David L. "Prevalence of neural collapse during the terminal phase of deep learning training." Proceedings of the National Academy of Sciences 117 (40), 24652–24663 (2020). PDF in repo:
literature/Vardan Papayan – Prevalence of neural collapse during the terminal phase of deep learning training [2020].pdf

Papayan's NC1 ("variability collapse"): the last-layer features of examples from the same class stop varying – the within-class scatter shrinks relative to the between-class scatter, i.e.

$$NC1 = \text{Tr}(\text{Sigma}_W) / \text{Tr}(\text{Sigma}_B) \gg 0$$

Sigma_W = within-class covariance of the penultimate features,
Sigma_B = between-class covariance of the class means.

AGE reads this collapse epoch off frozen checkpoints and compares it to the grok epoch. The prediction under test: NC1 falls sharply at or BEFORE the test-accuracy jump, so the epoch of minimum NC1 is a leading (or coincident) marker of grokking.

MECHANISM SUPPORT

Why last-layer neural collapse is expected to track grokking on this substrate at all:

- Beaglehole, Daniel; Radhakrishnan, Adityanarayanan; Pandit, Parthe; Belkin, Mikhail. "Average gradient outer product as a mechanism for deep neural collapse." arXiv:2402.13728 (2024). NC1 collapse is driven by the Average Gradient Outer Product (AGOP) reshaping features class-wise.
- Mallinar, Neil; Beaglehole, Daniel; Zhu, Libin; Radhakrishnan, Adityanarayanan; Pandit, Parthe; Belkin, Mikhail. "Emergence in non-neural models: grokking modular arithmetic via average gradient outer product." arXiv:2407.20199 (2024). The same AGOP mechanism produces grokking on (a+b) mod p specifically – the task this benchmark runs.

Threshold / collapse-epoch reading follows Paul & Rupa 2026, "Neural Collapse Dynamics".

WHAT THIS FILE COMPUTES (and what it does NOT)

Per frozen checkpoint, on the model's own penultimate representation at the read position (the vector that feeds `output\_head`), one per TRAINING example: $\Phi \in \mathbb{R}^{N \times d_{\text{model}}}$, $N = |\text{train split}| = \text{floor}(0.3 * p * p) = 3830$ for $p = 113$.

μ_c = mean of Φ over examples of class c
 μ_G = global mean of Φ
 Tr_W = $\text{mean}_c \text{ mean}_{\{i \text{ in } c\}} || h_i - \mu_c ||^2$ (within-class)
 Tr_B = $\text{mean}_c || \mu_c - \mu_G ||^2$ (between-class)
 $NC1$ = $\text{Tr}_W / (\text{Tr}_B + 1e-12)$
 fn = $\text{mean}_i || h_i ||_2$ (feature norm)

There is NO weight-matrix SVD here. AGE is a representation-space, loss-agnostic, checkpoint-only signal – completely independent of the Nanda training substrate, the optimiser, and the loss curve. It needs only frozen weights and the training inputs.

MPS NOTE

Representations are moved off the "mps" backend BEFORE promotion to float64 (MPS has no float64): `captured["h"].cpu().double()`. All the class-mean / scatter arithmetic then runs on CPU in double precision.
""""

import torch

_EPS = 1e-12

def _collect_representations_and_labels(model, train_loader, device):
 """Run the model over the full training split and capture, per example,
 the hidden state that feeds `output\_head` at the read position

```

(sequence index 2, the "=" token). Returns:
    Phi : [N, d_model] float64, CPU
    Y   : [N]          int64, CPU   (class labels 0..p-1)

Identical collection logic to src/predictors/spectral.py – hook on
model.output_head, take inputs[0][:, 2, :], move off MPS first, then
promote to float64.
"""
captured = {}

def _hook(_module, inputs, _output):
    # inputs[0] is mlp_output, shape [B, seq_len, d_model]; the vector
    # that goes into output_head. read_position = 2 (configs/nanda_unified.yaml).
    captured["h"] = inputs[0][:, 2, :].detach()

handle = model.output_head.register_forward_hook(_hook)
model.eval()
phi_parts, y_parts = [], []
try:
    with torch.no_grad():
        for x, y in train_loader:
            x = x.to(device)
            model.forward(x)
            # move off MPS FIRST, then promote – MPS has no float64.
            phi_parts.append(captured["h"].cpu().double())
            y_parts.append(y.detach().cpu())
finally:
    handle.remove()

Phi = torch.cat(phi_parts, dim=0)
Y = torch.cat(y_parts, dim=0).long()
return Phi, Y

def compute_age_metrics_for_checkpoint(model, train_loader, device):
    """Papayan et al. 2020 NC1 variability-collapse metrics for ONE frozen
    model checkpoint. `train_loader` must iterate the full training split
    (for p=113 that is N = floor(0.3 * 113 * 113) = 3830 examples; the
    Nanda-Unified run is full-batch, so one iteration already covers it).

    Returns a dict:
        nc1 : float    Tr(Sigma_W) / (Tr(Sigma_B) + 1e-12) -- lower = more collapsed
        fn  : float    mean_i ||h_i||_2 -- feature-norm scale
        N   : int      number of training examples used
    """
    Phi, Y = _collect_representations_and_labels(model, train_loader, device)
    N, _d_model = Phi.shape
    p = model.output_head.out_features - 1 # vocab = p + 1 ("=" token id is p)

    mu_G = Phi.mean(dim=0) # [d_model]

    within_sq = [] # per-class mean ||h_i - mu_c||^2
    between_sq = [] # per-class ||mu_c - mu_G||^2
    for c in range(p):
        mask = (Y == c)
        n_c = int(mask.sum())
        if n_c == 0:
            continue # class unseen in this train split – skip, do not bias the mean
        Phi_c = Phi[mask] # [n_c, d_model]
        mu_c = Phi_c.mean(dim=0) # [d_model]
        within_sq.append(((Phi_c - mu_c) ** 2).sum(dim=1).mean())
        between_sq.append(((mu_c - mu_G) ** 2).sum())

    tr_w = float(torch.stack(within_sq).mean())
    tr_b = float(torch.stack(between_sq).mean())
    nc1 = tr_w / (tr_b + _EPS)

    fn = float(Phi.norm(dim=1).mean())

    return {"nc1": nc1, "fn": fn, "N": int(N)}

def compute_age_for_model(model, train_loader, device):
    """Thin wrapper kept parallel to
    src/predictors/spectral.py::compute_spectral_for_model. One frozen
    checkpoint in –> one NC1-metrics dict out (see
    compute_age_metrics_for_checkpoint)."""
    return compute_age_metrics_for_checkpoint(model, train_loader, device)

```

21. 06_Code/src/predictors/dropout.py

```
import numpy as np
import torch

def compute_accuracy(model, data_loader):
    device = next(model.parameters()).device
    total_correct = 0
    total_samples = 0
    for x, y in data_loader:
        x, y = x.to(device), y.to(device)
        with torch.no_grad():
            logit = model.forward(x)
            equal_sign_logit = logit[:, 2, :]
            predicted = equal_sign_logit.argmax(dim=1)
            total_correct += (predicted == y).sum().item()
            total_samples += len(y)
    return total_correct / total_samples if total_samples > 0 else 0.0

# NOTE: the old single-rate compute_dropout_gap(model, data_loader, dropout_rate)
# has been removed. The Dropout predictor is now a full multi-rate sweep only -
# no hardcoded p=0.9 anywhere. Use compute_dropout_gap_multi_rate.

def compute_dropout_gap_multi_rate(model, data_loader, dropout_rates):
    # Step 1: START (inputs: model, data_loader, dropout_rates)

    # Step 2: Setup Evaluation Mode
    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0

    # Step 3: Compute Clean Accuracy (execute ONLY ONCE)
    clean_accuracy = compute_accuracy(model, data_loader)

    # Step 4: Initialize Results Storage
    results = {}

    # Step 5: FOR each rate in dropout_rates
    for dropout_rate in dropout_rates:
        # --- Switch to train mode with dropout active ---
        model.train()
        model.dropout1.p = dropout_rate
        model.dropout2.p = dropout_rate

        # Compute train accuracy (with dropout)
        train_accuracy = compute_accuracy(model, data_loader)

        # Compute dropout gap (train_accuracy - clean_accuracy)
        dropout_gap = train_accuracy - clean_accuracy

        # Store results
        results[dropout_rate] = {
            "train_accuracy": train_accuracy,
            "eval_accuracy": clean_accuracy,      # == clean_accuracy (dropout=0, eval mode)
            "dropout_gap": dropout_gap,
            "clean_accuracy": clean_accuracy
        }

    # Step 6: Loop Decision - "More rates?" (handled by for-loop)

    # Step 7: Final Cleanup - restore model to clean eval state
    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0

    # Step 8: RETURN results - END
    return results

def compute_dropout_variance(model, data_loader, n_samples=30, dropout_rate=0.5, device=None):
    """
    Dropout-variance predictor signal (Salah & Yevick, arXiv:2507.11645):
    at a frozen checkpoint, run `n_samples` independent stochastic forward
    passes over the test set with dropout ACTIVE, and look at how much the
    resulting test accuracy jumps around from pass to pass.

    Why rate=0.5: this is the rate at which Salah & Yevick's own Dropout
    Robustness Curve shows the clearest pre-/post-grok separation (their
    Fig. 2 - post-grok accuracy survives dropout up to about rate 0.5),
    and it keeps the primary signal to one rate for compute-cost reasons
    (n_samples stochastic passes at every checkpoint, for every seed, adds
    up fast).
    "Dropout active during eval" = model.train() (so nn.Dropout samples a

```

fresh mask every forward pass) wrapped in torch.no_grad() (so no gradients/graph are built – this is a measurement, not a training step). There is no BatchNorm anywhere in TransformerFourHead, so model.train() only affects the two nn.Dropout layers here; nothing else needs to be forced back to eval mode.

Returns (mean_accuracy, variance_of_accuracy) over the n_samples passes. Restores model.eval() / dropout p=0.0 before returning, same cleanup discipline as compute_dropout_gap_multi_rate.

```
"""
if device is None:
    device = next(model.parameters()).device

model.train()
model.dropout1.p = dropout_rate
model.dropout2.p = dropout_rate

accuracies = []
with torch.no_grad():
    for _ in range(n_samples):
        total_correct = 0
        total_samples = 0
        for x, y in data_loader:
            x, y = x.to(device), y.to(device)
            logit = model.forward(x)
            equal_sign_logit = logit[:, 2, :]
            predicted = equal_sign_logit.argmax(dim=1)
            total_correct += (predicted == y).sum().item()
            total_samples += len(y)
        accuracies.append(total_correct / total_samples if total_samples > 0 else 0.0)

model.eval()
model.dropout1.p = 0.0
model.dropout2.p = 0.0

accuracies = np.array(accuracies, dtype=float)
return float(accuracies.mean()), float(accuracies.var())
```

22. 06_Code/src/predictors/l2_norm.py

```
import numpy as np
import torch
from scipy.ndimage import uniform_filter1d

def compute_l2_norm(model):
    all_params = torch.cat([p.flatten() for p in model.parameters()])
    l2_norm = torch.norm(all_params).item()
    return l2_norm

def compute_sum_of_squared_weights(model):
    """
    Sum of squared weights over ALL parameters (sum w^2). This is the exact
    quantity Nanda et al. plot in Figure 7 ("Total Sum of Squared Weights").
    Note compute_l2_norm(model) == sqrt(compute_sum_of_squared_weights(model));
    both are logged so the Nanda-Unified weight-norm curve can be compared
    to the paper directly without re-squaring.
    """
    total = 0.0
    for p in model.parameters():
        total += torch.sum(p.detach() ** 2).item()
    return total

# Which top-level module names roll up into which reported group. Used by
# compute_per_module_sum_of_squared_weights(). Groups follow
# configs/nanda_unified.yaml &gt; logging.weight_norm_per_module_sum_w2.
PER_MODULE_GROUPS = {
    "token_embedding": ["token_embedding"],
    "position_embedding": ["position_embedding"],
    "attention_qkv": ["query", "key", "value"],
    "mlp": ["mlp_in", "mlp_out"],
    "output_head": ["output_head"],
}

def compute_per_module_sum_of_squared_weights(model):
    """
    Sum w^2 split by module group (token_embedding, position_embedding,
    attention_qkv, mlp, output_head). Lets the weight-space predictors see
    WHERE the weight norm moves – e.g. confirm the early transient (if any)
    is the embedding table and a pre-grok bump is in attention/MLP.

    Returns a dict {group_name: sum_w2}. Any parameter whose top-level name
    is not in PER_MODULE_GROUPS is ignored (there are none in the current
    model, but this keeps the function safe against future modules).
    """
    prefix_to_group = {}
    for group, prefixes in PER_MODULE_GROUPS.items():
        for prefix in prefixes:
            prefix_to_group[prefix] = group

    out = {group: 0.0 for group in PER_MODULE_GROUPS}
    for name, p in model.named_parameters():
        top_level = name.split(".")[0]
        group = prefix_to_group.get(top_level)
        if group is not None:
            out[group] += torch.sum(p.detach() ** 2).item()
    return out

def compute_l2_norm_rate_of_decline(l2_norm_history):
    return np.diff(l2_norm_history) * -1

def detect_l2_norm_drop(rate_of_decline, threshold=0.05, skip_epochs=200):
    for i, rate in enumerate(rate_of_decline):
        if i < skip_epochs: # Skip early epochs
            continue
        if rate > threshold:
            return i
    return None

def compute_acceleration(rate_of_decline):
    """
    Compute the second derivative of L2 norm (acceleration).
    acceleration[i] = rate[i] - rate[i-1]
    Positive = decay rate is increasing (speeding up)
    Negative = decay rate is decreasing (slowing down)
    Sign change = inflection point (change in decay character)
    """
    acceleration = np.diff(rate_of_decline)
    return acceleration

def detect_inflection(acceleration, skip_epochs=200):
```

```

"""
Detect inflection point where acceleration changes sign.
This is where the decay pattern changes character – exactly what happens
before/during the grok transition.
Returns the epoch index where the sign flip occurs, or None.
"""
if len(acceleration) <= skip_epochs:
    return None

# Check from skip_epochs onwards for sign changes
for i in range(skip_epochs, len(acceleration)):
    # acceleration[i-1] * acceleration[i] < 0 means opposite signs (sign flip)
    if i > 0 and acceleration[i-1] * acceleration[i] < 0:
        return i

return None

def apply_moving_average(data, window_size=50):
    """
    Apply moving average smoothing to reduce noise.
    window_size: number of epochs to average over (default 50).
    Returns smoothed array (same length as input, padded at edges).
    """
    return uniform_filter1d(np.array(data), size=window_size, mode='nearest')

def resample_to_log_uniform_grid(data, num_points=None):
    """
    Resample data onto a grid uniformly spaced in log10(epoch) – i.e. the
    SAME spacing the data has when viewed on a log-x plot.
    Without this, a fixed window_size (in raw epoch index) covers a huge
    visual chunk of the plot near epoch 1 and a tiny sliver near epoch N,
    which is why smoothing looked over-smoothed early and scribbly late.
    Returns (epoch_grid, resampled_data) where epoch_grid is uniform in
    log10-space (non-uniform in linear epoch terms).
    """
    data = np.array(data)
    num_epochs = len(data)
    epochs = np.arange(1, num_epochs + 1)
    log_epochs = np.log10(epochs)

    if num_points is None:
        num_points = num_epochs

    log_grid = np.linspace(log_epochs[0], log_epochs[-1], num_points)
    resampled = np.interp(log_grid, log_epochs, data)
    epoch_grid = 10 ** log_grid

    return epoch_grid, resampled

def compute_fast_slow_moving_averages(l2_norm_history, fast_window=50, slow_window=200, num_points=None):
    """
    Compute two moving averages on a LOG-EPOCH-UNIFORM grid, so the window
    covers equal VISUAL width everywhere on a log-x plot (fixes both the
    over-smoothing near epoch 1 and the scribbling near epoch N).

    Steps:
    1. Log-transform L2 norm values (handles the exponential decay in y).
    2. Resample onto a grid uniform in log10(epoch) (handles the log-x axis).
    3. Apply the moving average on that resampled grid.
    4. Convert back to linear L2 norm space.

    fast_window / slow_window are now measured in grid points, which (since
    the grid is log-uniform) corresponds to a constant *proportional* width
    in epochs everywhere along the curve.

    Returns (epoch_grid, fast_ma, slow_ma) – always plot fast_ma/slow_ma
    against epoch_grid, not against range(1, num_epochs+1).
    """
    l2_norm_history = np.array(l2_norm_history)
    log_l2_norm = np.log(l2_norm_history)

    epoch_grid, log_l2_norm_resampled = resample_to_log_uniform_grid(log_l2_norm, num_points=num_points)

    fast_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=fast_window)
    slow_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=slow_window)

    fast_ma = np.exp(fast_ma_log)
    slow_ma = np.exp(slow_ma_log)

    return epoch_grid, fast_ma, slow_ma

def detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100):

```

```

"""
Detect where fast MA crosses above slow MA, searching along the
log-uniform grid. skip_epochs is a real epoch value (not a grid index) -
crossovers before this epoch are ignored (initialization noise).
Returns the real epoch (float) of the crossover, or None.
"""
if len(fast_ma) != len(slow_ma):
    return None

for i in range(len(fast_ma) - 1):
    if epoch_grid[i] < skip_epochs:
        continue
    # Previous: fast <= slow, Current: fast > slow (crossover from below)
    if fast_ma[i] <= slow_ma[i] and fast_ma[i + 1] > slow_ma[i + 1]:
        return epoch_grid[i + 1]

return None

def compute_ma_of_slow_ma(slow_ma, fast_window=20):
    """
    Apply a second, faster-window moving average on top of the already-smoothed
    slow_ma. Since slow_ma has already filtered out raw L2 norm noise, this
    reveals slow_ma's OWN turning points cleanly - the difference between the
    two tracks how sharply slow_ma is currently bending.
    Returns (fast_ma_of_slow_ma, diff) where diff = fast_ma_of_slow_ma - slow_ma.
    """
    fast_ma_of_slow_ma = apply_moving_average(slow_ma, window_size=fast_window)
    diff = fast_ma_of_slow_ma - slow_ma
    return fast_ma_of_slow_ma, diff

def compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=90):
    """
    Estimate the normal noise level of the MA-of-MA difference from the quiet
    early-training region (before any real dynamics begin), so later spikes
    can be judged against what "flat" actually looks like for this run.
    """
    quiet_mask = epoch_grid < quiet_epoch_cutoff
    return np.std(diff[quiet_mask])

def detect_ma_of_ma_trigger(epoch_grid, diff, noise_floor, threshold_multiplier=10, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff climbs past
    threshold_multiplier x noise_floor - the rising edge of a real signal.
    Causal: only needs the noise floor (known early) and points seen so far,
    unlike picking the curve's peak, which requires seeing the whole future.
    Returns the real epoch (float) of the trigger, or None.
    """
    threshold = threshold_multiplier * noise_floor
    for i in range(len(diff)):
        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] > threshold:
            return epoch_grid[i]
    return None

def detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff crosses from
    positive to negative (the first "green -> red" zero crossing).
    Confirmed on two separate training runs to land in a consistent,
    unambiguous spot even though the crossing's magnitude is weak.
    Returns the real epoch (float) of the crossing, or None.
    """
    for i in range(len(diff) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] >= 0 and diff[i + 1] < 0:
            return epoch_grid[i + 1]
    return None

```

23.06_Code/src/predictors/spectral.py

"""

Spectral predictor (Predictor 3 of 9) – FAITHFUL Canatar et al. 2021.

Paper: Abdulkadir Canatar, Blake Bordelon, Cengiz Pehlevan, "Spectral bias and task-model alignment explain generalization in kernel regression and infinitely wide neural networks", Nature Communications 12, 2914 (2021). PDF in repo: literature/Abdulkadir Canatar – Spectral bias and task-model alignmentpdf

WHAT THIS NOW COMPUTES (and what it replaced)

The previous version of this file computed per-weight-matrix singular value statistics (spectral_norm, fro_norm, stable_rank, effective_rank). That was a cheap proxy inspired by Nanda et al.'s low-rank Fourier circuit, NOT anything from Canatar et al. It has been deleted entirely – no `stable\_rank`, no `svdvals(W)` on weight matrices anywhere below.

This file is now a direct implementation of Canatar et al. 2021, Eq. (3)–(6), applied to the trained transformer's own representation kernel:

1. Representation kernel. Take the model's final hidden state at the read position (the vector that feeds `output\_head`), one per TRAINING example: Φ in $\mathbb{R}^{(N \times d_{\text{model}})}$, $N = |\text{train split}| = 3830$ for $p = 113$ ($\text{floor}(0.3 * 113 * 113)$). Centre it and form the Gram matrix $K = \Phi_c @ \Phi_c.T$ ($N \times N$). This is the empirical, sampled version of the kernel whose Mercer spectrum Canatar's theory needs (their Eq. 3: the eigenproblem of the kernel under the data measure).
2. Kernel eigenvalues η_k (Canatar Eq. 3). $\text{eigh}(K)$, sorted descending. "Spectral bias" = the model learns the eigenfunctions with the largest η_k first.
3. Task power w_k^2 (Canatar Eq. 4–5). Project the centred one-hot label matrix Y_c in $\mathbb{R}^{(N \times p)}$ onto the kernel eigenvectors U : $W = U.T @ Y_c$, and $w_k^2 = ||W_k||^2$ (row-norm squared, summed over the p classes). Normalised: $p_k = w_k^2 / \sum_j w_j^2$ is the fraction of the target function's power carried by eigenmode k .
4. Cumulative power $C(k)$ (Canatar Eq. 6, "task-model alignment"). $C(k) = \sum_{j \leq k} p_j$. If the target's power is concentrated in the top (large- η) modes, $C(k)$ rises to 1 for small k and the task is "aligned" with the kernel – Canatar's condition for sample-efficient generalization.
5. Predictor signals per checkpoint:
 - k_{90} / k_{95} : smallest k with $C(k) \geq 0.90 / 0.95$ (how many leading eigenmodes hold 90 / 95 % of the task power).
 - $\text{alignment_score} = \sum_k \eta_k \text{norm} * p_k$: task power weighted by normalised eigenvalue. High $\Rightarrow$ task power sits in the large eigenvalue directions.
 - $\text{entropy} = -\sum_k p_k \log p_k$: spread of the task power over eigenmodes.

THE PREDICTION. Canatar's mechanism says generalization becomes possible once the target aligns with the top of the kernel spectrum. So the grokking signal this predictor tests is: ****k₉₀ should DROP (and alignment_score should RISE) at or before the grok epoch**** – task-model alignment forming ahead of the test-accuracy jump. $k_{90_min_epoch}$ vs grok_epoch is the head-to-head number the benchmark records.

MPS NOTE. `torch.linalg.eigh` does not run on the "mps" backend. The Gram matrix is moved to CPU (and promoted to float64) before `eigh` – cheap, since this is one eigendecomposition per saved checkpoint, not per training epoch.

"""

```
import torch
import torch.nn.functional as F
```

```
# How many leading eigenvalues / cumulative-power points to persist per
# checkpoint (full N=3830 vectors are not needed downstream; the signal
# lives in the head of the spectrum).
```

```
N_EIGENVALUES_SAVED = 50
N_CUMULATIVE_SAVED = 100
```

```
_EPS = 1e-12
```

```
def _collect_representations_and_labels(model, train_loader, device):
    """Run the model over the full training split and capture, per example,
    the hidden state that feeds `output_head` at the read position
    (sequence index 2, the "=" token). Returns:
```

```

        Phi : [N, d_model] float64, CPU
        Y   : [N]          int64, CPU   (class labels 0..p-1)
    """
    captured = {}

    def _hook(_module, inputs, _output):
        # inputs[0] is mlp_output, shape [B, seq_len, d_model]; the vector
        # that goes into output_head. read_position = 2 (configs/nanda_unified.yaml).
        captured["h"] = inputs[0][:, 2, :].detach()

    handle = model.output_head.register_forward_hook(_hook)
    model.eval()
    phi_parts, y_parts = [], []
    try:
        with torch.no_grad():
            for x, y in train_loader:
                x = x.to(device)
                model.forward(x)
                # move off MPS FIRST, then promote - MPS has no float64.
                phi_parts.append(captured["h"].cpu().double())
                y_parts.append(y.detach().cpu())
    finally:
        handle.remove()

    Phi = torch.cat(phi_parts, dim=0)
    Y = torch.cat(y_parts, dim=0).long()
    return Phi, Y

def compute_spectral_metrics_for_checkpoint(model, train_loader, device):
    """Canatar et al. 2021 task-model-alignment metrics for ONE frozen
    model checkpoint. `train_loader` must iterate the full training split
    (for p=113 that is N = floor(0.3 * 113 * 113) = 3830 examples; the
    Nanda-Unified run is full-batch, so one iteration already covers it).

    Returns a dict:
        eigenvalues      : list[float]  top N_EIGENVALUES_SAVED of eta_k (descending)
        k_90, k_95       : int           smallest k with C(k) >= 0.90 / 0.95
        alignment_score   : float         sum_k eta_k_norm * p_k
        entropy           : float         -sum_k p_k log p_k
        cumulative_power  : list[float]  C(1..N_CUMULATIVE_SAVED)
        N                 : int           number of training examples used
    """
    Phi, Y = _collect_representations_and_labels(model, train_loader, device)
    N, _d_model = Phi.shape
    p = model.output_head.out_features - 1 # vocab = p + 1 ("=" token id is p)

    # ---- (1) centred representation Gram matrix K = Phi_c Phi_c^T ----
    Phi_c = Phi - Phi.mean(dim=0, keepdim=True)
    K = Phi_c @ Phi_c.T # [N, N], float64, CPU

    # ---- (2) kernel eigenvalues eta_k (Canatar Eq. 3) ----
    # eigh runs on CPU only (MPS has no aten::_linalg_eigh); K is already CPU.
    eigvals, eigvecs = torch.linalg.eigh(K)
    # eigh returns ascending -> flip to descending.
    eta = eigvals.flip(0).clamp_min(0.0) # [N]
    U = eigvecs.flip(1) # [N, N], column k <-> eta[k]

    # ---- (3) task power w_k^2 = || (U^T Y_c)_k ||^2 (Canatar Eq. 4-5) ----
    Y_onehot = F.one_hot(Y, num_classes=p).to(torch.float64) # [N, p]
    Y_c = Y_onehot - Y_onehot.mean(dim=0, keepdim=True)
    W = U.T @ Y_c # [N, p]
    power = (W ** 2).sum(dim=1) # [N] w_k^2
    power_norm = power / (power.sum() + _EPS) # p_k

    # ---- (4) cumulative power C(k) (Canatar Eq. 6) ----
    C = torch.cumsum(power_norm, dim=0) # [N]

    def _first_k_at_least(threshold):
        hits = (C >= threshold).nonzero()
        return int(hits[0].item()) + 1 if len(hits) else int(N)

    k_90 = _first_k_at_least(0.90)
    k_95 = _first_k_at_least(0.95)

    # ---- (5) scalar signals ----
    eta_norm = eta / (eta.sum() + _EPS)
    alignment_score = float((eta_norm * power_norm).sum())
    entropy = float(-(power_norm * torch.log(power_norm + _EPS)).sum())

    n_eig = min(N_EIGENVALUES_SAVED, int(N))
    n_cum = min(N_CUMULATIVE_SAVED, int(N))

    return {
        "eigenvalues": [float(v) for v in eta[:n_eig]],
        "k_90": k_90,

```

```

    "k_95": k_95,
    "alignment_score": alignment_score,
    "entropy": entropy,
    "cumulative_power": [float(v) for v in C[:n_cum]],
    "N": int(N),
}

```

```

def compute_spectral_for_model(model, train_loader, device):
    """Thin wrapper kept for run_nanda_benchmark.py's import. One frozen
    checkpoint in -&gt; one Canatar-metrics dict out (see
    compute_spectral_metrics_for_checkpoint)."""
    return compute_spectral_metrics_for_checkpoint(model, train_loader, device)

```

24.06_Code/src/train_four_head.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
import re
import yaml
from predictors.l2_norm import (
    compute_l2_norm,
    compute_sum_of_squared_weights,
    compute_per_module_sum_of_squared_weights,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from data.modular_arithmetic import get_dataloaders
from models.transformer_four_head import TransformerFourHead
from predictors.dropout import compute_dropout_gap_multi_rate
from unified_measurements import PredictorMeasurements

# =====
# 4-HEAD training loop – Nanda-Unified benchmark protocol.
#
# The substrate is defined ONCE in configs/nanda_unified.yaml and loaded
# below. This script asserts its own constants against that file so the
# two cannot drift silently. Every predictor in the thesis is measured on
# a model trained by this exact loop.
#
# Nanda-Unified protocol (see configs/nanda_unified.yaml, context.md):
# - task : (a + b) mod 113 ("=" token id 113, vocab 114)
# - architecture : 1-layer, 4-head, d_model=128, d_mlp=512, ReLU,
#   NO LayerNorm, learned positions, untied embed/unembed,
#   no bias in attention/embedding/unembedding, MLP has bias
# - init : N(0, 0.8/sqrt(d_model)) for EVERY weight matrix
#   (TransformerLens / Nanda scale)
# - optimiser : AdamW, lr=1e-3, weight_decay=1.0, betas=(0.9, 0.98)
# - training : full-batch, no warmup, 40000 epochs, 30/70 split
#
# This replaced the old default substrate (p=97, PyTorch-default init,
# betas 0.9/0.999). Those 3 runs are archived at archive/p97_default/.
#
# Outputs are saved under results/four_head/run_<N>/ (created if it does not
# exist yet), NOT in the project root, so they never overwrite train.py's
# own train_acc_history.npy / l2_norm_history.npy / dropout_gap_history.npy
# / etc., which belong to the original single-head run.
#
# RUN NUMBERING: every time this script is run, it saves into a NEW
# numbered subfolder (run_1, run_2, run_3, ...) instead of overwriting the
# previous run's results. This is needed because grokking is stochastic –
# the grok epoch moves with the random seed – so a real comparison needs
# several independent runs kept side by side, not one run's numbers
# overwritten by the next. plot_results_four_head.py reads every run_<N>;
# folder it finds and plots them together for comparison.
# =====

LEGACY_FILENAMES = [
    "train_acc_history.npy", "test_acc_history.npy", "loss_history.npy",
    "l2_norm_history.npy", "dropout_gap_epochs.npy", "dropout_gap_history.npy",
    "dropout_train_acc_history.npy", "dropout_eval_acc_history.npy",
    "epoch_grid.npy", "fast_ma.npy", "slow_ma.npy", "fast_ma_of_slow_ma.npy",
    "ma_of_ma_diff.npy", "training_report.pdf",
    "ma_of_slow_ma_crossover.png", "ma_of_slow_ma_diff.png",
    "ma_of_slow_ma_diff_linear.png", "ma_of_ma_diff_vs_grokking_linear.png",
    "grokking_curve.png", "loss_curve.png", "l2_norm_curve.png", "dropout_gap_curve.png",
]

def migrate_legacy_flat_run(four_head_dir):
    """
    Earlier versions of this script saved directly into results/four_head/
    instead of results/four_head/run_<N>/. If such files are found here, and
    run_1/ does not exist yet, move them into run_1/ so they are counted
    as the first run instead of silently sitting outside the new
    numbering scheme (and instead of being silently overwritten by this
    run, which is what would have happened before this change).
    """
    run_1_dir = os.path.join(four_head_dir, "run_1")
    if os.path.isdir(run_1_dir):
        return
    legacy_present = any(
        os.path.isfile(os.path.join(four_head_dir, name)) for name in LEGACY_FILENAMES
    )
```

```

    if not legacy_present:
        return
    os.makedirs(run_1_dir, exist_ok=True)
    for name in LEGACY_FILENAMES:
        src = os.path.join(four_head_dir, name)
        if os.path.isfile(src):
            os.rename(src, os.path.join(run_1_dir, name))
    print("[MIGRATION] Found results saved directly in results/four_head/ by an earlier "
          "version of this script – moved them into results/four_head/run_1/ so they are "
          "counted as Run 1 instead of being overwritten.")

def get_next_run_number(four_head_dir):
    if not os.path.isdir(four_head_dir):
        return 1
    existing_numbers = []
    for name in os.listdir(four_head_dir):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(four_head_dir, name)):
            existing_numbers.append(int(match.group(1)))
    return max(existing_numbers, default=0) + 1

project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# ---- Load the single common benchmark protocol -----
CONFIG_PATH = os.path.join(project_root, "configs", "nanda_unified.yaml")
with open(CONFIG_PATH) as f:
    CFG = yaml.safe_load(f)

MODULUS = CFG["task"]["modulus"]
VOCAB_SIZE = CFG["task"]["vocab_size"]
TRAIN_FRACTION = CFG["task"]["train_fraction"]
D_MODEL = CFG["architecture"]["d_model"]
NUM_HEADS = CFG["architecture"]["num_heads"]
INIT_STD = CFG["init"]["init_std"]
if INIT_STD is None:
    INIT_STD = 0.8 / (D_MODEL ** 0.5)
LR = CFG["optimizer"]["lr"]
WEIGHT_DECAY = CFG["optimizer"]["weight_decay"]
BETAS = tuple(CFG["optimizer"]["betas"])
NUM_EPOCHS = CFG["training"]["epochs"]
BATCH_SIZE = int(TRAIN_FRACTION * MODULUS * MODULUS)

# Fail loud if this script and the protocol file ever disagree.
assert VOCAB_SIZE == MODULUS + 1, "vocab_size must be modulus + 1"
assert D_MODEL % NUM_HEADS == 0, "d_model must be divisible by num_heads"
print(f"Protocol: {CONFIG_PATH}")
print(f"p={MODULUS} vocab={VOCAB_SIZE} d_model={D_MODEL} heads={NUM_HEADS}")
print(f"init_std={INIT_STD:.5f} betas={BETAS} wd={WEIGHT_DECAY} lr={LR}")
print(f"epochs={NUM_EPOCHS} full-batch size={BATCH_SIZE}")

four_head_dir = os.path.join(project_root, "runs", "four_head")
migrate_legacy_flat_run(four_head_dir)
run_number = get_next_run_number(four_head_dir)

output_dir = os.path.join(four_head_dir, f"run_{run_number}")
os.makedirs(output_dir, exist_ok=True)
os.chdir(output_dir)

# Initialize unified measurements system
measurements = PredictorMeasurements(output_dir, model_type="four_head")

print(f"This is Run {run_number}. All outputs will be saved to runs/four_head/run_{run_number}/")

# NEW random seed on every run, instead of a fixed one. torch.seed() picks
# a fresh, non-deterministic seed itself and also returns the exact number
# it used, so we can both use it and record it in the same step. This is
# needed because grokking is stochastic (grok epoch moves with the seed –
# see the L2 Norm predictor's 3-run history) and the thesis needs several
# genuinely independent 4-head runs, not the same run repeated.
seed = torch.seed()
print(f"Seed: {seed}")
np.save("seed.npy", seed)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_data_loaders(MODULUS, batch_size=BATCH_SIZE)
model = TransformerFourHead(
    vocab_size=VOCAB_SIZE, d_model=D_MODEL, num_heads=NUM_HEADS, init_std=INIT_STD
).to(device)
optimizer = AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY, betas=BETAS)

print("Model: TransformerFourHead (num_heads=4, Nanda-Unified substrate)")

```

```

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = NUM_EPOCHS
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []
sum_w2_history = []
per_module_sum_w2_history = []
dropout_gap_epochs = []
dropout_rates = [0.1, 0.3, 0.5, 0.7, 0.9]
dropout_gap_history_by_rate = {rate: [] for rate in dropout_rates}
dropout_train_acc_by_rate = {rate: [] for rate in dropout_rates}
dropout_eval_acc_by_rate = {rate: [] for rate in dropout_rates}

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())
    l2_norm_history.append(compute_l2_norm(model))
    sum_w2_history.append(compute_sum_of_squared_weights(model))
    per_module_sum_w2_history.append(compute_per_module_sum_of_squared_weights(model))

    # Dropout Gap Predictor: full multi-rate sweep across all 5 rates
    # (0.1, 0.3, 0.5, 0.7, 0.9). There is no single "primary" rate – every
    # rate is stored, and the analysis works on the whole sweep.
    results = compute_dropout_gap_multi_rate(model, data_loader[1], dropout_rates)

    dropout_gap_epochs.append(epoch + 1)
    for rate in dropout_rates:
        dropout_gap_history_by_rate[rate].append(results[rate]["dropout_gap"])
        dropout_train_acc_by_rate[rate].append(results[rate]["train_accuracy"])
        dropout_eval_acc_by_rate[rate].append(results[rate]["eval_accuracy"])

    # compute_dropout_gap_multi_rate() leaves the model in eval() mode with p
    # reset to 0.0. Training must resume in train() mode, so we restore it here
    # before the next epoch's training pass begins.
    model.train()

    gap_summary = ", ".join(f"p{rate}={results[rate]['dropout_gap']:.3f}" for rate in dropout_rates)
    print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
        {train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
        {l2_norm_history[-1]:.4f}, Sum w^2: {sum_w2_history[-1]:.1f}, Dropout Gap sweep: {gap_summary}")

    # Save all measurements using unified system
    measurements.save_training_data(train_acc_history, test_acc_history, loss_history)

    grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

    # L2 Norm Predictor: Moving Average Crossover Strategy
    print("\n" + "="*60)
    print("L2 NORM PREDICTOR (4-head model): Moving Average Crossover")
    print("="*60)

    epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history, fast_window=50,

```

```

slow_window=200)

detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)

print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f"  MA Crossover epoch: {detection_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {lead_time:.1f} epochs")
else:
    print("  No MA crossover detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

print(f"\nL2 Norm Stats:")
print(f"  Min: {np.min(l2_norm_history):.4f}")
print(f"  Max: {np.max(l2_norm_history):.4f}")
print(f"  Final: {l2_norm_history[-1]:.4f}")

# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model): MA of Slow MA - Zero-Crossing Trigger")
print("="*60)

fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff, skip_epochs=100)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:
    trigger_lead_time = grok_epoch - trigger_epoch
    print(f"  Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {trigger_lead_time:.1f} epochs")
else:
    print("  No trigger detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

# Save all L2 Norm measurements
measurements.save_l2_norm_data(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch,
    sum_w2_history=sum_w2_history,
    per_module_sum_w2_history=per_module_sum_w2_history,
)

# Dropout Predictor: gap tracked at every epoch during training
print("\n" + "="*60)
print("DROPOUT PREDICTOR (4-head model): Gap tracked every epoch")
print("="*60)
print(f"\nFinal Dropout Gap sweep (epoch {dropout_gap_epochs[-1]}):")
for rate in dropout_rates:
    print(f"  rate={rate}: gap={dropout_gap_history_by_rate[rate][-1]:+.4f} "
          f"train p={rate}: {dropout_train_acc_by_rate[rate][-1]:.4f}, "
          f"clean p=0.0: {dropout_eval_acc_by_rate[rate][-1]:.4f}")

# Save all Dropout measurements - full sweep, no single-rate record
measurements.save_dropout_data(
    dropout_gap_epochs, dropout_gap_history_by_rate,
    dropout_train_acc_by_rate, dropout_eval_acc_by_rate, dropout_rates
)

# =====
# VISUALIZATION & REPORT GENERATION
# =====
print("\n" + "="*60)
print("VISUALIZATION & REPORT GENERATION (4-head model)")
print("="*60)

# Generate standalone L2 Norm visualizations
measurements.generate_l2_norm_visualizations(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
    grok_epoch, detection_epoch, trigger_epoch
)
print("[OK] L2 Norm visualizations saved to runs/four_head/run_{}/l2_norm/".format(run_number))

# Generate standalone Dropout visualizations
measurements.generate_dropout_visualizations(
    dropout_gap_epochs, dropout_gap_history_by_rate,
    dropout_rates, test_acc_history, grok_epoch
)

```

```

print("[OK] Dropout visualizations saved to runs/four_head/run_{}/dropout/".format(run_number))

# Generate combined PDF report
report_path = measurements.generate_combined_report(
    train_acc_history, test_acc_history, loss_history, l2_norm_history,
    dropout_gap_epochs, dropout_gap_history_by_rate, dropout_rates,
    epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch
)
print(f"[OK] PDF report saved to {report_path} (4 pages: all measurements)")

print("\n" + "="*60)
print("RUN {} COMPLETE".format(run_number))
print("="*60)
print(f"Grok epoch: {grok_epoch}")
print(f"L2 Norm MA crossover: {detection_epoch if detection_epoch else 'Not detected'}")
print(f"L2 Norm MA-of-MA trigger: {trigger_epoch if trigger_epoch else 'Not detected'}")
print(f"\nAll measurements saved to runs/four_head/run_{run_number}/")
print(f" - training/ : raw training data")
print(f" - l2_norm/ : L2 norm measurements + visualizations")
print(f" - dropout/ : dropout measurements + visualizations")
print(f" - reports/ : combined PDF report")

```

25. 06_Code/src/train_shadow_layernorm.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os

from data.modular_arithmetic import get_dataloaders
from models.model_shadow_with_layernorm import Transformer

# Save results to project root (same pattern as train.py)
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Fixed seed, so this run can later be repeated with the same starting
# weights and the same train/test split. NOTE: train.py itself does not
# currently set any seed. If you want a true apples-to-apples comparison
# against the original model, add this same line to train.py yourself
# before that run too (temporarily) - this script does not modify train.py.
torch.manual_seed(1337)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_dataloaders(97, batch_size=int(0.3 * 97 * 97))
model = Transformer(vocab_size=98, d_model=128).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 10000
train_acc_history = []
test_acc_history = []
loss_history = []

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())

    if (epoch + 1) % 100 == 0 or epoch == 0:
        print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, "
              f"Train Accuracy: {train_acc_history[-1]:.4f}, "
              f"Test Accuracy: {test_acc_history[-1]:.4f}")

# Saved under a "shadow_ln" prefix on purpose, so these files never
# overwrite train.py's own train_acc_history.npy / test_acc_history.npy /
# loss_history.npy, which the L2 Norm and Dropout predictors depend on.
np.save("shadow_ln_train_acc_history.npy", train_acc_history)
np.save("shadow_ln_test_acc_history.npy", test_acc_history)
np.save("shadow_ln_loss_history.npy", loss_history)

final_test_acc = test_acc_history[-1]
test_acc_array = np.array(test_acc_history)
grok_epoch = int(np.argmax(test_acc_array > 0.9)) if (test_acc_array > 0.9).any() else None
print("\n" + "=" * 60)
```

```
print("SHADOW MODEL (LayerNorm) RESULT")
print("=" * 60)
print(f"Final Test Accuracy: {final_test_acc:.4f}")
print(f"Grok epoch (test acc > 90%): {grok_epoch}")
print("Compare this Final Test Accuracy against the original model's known")
print("plateau value: 0.995446 (99.5446%) - see context.md, Aug 7 session.")
```

26. 06_Code/src/unified_measurements.py

```
"""
Unified measurement collection for L2 Norm and Dropout predictors.
Ensures consistent, complete measurements across single-head and four-head models.
"""

import numpy as np
import os
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

class PredictorMeasurements:
    """Collects and saves all measurements for both predictors."""

    def __init__(self, output_dir, model_type="single_head"):
        self.output_dir = output_dir
        self.model_type = model_type
        self.create_subdirs()

    def create_subdirs(self):
        """Create subdirectories for each predictor."""
        self.l2_norm_dir = os.path.join(self.output_dir, "l2_norm")
        self.dropout_dir = os.path.join(self.output_dir, "dropout")
        self.spectral_dir = os.path.join(self.output_dir, "spectral")
        self.age_dir = os.path.join(self.output_dir, "age")
        self.reports_dir = os.path.join(self.output_dir, "reports")
        self.training_dir = os.path.join(self.output_dir, "training")

        for dir_path in [self.l2_norm_dir, self.dropout_dir, self.spectral_dir,
                        self.age_dir, self.reports_dir, self.training_dir]:
            os.makedirs(dir_path, exist_ok=True)

    def save_training_data(self, train_acc, test_acc, loss):
        """Save core training metrics."""
        np.save(os.path.join(self.training_dir, "train_acc_history.npy"), train_acc)
        np.save(os.path.join(self.training_dir, "test_acc_history.npy"), test_acc)
        np.save(os.path.join(self.training_dir, "loss_history.npy"), loss)

# ===== L2 NORM MEASUREMENTS =====

def save_l2_norm_data(self, l2_norm_history, epoch_grid, fast_ma, slow_ma,
                    fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch=None,
                    sum_w2_history=None, per_module_sum_w2_history=None):
    """Save all L2 Norm measurements.

    sum_w2_history: optional per-epoch list of sum-of-squared-weights
        (the Nanda Figure 7 quantity; == l2_norm_history ** 2).
    per_module_sum_w2_history: optional per-epoch list of dicts
        {group_name: sum_w2} from
        predictors.l2_norm.compute_per_module_sum_of_squared_weights.
        Saved as per_module_sum_w2.npy shaped [n_groups, n_epochs] with
        the row order recorded in per_module_sum_w2_names.npy.
    """

    # Raw L2 norm
    np.save(os.path.join(self.l2_norm_dir, "l2_norm_history.npy"), l2_norm_history)

    # sum w^2 (Nanda Fig 7 quantity) – total and per module
    if sum_w2_history is not None:
        np.save(os.path.join(self.l2_norm_dir, "sum_w2_history.npy"),
                np.asarray(sum_w2_history, dtype=float))
    if per_module_sum_w2_history:
        group_names = list(per_module_sum_w2_history[0].keys())
        per_module_matrix = np.array(
            [[row[g] for row in per_module_sum_w2_history] for g in group_names],
            dtype=float,
        )
        np.save(os.path.join(self.l2_norm_dir, "per_module_sum_w2.npy"), per_module_matrix)
        np.save(os.path.join(self.l2_norm_dir, "per_module_sum_w2_names.npy"),
                np.array(group_names))

    # Smoothed L2 norm (simple low-pass filter)
    l2_norm_smoothed = self._smooth_signal(l2_norm_history, window=50)
    np.save(os.path.join(self.l2_norm_dir, "l2_norm_smoothed.npy"), l2_norm_smoothed)

    # Moving averages
    np.save(os.path.join(self.l2_norm_dir, "epoch_grid.npy"), epoch_grid)
    np.save(os.path.join(self.l2_norm_dir, "fast_ma.npy"), fast_ma)
    np.save(os.path.join(self.l2_norm_dir, "slow_ma.npy"), slow_ma)
    np.save(os.path.join(self.l2_norm_dir, "fast_ma_of_slow_ma.npy"), fast_ma_of_slow_ma)
    np.save(os.path.join(self.l2_norm_dir, "ma_of_ma_diff.npy"), ma_of_ma_diff)
    # Acceleration derivatives
```

```

accel_raw = np.diff(l2_norm_history)
accel_smoothed = self._smooth_signal(accel_raw, window=50)
accel_double_smoothed = self._smooth_signal(accel_smoothed, window=50)

np.save(os.path.join(self.l2_norm_dir, "acceleration_raw.npy"), accel_raw)
np.save(os.path.join(self.l2_norm_dir, "acceleration_smoothed.npy"), accel_smoothed)
np.save(os.path.join(self.l2_norm_dir, "acceleration_double_smoothed.npy"), accel_double_smoothed)

if detection_epoch is not None:
    np.save(os.path.join(self.l2_norm_dir, "detection_epoch.npy"), np.array([detection_epoch]))

# ===== DROPOUT MEASUREMENTS =====

def save_dropout_data(self, dropout_gap_epochs, dropout_gap_history_by_rate,
                      dropout_train_acc_by_rate, dropout_eval_acc_by_rate, dropout_rates):
    """Save all Dropout measurements – full multi-rate sweep only.

    There is no single-rate (p=0.9) record any more. Every array is
    indexed [rate_index, epoch_index], with rate_index following the
    order of dropout_rates.npy.
    """

    np.save(os.path.join(self.dropout_dir, "dropout_gap_epochs.npy"), dropout_gap_epochs)
    np.save(os.path.join(self.dropout_dir, "dropout_rates.npy"), np.array(dropout_rates))

    gap_by_rate = np.array([dropout_gap_history_by_rate[r] for r in dropout_rates])
    train_acc_by_rate = np.array([dropout_train_acc_by_rate[r] for r in dropout_rates])
    eval_acc_by_rate = np.array([dropout_eval_acc_by_rate[r] for r in dropout_rates])

    np.save(os.path.join(self.dropout_dir, "dropout_gap_by_rate.npy"), gap_by_rate)
    np.save(os.path.join(self.dropout_dir, "dropout_train_acc_by_rate.npy"), train_acc_by_rate)
    np.save(os.path.join(self.dropout_dir, "dropout_eval_acc_by_rate.npy"), eval_acc_by_rate)

# ===== SPECTRAL MEASUREMENTS =====

def save_spectral_data(self, spectral_checkpoints, spectral_history):
    """Save all Spectral (Predictor 3) measurements – Canatar et al.
    2021 task-model-alignment version (NOT the old per-weight-matrix
    stable_rank version, which has been removed).

    spectral_checkpoints: list/array of epoch indices, one per
        checkpoint that was evaluated (same role as
        dropout_variance_checkpoints.npy).
    spectral_history: dict of per-checkpoint lists, all the same
        length as spectral_checkpoints:
        k_90 : list[int]
        k_95 : list[int]
        alignment_score : list[float]
        entropy : list[float]
        eigenvalues_history: list[list[float]] (top-50 eta_k per checkpoint)
        cumulative_power_history: list[list[float]] (C(1..100) per checkpoint)
    built by calling
    predictors.spectral.compute_spectral_metrics_for_checkpoint at
    each checkpoint and collecting each returned field.

    Saves (all under self.spectral_dir):
    spectral_checkpoints.npy [n_checkpoints]
    spectral_k90.npy [n_checkpoints]
    spectral_k95.npy [n_checkpoints]
    spectral_alignment.npy [n_checkpoints]
    spectral_entropy.npy [n_checkpoints]
    spectral_eigenvalues.npy [n_checkpoints, 50]
    spectral_cumulative_power.npy [n_checkpoints, 100]
    """

    np.save(os.path.join(self.spectral_dir, "spectral_checkpoints.npy"),
            np.array(spectral_checkpoints, dtype=int))
    np.save(os.path.join(self.spectral_dir, "spectral_k90.npy"),
            np.array(spectral_history["k_90"], dtype=float))
    np.save(os.path.join(self.spectral_dir, "spectral_k95.npy"),
            np.array(spectral_history["k_95"], dtype=float))
    np.save(os.path.join(self.spectral_dir, "spectral_alignment.npy"),
            np.array(spectral_history["alignment_score"], dtype=float))
    np.save(os.path.join(self.spectral_dir, "spectral_entropy.npy"),
            np.array(spectral_history["entropy"], dtype=float))

    eig = self._stack_ragged(spectral_history["eigenvalues_history"], width=50)
    np.save(os.path.join(self.spectral_dir, "spectral_eigenvalues.npy"), eig)

    cum = self._stack_ragged(spectral_history["cumulative_power_history"], width=100)
    np.save(os.path.join(self.spectral_dir, "spectral_cumulative_power.npy"), cum)

# ===== AGE MEASUREMENTS =====

def save_age_data(self, age_checkpoints, age_history):
    """Save all AGE (Predictor 4) measurements – Papyan et al. 2020 NC1
    variability-collapse version (see src/predictors/age.py). AGE is a

```

checkpoint-only, loss-agnostic signal, so its arrays follow the same per-checkpoint layout as save_spectral_data (NOT a per-epoch history like L2 Norm).

age_checkpoints: list/array of epoch indices, one per checkpoint that was evaluated (same role as spectral_checkpoints.npy).
age_history: dict of per-checkpoint lists, all the same length as age_checkpoints:
 nc1 : list[float] Tr(Sigma_W) / Tr(Sigma_B) per checkpoint
 fn : list[float] mean feature norm per checkpoint
built by calling
predictors.age.compute_age_metrics_for_checkpoint at each
checkpoint and collecting each returned field.

```
Saves (all under self.age_dir):
age_checkpoints.npy   [n_checkpoints]
age_nc1.npy           [n_checkpoints]
age_fn.npy           [n_checkpoints]
"""
np.save(os.path.join(self.age_dir, "age_checkpoints.npy"),
        np.array(age_checkpoints, dtype=int))
np.save(os.path.join(self.age_dir, "age_nc1.npy"),
        np.array(age_history["nc1"], dtype=float))
np.save(os.path.join(self.age_dir, "age_fn.npy"),
        np.array(age_history["fn"], dtype=float))
```

```
@staticmethod
def _stack_ragged(list_of_lists, width):
    """[n_checkpoints] lists of <= `width` floats -> float array
    [n_checkpoints, width], right-padded with the last value (NaN if a
    row is empty). For the Nanda-Unified run every row is already
    exactly `width` long (N = 3830 & width), so this is just a guard."""
    n = len(list_of_lists)
    out = np.full((n, width), np.nan, dtype=float)
    for i, row in enumerate(list_of_lists):
        row = np.asarray(row, dtype=float)
        m = min(len(row), width)
        out[i, :m] = row[:m]
        if 0 < m < width:
            out[i, m:] = row[m - 1]
    return out
```

===== VISUALIZATION GENERATION =====

```
def generate_l2_norm_visualizations(self, l2_norm_history, epoch_grid, fast_ma, slow_ma,
                                     fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
                                     grok_epoch, detection_epoch=None, trigger_epoch=None):
    """Generate standalone L2 Norm visualization graphs."""

    num_epochs = len(l2_norm_history)
    epochs_axis = range(1, num_epochs + 1)

    # 1. L2 Norm curve
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2)
    if detection_epoch is not None:
        ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                  label=f"MA Crossover (epoch {detection_epoch:.0f})")
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch (epoch {grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("L2 Norm")
    ax.set_title("L2 Norm of Model Weights")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.l2_norm_dir, "l2_norm_curve.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)

    # 2. MA crossover detection
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epoch_grid, fast_ma, label="Fast MA (window=50)", linewidth=2, color="blue")
    ax.plot(epoch_grid, slow_ma, label="Slow MA (window=200)", linewidth=2, color="orange")
    if detection_epoch is not None:
        ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                  label=f"Crossover (epoch {detection_epoch:.0f})")
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch (epoch {grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("L2 Norm")
    ax.set_title("Moving Average Crossover Detection")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_slow_ma_crossover.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)
```

```

# 3. MA-of-MA differential
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epoch_grid, ma_of_ma_diff, color="darkred", linewidth=2, label="MA-of-MA Differential")
ax.axhline(y=0, color="black", linestyle="--", linewidth=0.5, alpha=0.5)
if trigger_epoch is not None:
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2,
               label=f"Zero-crossing (epoch {trigger_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok epoch (epoch {grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Differential")
ax.set_title("MA-of-MA Differential: Zero-Crossing Trigger")
ax.legend()
ax.grid(True, alpha=0.3)
plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_ma_diff.png"), dpi=150, bbox_inches='tight')
plt.close(fig)

# 4. MA-of-MA differential vs grokking (linear scale)
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epoch_grid, ma_of_ma_diff, color="darkred", linewidth=2, label="MA-of-MA Differential")
ax.axhline(y=0, color="black", linestyle="--", linewidth=0.5, alpha=0.5)
if trigger_epoch is not None:
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2,
               label=f"Zero-crossing (epoch {trigger_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok epoch (epoch {grok_epoch})")
ax.set_xscale("linear")
ax.set_xlabel("Epoch (linear scale)")
ax.set_ylabel("Differential")
ax.set_title("MA-of-MA Differential: Linear Scale")
ax.legend()
ax.grid(True, alpha=0.3)
plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_ma_diff_linear.png"), dpi=150, bbox_inches='tight')
plt.close(fig)

def generate_dropout_visualizations(self, dropout_gap_epochs, dropout_gap_history_by_rate,
                                   dropout_rates, test_acc_history, grok_epoch):
    """Generate standalone Dropout visualization graphs."""

    # Multi-rate sweep: all 5 rates on one plot
    fig, ax = plt.subplots(figsize=(12, 7))
    for rate in dropout_rates:
        ax.plot(
            dropout_gap_epochs,
            dropout_gap_history_by_rate[rate],
            linewidth=2,
            label=f"rate={rate}"
        )
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch ({grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("Dropout Gap")
    ax.set_title("Dropout Gap: Multi-Rate Sweep (p=0.1, 0.3, 0.5, 0.7, 0.9)")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.dropout_dir, "dropout_gap_curve.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)

def generate_combined_report(self, train_acc, test_acc, loss, l2_norm_history,
                             dropout_gap_epochs, dropout_gap_history_by_rate, dropout_rates,
                             epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch=None):
    """Generate combined PDF report with all measurements."""

    num_epochs = len(train_acc)
    epochs_axis = range(1, num_epochs + 1)

    with PdfPages(os.path.join(self.reports_dir, "training_report.pdf")) as pdf:

        # Page 1: Grokking curve
        fig, ax = plt.subplots(figsize=(12, 7))
        ax.plot(epochs_axis, train_acc, label="Train Accuracy", color="steelblue", linewidth=2)
        ax.plot(epochs_axis, test_acc, label="Test Accuracy", color="seagreen", linewidth=2)
        ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
                  label=f"Grok epoch ({grok_epoch})")
        ax.set_xscale("log")
        ax.set_xlabel("Epoch (log scale)")
        ax.set_ylabel("Accuracy")
        ax.set_title("Grokking Curve: Train vs. Test Accuracy")
        ax.legend()
        ax.grid(True, alpha=0.3)
        pdf.savefig(fig)
        plt.close(fig)

        # Page 2: Loss curve

```

```

fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epochs_axis, loss, color="darkorange", linewidth=2)
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Loss")
ax.set_title("Training Loss")
ax.grid(True, alpha=0.3)
pdf.savefig(fig)
plt.close(fig)

# Page 3: L2 Norm with MA crossover
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
ax.plot(epoch_grid, fast_ma, label="Fast MA", linewidth=2, color="blue", alpha=0.7)
ax.plot(epoch_grid, slow_ma, label="Slow MA", linewidth=2, color="orange", alpha=0.7)
if detection_epoch is not None:
    ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
               label=f"Crossover ({detection_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok ({grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("L2 Norm")
ax.set_title("L2 Norm & Moving Averages")
ax.legend()
ax.grid(True, alpha=0.3)
pdf.savefig(fig)
plt.close(fig)

# Page 4: Dropout Gap (multi-rate)
fig, ax = plt.subplots(figsize=(12, 7))
for rate in dropout_rates:
    ax.plot(
        dropout_gap_epochs,
        dropout_gap_history_by_rate[rate],
        linewidth=2,
        label=f"rate={rate}"
    )
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok ({grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Dropout Gap")
ax.set_title("Dropout Gap: Multi-Rate Sweep")
ax.legend()
ax.grid(True, alpha=0.3)
pdf.savefig(fig)
plt.close(fig)

return os.path.join(self.reports_dir, "training_report.pdf")

# ===== UTILITIES =====

@staticmethod
def _smooth_signal(signal, window=50):
    """Apply simple moving average smoothing."""
    if window > len(signal):
        window = len(signal)
    return np.convolve(signal, np.ones(window)/window, mode='same')

```

27. 08_Experiments/nanda_l2_p113/___init___.py

(empty file)

28. 08_Experiments/nanda_l2_p113/data/__init__.py

(empty file)

29. 08_Experiments/nanda_l2_p113/data/modular_arithmetic.py

```
from torch import long, tensor
from torch.utils.data import Dataset, DataLoader, random_split

# =====
# Copy of src/data/modular_arithmetic.py for the nanda_l2_p113/
# experiment. Only one change from the src/ copy: the "=" token id is no
# longer hardcoded as 97. It is now the modulus itself (self.number), so
# that passing number=113 gives the "=" token id 113 – matching the main
# experiment's convention that number tokens are 0..p-1 and the "=" token
# id is p.
# =====

def generate_pairs(number):
    pairs = []
    for i in range(number):
        for j in range(number):
            pairs.append((i, j, (i + j) % number))
    return pairs

def get_dataloaders(number, batch_size):
    modular_arithmetic_dataset = ModularArithmeticDataset(number)
    train_size = int(0.3 * len(modular_arithmetic_dataset))
    test_size = len(modular_arithmetic_dataset) - train_size

    train_dataset, test_dataset = random_split(modular_arithmetic_dataset, [train_size, test_size])
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_dataloader = DataLoader(test_dataset, batch_size=batch_size)

    return train_dataloader, test_dataloader

class ModularArithmeticDataset(Dataset):
    def __init__(self, number):
        self.number = number
        self.pairs = generate_pairs(number)

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        return (self.get_tensor(self.pairs[idx]), self.pairs[idx][2])

    def get_tensor(self, item):
        # item is (a, b, answer). The third sequence position is the "="
        # token, whose id is the modulus (113 for this experiment).
        sequence = [item[0], item[1], self.number]
        return tensor(sequence)

if __name__ == "__main__":
    print(tensor([5, 3, 8, 113]))
```

30.08_Experiments/nanda_l2_p113/measurements.py

```
"""
Measurement collection for the L2 Norm predictor only.

This is a trimmed copy of src/unified_measurements.py. The main experiment's
version also collects Dropout predictor data and builds a Dropout page in the
combined report. This experiment (nanda_l2_p113) has no Dropout predictor, so
every Dropout member has been removed:
    * no dropout_dir
    * no save_dropout_data()
    * no generate_dropout_visualizations()
    * generate_combined_report() has only the training + L2 Norm pages

Everything L2-Norm and training related is kept identical to the src/ copy.
"""

import numpy as np
import os
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

class PredictorMeasurements:
    """Collects and saves all measurements for the L2 Norm predictor."""

    def __init__(self, output_dir, model_type="four_head"):
        self.output_dir = output_dir
        self.model_type = model_type
        self.create_subdirs()

    def create_subdirs(self):
        """Create subdirectories for the predictor outputs."""
        self.l2_norm_dir = os.path.join(self.output_dir, "l2_norm")
        self.reports_dir = os.path.join(self.output_dir, "reports")
        self.training_dir = os.path.join(self.output_dir, "training")

        for dir_path in [self.l2_norm_dir, self.reports_dir, self.training_dir]:
            os.makedirs(dir_path, exist_ok=True)

    def save_training_data(self, train_acc, test_acc, loss):
        """Save core training metrics."""
        np.save(os.path.join(self.training_dir, "train_acc_history.npy"), train_acc)
        np.save(os.path.join(self.training_dir, "test_acc_history.npy"), test_acc)
        np.save(os.path.join(self.training_dir, "loss_history.npy"), loss)

    # ===== L2 NORM MEASUREMENTS =====

    def save_l2_norm_data(self, l2_norm_history, epoch_grid, fast_ma, slow_ma,
                          fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch=None):
        """Save all L2 Norm measurements."""

        # Raw L2 norm
        np.save(os.path.join(self.l2_norm_dir, "l2_norm_history.npy"), l2_norm_history)

        # Smoothed L2 norm (simple low-pass filter)
        l2_norm_smoothed = self._smooth_signal(l2_norm_history, window=50)
        np.save(os.path.join(self.l2_norm_dir, "l2_norm_smoothed.npy"), l2_norm_smoothed)

        # Moving averages
        np.save(os.path.join(self.l2_norm_dir, "epoch_grid.npy"), epoch_grid)
        np.save(os.path.join(self.l2_norm_dir, "fast_ma.npy"), fast_ma)
        np.save(os.path.join(self.l2_norm_dir, "slow_ma.npy"), slow_ma)
        np.save(os.path.join(self.l2_norm_dir, "fast_ma_of_slow_ma.npy"), fast_ma_of_slow_ma)
        np.save(os.path.join(self.l2_norm_dir, "ma_of_ma_diff.npy"), ma_of_ma_diff)

        # Acceleration derivatives
        accel_raw = np.diff(l2_norm_history)
        accel_smoothed = self._smooth_signal(accel_raw, window=50)
        accel_double_smoothed = self._smooth_signal(accel_smoothed, window=50)

        np.save(os.path.join(self.l2_norm_dir, "acceleration_raw.npy"), accel_raw)
        np.save(os.path.join(self.l2_norm_dir, "acceleration_smoothed.npy"), accel_smoothed)
        np.save(os.path.join(self.l2_norm_dir, "acceleration_double_smoothed.npy"), accel_double_smoothed)

        if detection_epoch is not None:
            np.save(os.path.join(self.l2_norm_dir, "detection_epoch.npy"), np.array([detection_epoch]))

    # ===== VISUALIZATION GENERATION =====

    def generate_l2_norm_visualizations(self, l2_norm_history, epoch_grid, fast_ma, slow_ma,
                                       fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
                                       grok_epoch, detection_epoch=None, trigger_epoch=None):
        """Generate standalone L2 Norm visualization graphs."""
```

```

num_epochs = len(l2_norm_history)
epochs_axis = range(1, num_epochs + 1)

# 1. L2 Norm curve
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2)
if detection_epoch is not None:
    ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
               label=f"MA Crossover (epoch {detection_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok epoch (epoch {grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("L2 Norm")
ax.set_title("L2 Norm of Model Weights")
ax.legend()
ax.grid(True, alpha=0.3)
plt.savefig(os.path.join(self.l2_norm_dir, "l2_norm_curve.png"), dpi=150, bbox_inches='tight')
plt.close(fig)

# 2. MA crossover detection
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epoch_grid, fast_ma, label="Fast MA (window=50)", linewidth=2, color="blue")
ax.plot(epoch_grid, slow_ma, label="Slow MA (window=200)", linewidth=2, color="orange")
if detection_epoch is not None:
    ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
               label=f"Crossover (epoch {detection_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok epoch (epoch {grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("L2 Norm")
ax.set_title("Moving Average Crossover Detection")
ax.legend()
ax.grid(True, alpha=0.3)
plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_slow_ma_crossover.png"), dpi=150, bbox_inches='tight')
plt.close(fig)

# 3. MA-of-MA differential
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epoch_grid, ma_of_ma_diff, color="darkred", linewidth=2, label="MA-of-MA Differential")
ax.axhline(y=0, color="black", linestyle="--", linewidth=0.5, alpha=0.5)
if trigger_epoch is not None:
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2,
               label=f"Zero-crossing (epoch {trigger_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok epoch (epoch {grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Differential")
ax.set_title("MA-of-MA Differential: Zero-Crossing Trigger")
ax.legend()
ax.grid(True, alpha=0.3)
plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_ma_diff.png"), dpi=150, bbox_inches='tight')
plt.close(fig)

# 4. MA-of-MA differential vs grokking (linear scale)
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epoch_grid, ma_of_ma_diff, color="darkred", linewidth=2, label="MA-of-MA Differential")
ax.axhline(y=0, color="black", linestyle="--", linewidth=0.5, alpha=0.5)
if trigger_epoch is not None:
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2,
               label=f"Zero-crossing (epoch {trigger_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
           label=f"Grok epoch (epoch {grok_epoch})")
ax.set_xscale("linear")
ax.set_xlabel("Epoch (linear scale)")
ax.set_ylabel("Differential")
ax.set_title("MA-of-MA Differential: Linear Scale")
ax.legend()
ax.grid(True, alpha=0.3)
plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_ma_diff_linear.png"), dpi=150, bbox_inches='tight')
plt.close(fig)

def generate_combined_report(self, train_acc, test_acc, loss, l2_norm_history,
                             epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch=None):
    """Generate combined PDF report: training + L2 Norm pages only."""

    num_epochs = len(train_acc)
    epochs_axis = range(1, num_epochs + 1)

    with PdfPages(os.path.join(self.reports_dir, "training_report.pdf")) as pdf:

        # Page 1: Grokking curve
        fig, ax = plt.subplots(figsize=(12, 7))
        ax.plot(epochs_axis, train_acc, label="Train Accuracy", color="steelblue", linewidth=2)

```

```

ax.plot(epochs_axis, test_acc, label="Test Accuracy", color="seagreen", linewidth=2)
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
          label=f"Grok epoch ({grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Accuracy")
ax.set_title("Grokking Curve: Train vs. Test Accuracy")
ax.legend()
ax.grid(True, alpha=0.3)
pdf.savefig(fig)
plt.close(fig)

# Page 2: Loss curve
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epochs_axis, loss, color="darkorange", linewidth=2)
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Loss")
ax.set_title("Training Loss")
ax.grid(True, alpha=0.3)
pdf.savefig(fig)
plt.close(fig)

# Page 3: L2 Norm with MA crossover
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
ax.plot(epoch_grid, fast_ma, label="Fast MA", linewidth=2, color="blue", alpha=0.7)
ax.plot(epoch_grid, slow_ma, label="Slow MA", linewidth=2, color="orange", alpha=0.7)
if detection_epoch is not None:
    ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
              label=f"Crossover ({detection_epoch:.0f})")
ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
          label=f"Grok ({grok_epoch})")
ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("L2 Norm")
ax.set_title("L2 Norm & Moving Averages")
ax.legend()
ax.grid(True, alpha=0.3)
pdf.savefig(fig)
plt.close(fig)

return os.path.join(self.reports_dir, "training_report.pdf")

# ===== UTILITIES =====

@staticmethod
def _smooth_signal(signal, window=50):
    """Apply simple moving average smoothing."""
    if window > len(signal):
        window = len(signal)
    return np.convolve(signal, np.ones(window)/window, mode='same')

```

31. 08_Experiments/nanda_l2_p113/models/__init__.py

(empty file)

32. 08_Experiments/nanda_l2_p113/models/transformer_four_head.py

```
from torch import arange, nn
import torch

# =====
# 4-HEAD VARIANT - matches Nanda et al.'s actual architecture
# (d_model=128, 4 attention heads, head dimension = 128/4 = 32 each).
#
# This started as a copy of src/models/transformer_four_head.py. It lives
# here so the nanda_l2_p113/ experiment is fully self-contained and does
# not import anything from src/.
#
# Two changes from the src/ copy:
#
# 1. The __main__ demo vocab_size is 114 (p = 113: number tokens 0..112
#    plus one "=" token with id 113).
#
# 2. WEIGHT INITIALISATION. src/ uses PyTorch defaults - nn.Embedding is
#    N(0, 1) (std = 1, very large; the token-embedding table alone is
#    ~94% of the starting weight norm) and nn.Linear is Kaiming-uniform.
#    Nanda et al. use TransformerLens, which initialises EVERY matrix
#    (embeddings included) as N(0, std) with std = 0.8 / sqrt(d_model)
#    (~0.0707 for d_model = 128). _init_weights() below applies that
#    scheme to every parameter, so this experiment starts from small
#    random weights like Nanda's - the model then has to grow its
#    weights to memorise, instead of collapsing an oversized init.
#
# Architecture is otherwise identical to the main experiment: same token +
# position embedding, same MLP (4x expansion + ReLU), no LayerNorm, no
# Linear biases, same dropout1/dropout2 hooks (kept at p=0.0 here - this
# experiment has no Dropout predictor, but the hooks are harmless and
# keeping them keeps the class identical to src/), same output head.
# =====

class TransformerFourHead(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads=4, init_std=None):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        # TransformerLens / Nanda init scale: N(0, 0.8 / sqrt(d_model)).
        if init_std is None:
            init_std = 0.8 / (d_model ** 0.5)
        self.init_std = init_std

        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

        self._init_weights()

    def _init_weights(self):
        # Match Nanda et al. / TransformerLens: every weight matrix,
        # embeddings included, drawn from N(0, init_std). PyTorch's own
        # defaults (nn.Embedding = N(0, 1), nn.Linear = Kaiming-uniform)
        # would otherwise make the token-embedding table ~94% of the
        # starting weight norm and force a large early collapse under
        # weight decay that has nothing to do with grokking.
        for module in [self.token_embedding, self.position_embedding,
                        self.query, self.key, self.value,
                        self.mlp_in, self.mlp_out, self.output_head]:
            nn.init.normal_(module.weight, mean=0.0, std=self.init_std)

    def split_into_heads(self, x):
        # x: (batch_size, seq_len, d_model) -> (batch_size, num_heads, seq_len, head_dim)
        # This is the step that turns one big d_model-wide attention into
        # num_heads smaller, independent attention computations running
        # side by side.
        batch_size, seq_len, d_model = x.shape
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.permute(0, 2, 1, 3)
```

```

def combine_heads(self, x):
    # x: (batch_size, num_heads, seq_len, head_dim) -> (batch_size, seq_len, d_model)
    # Reverses split_into_heads - puts all heads' outputs back side by
    # side into one d_model-wide vector, exactly as Nanda et al.
    # describe combining the 4 heads' outputs before the MLP.
    batch_size, num_heads, seq_len, head_dim = x.shape
    x = x.permute(0, 2, 1, 3)
    return x.reshape(batch_size, seq_len, num_heads * head_dim)

def forward(self, x):
    token_vectors = self.token_embedding(x)
    position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
    combined_vector = token_vectors + position_vectors

    query_vector = self.split_into_heads(self.query(combined_vector))
    key_vector = self.split_into_heads(self.key(combined_vector))
    value_vector = self.split_into_heads(self.value(combined_vector))

    # Scaling uses head_dim (32), not the full d_model (128) - each
    # head does its own scaled dot-product attention independently.
    scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (self.head_dim ** 0.5)
    attention_weights = torch.softmax(scores, dim=-1)
    attention_output = torch.matmul(attention_weights, value_vector)
    attention_output = self.combine_heads(attention_output)

    attention_values = combined_vector + self.dropout1(attention_output)
    mlp_output = attention_values +
        self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))

    logits = self.output_head(mlp_output)

    return logits

if __name__ == "__main__":
    # p = 113 -> vocab_size = 113 + 1 = 114
    model = TransformerFourHead(vocab_size=114, d_model=128, num_heads=4)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

```

33. 08_Experiments/nanda_l2_p113/predictors/__init__.py

(empty file)

34. 08_Experiments/nanda_l2_p113/predictors/l2_norm.py

```
import numpy as np
import torch
from scipy.ndimage import uniform_filter1d

def compute_l2_norm(model):
    all_params = torch.cat([p.flatten() for p in model.parameters()])
    l2_norm = torch.norm(all_params).item()
    return l2_norm

def compute_l2_norm_rate_of_decline(l2_norm_history):
    return np.diff(l2_norm_history) * -1

def detect_l2_norm_drop(rate_of_decline, threshold=0.05, skip_epochs=200):
    for i, rate in enumerate(rate_of_decline):
        if i < skip_epochs: # Skip early epochs
            continue
        if rate > threshold:
            return i
    return None

def compute_acceleration(rate_of_decline):
    """
    Compute the second derivative of L2 norm (acceleration).
    acceleration[i] = rate[i] - rate[i-1]
    Positive = decay rate is increasing (speeding up)
    Negative = decay rate is decreasing (slowing down)
    Sign change = inflection point (change in decay character)
    """
    acceleration = np.diff(rate_of_decline)
    return acceleration

def detect_inflection(acceleration, skip_epochs=200):
    """
    Detect inflection point where acceleration changes sign.
    This is where the decay pattern changes character – exactly what happens
    before/during the grok transition.
    Returns the epoch index where the sign flip occurs, or None.
    """
    if len(acceleration) <= skip_epochs:
        return None

    # Check from skip_epochs onwards for sign changes
    for i in range(skip_epochs, len(acceleration)):
        # acceleration[i-1] * acceleration[i] < 0 means opposite signs (sign flip)
        if i > 0 and acceleration[i-1] * acceleration[i] < 0:
            return i

    return None

def apply_moving_average(data, window_size=50):
    """
    Apply moving average smoothing to reduce noise.
    window_size: number of epochs to average over (default 50).
    Returns smoothed array (same length as input, padded at edges).
    """
    return uniform_filter1d(np.array(data), size=window_size, mode='nearest')

def resample_to_log_uniform_grid(data, num_points=None):
    """
    Resample data onto a grid uniformly spaced in log10(epoch) – i.e. the
    SAME spacing the data has when viewed on a log-x plot.
    Without this, a fixed window_size (in raw epoch index) covers a huge
    visual chunk of the plot near epoch 1 and a tiny sliver near epoch N,
    which is why smoothing looked over-smoothed early and scribbly late.
    Returns (epoch_grid, resampled_data) where epoch_grid is uniform in
    log10-space (non-uniform in linear epoch terms).
    """
    data = np.array(data)
    num_epochs = len(data)
    epochs = np.arange(1, num_epochs + 1)
    log_epochs = np.log10(epochs)

    if num_points is None:
        num_points = num_epochs

    log_grid = np.linspace(log_epochs[0], log_epochs[-1], num_points)
    resampled = np.interp(log_grid, log_epochs, data)
    epoch_grid = 10 ** log_grid
    return epoch_grid, resampled
```

```

def compute_fast_slow_moving_averages(l2_norm_history, fast_window=50, slow_window=200, num_points=None):
    """
    Compute two moving averages on a LOG-EPOCH-UNIFORM grid, so the window
    covers equal VISUAL width everywhere on a log-x plot (fixes both the
    over-smoothing near epoch 1 and the scribbling near epoch N).

    Steps:
    1. Log-transform L2 norm values (handles the exponential decay in y).
    2. Resample onto a grid uniform in log10(epoch) (handles the log-x axis).
    3. Apply the moving average on that resampled grid.
    4. Convert back to linear L2 norm space.

    fast_window / slow_window are now measured in grid points, which (since
    the grid is log-uniform) corresponds to a constant *proportional* width
    in epochs everywhere along the curve.

    Returns (epoch_grid, fast_ma, slow_ma) – always plot fast_ma/slow_ma
    against epoch_grid, not against range(1, num_epochs+1).
    """
    l2_norm_history = np.array(l2_norm_history)
    log_l2_norm = np.log(l2_norm_history)

    epoch_grid, log_l2_norm_resampled = resample_to_log_uniform_grid(log_l2_norm, num_points=num_points)

    fast_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=fast_window)
    slow_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=slow_window)

    fast_ma = np.exp(fast_ma_log)
    slow_ma = np.exp(slow_ma_log)

    return epoch_grid, fast_ma, slow_ma


def detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100):
    """
    Detect where fast MA crosses above slow MA, searching along the
    log-uniform grid. skip_epochs is a real epoch value (not a grid index) –
    crossovers before this epoch are ignored (initialization noise).
    Returns the real epoch (float) of the crossover, or None.
    """
    if len(fast_ma) != len(slow_ma):
        return None

    for i in range(len(fast_ma) - 1):
        if epoch_grid[i] <= skip_epochs:
            continue
        # Previous: fast <= slow, Current: fast > slow (crossover from below)
        if fast_ma[i] <= slow_ma[i] and fast_ma[i + 1] > slow_ma[i + 1]:
            return epoch_grid[i + 1]

    return None


def compute_ma_of_slow_ma(slow_ma, fast_window=20):
    """
    Apply a second, faster-window moving average on top of the already-smoothed
    slow_ma. Since slow_ma has already filtered out raw L2 norm noise, this
    reveals slow_ma's OWN turning points cleanly – the difference between the
    two tracks how sharply slow_ma is currently bending.
    Returns (fast_ma_of_slow_ma, diff) where diff = fast_ma_of_slow_ma - slow_ma.
    """
    fast_ma_of_slow_ma = apply_moving_average(slow_ma, window_size=fast_window)
    diff = fast_ma_of_slow_ma - slow_ma
    return fast_ma_of_slow_ma, diff


def compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=90):
    """
    Estimate the normal noise level of the MA-of-MA difference from the quiet
    early-training region (before any real dynamics begin), so later spikes
    can be judged against what "flat" actually looks like for this run.
    """
    quiet_mask = epoch_grid < quiet_epoch_cutoff
    return np.std(diff[quiet_mask])


def detect_ma_of_ma_trigger(epoch_grid, diff, noise_floor, threshold_multiplier=10, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff climbs past
    threshold_multiplier x noise_floor – the rising edge of a real signal.
    Causal: only needs the noise floor (known early) and points seen so far,
    unlike picking the curve's peak, which requires seeing the whole future.
    Returns the real epoch (float) of the trigger, or None.
    """
    threshold = threshold_multiplier * noise_floor
    for i in range(len(diff)):

```

```

        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] > threshold:
            return epoch_grid[i]
    return None

def detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff crosses from
    positive to negative (the first "green -> red" zero crossing).
    Confirmed on two separate training runs to land in a consistent,
    unambiguous spot even though the crossing's magnitude is weak.
    Returns the real epoch (float) of the crossing, or None.
    """
    for i in range(len(diff) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] >= 0 and diff[i + 1] < 0:
            return epoch_grid[i + 1]
    return None

```

35.08 Experiments/nanda_l2_p113/run_benchmark.py

```
#!/usr/bin/env python3
"""
nanda_l2_p113 benchmark pipeline: trains the four-head model on (a + b) mod 113
several times with independent seeds, then analyzes and visualizes the L2-Norm
predictor results. Single command, single script.
```

Stage 1: training (3 independent seeded runs)
Stage 2: analysis – cross-run comparison charts + PDF report

This is a trimmed copy of run_full_benchmark.py. Two differences:
* the task is (a + b) mod 113 with AdamW betas=(0.9, 0.98) – see train.py;
* there is no Dropout predictor, so every Dropout load / plot / file is gone,
and it does NOT shell out to src/generate_master_report.py (that tool is
Dropout-aware and belongs to the main experiment).

RESUMABLE: stop this at any point (Ctrl-C, crash, sleep) and re-run. Finished
runs are detected and kept, a half-built run folder is cleared, and the
analysis step is regenerated only when a new run was produced.

```
import re
import shutil
import subprocess
import sys
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
from pathlib import Path

PKG = Path(__file__).resolve().parent
RUNS_BASE = PKG / "runs"
ANALYSIS_DIR = PKG / "benchmark_analysis"
TRAIN_SCRIPT = PKG / "train.py"

TARGET_RUNS = 3

# A run_N/ folder counts as "finished" only when every one of these is on
# disk. train.py writes them all right at the end (after all epochs), and
# reports/training_report.pdf is the very last artifact, so its presence means
# the run truly completed. Anything less = interrupted, must be redone.
REQUIRED_RUN_FILES = [
    "training/train_acc_history.npy",
    "training/test_acc_history.npy",
    "training/loss_history.npy",
    "l2_norm/l2_norm_history.npy",
    "l2_norm/epoch_grid.npy",
    "reports/training_report.pdf",
]

ANALYSIS_OUTPUTS = [
    "01_grokking_curves.png",
    "02_l2_norm_comparison.png",
    "benchmark_report.pdf",
]

# =====
# STAGE 1: TRAINING (four-head, N seeded runs)
# =====

def _run_dirs():
    """All run_<N> dirs under runs/, sorted by number."""
    if not RUNS_BASE.is_dir():
        return []
    dirs = [d for d in RUNS_BASE.iterdir()
             if d.is_dir() and re.fullmatch(r"run_\d+", d.name)]
    return sorted(dirs, key=lambda d: int(d.name[4:]))

def _run_is_complete(run_dir):
    return all((run_dir / rel).exists() for rel in REQUIRED_RUN_FILES)

def prepare_runs_dir():
    """Make runs/ safe to resume into, and return how many runs are already
    fully finished.

    A stopped run leaves a half-built run_N/ folder (usually just seed.npy plus
    empty sub-folders – training does not checkpoint mid-way). Such folders are
    deleted here so they are never miscounted as finished and so train.py's own
    run-numbering picks the right next number.
```

```

Finished runs are produced strictly in order and the pipeline aborts on any
failure, so after this cleanup the surviving run_N/ folders are a contiguous
1..k and the next run is k+1.
"""
complete = 0
for d in _run_dirs():
    if _run_is_complete(d):
        complete += 1
    else:
        print(f" clearing incomplete run folder (interrupted / did not finish): {d}")
        shutil.rmtree(d)
return complete

def _analysis_outputs_present():
    return (ANALYSIS_DIR.is_dir()
            and all((ANALYSIS_DIR / f).exists() for f in ANALYSIS_OUTPUTS))

def run_training(run_num):
    """Run one training session, then verify it really finished."""
    print("\n" + "="*70)
    print(f"STAGE 1. {run_num}: Four-Head Training on (a + b) mod 113 (Run {run_num})")
    print("="*70)
    print(f"Command: python {TRAIN_SCRIPT}")
    print(f"Expected output: nanda_l2_p113/runs/run_{run_num}/{training,l2_norm,reports}")
    print("="*70 + "\n")

    result = subprocess.run([sys.executable, str(TRAIN_SCRIPT)])

    if result.returncode != 0:
        print(f"\nRun {run_num} FAILED (exit code {result.returncode})")
        return False

    produced = _run_dirs()[-1] if _run_dirs() else None
    if produced is None or not _run_is_complete(produced):
        print(f"\nRun {run_num}: process exited cleanly but its output folder is "
              f"incomplete ({produced}). Treating as a failed run.")
        return False

    print(f"\nRun {run_num} complete ({produced.name})")
    return True

# =====
# STAGE 2: ANALYSIS & VISUALIZATION
# =====

class BenchmarkAnalyzer:
    def __init__(self):
        self.output_dir = ANALYSIS_DIR
        self.output_dir.mkdir(exist_ok=True)
        self.results = {}
        self.run_colors = ["red", "green", "blue"]

    def load_runs(self):
        print("Loading runs...")

        for run_num in range(1, TARGET_RUNS + 1):
            run_dir = RUNS_BASE / f"run_{run_num}"
            training_dir = run_dir / "training"
            l2_norm_dir = run_dir / "l2_norm"

            if not training_dir.exists():
                print(f" Run {run_num} not found")
                continue

            self.results[f"run_{run_num}"] = {
                "train_acc": np.load(training_dir / "train_acc_history.npy"),
                "test_acc": np.load(training_dir / "test_acc_history.npy"),
                "loss": np.load(training_dir / "loss_history.npy"),
                "l2_norm": np.load(l2_norm_dir / "l2_norm_history.npy"),
            }

            print(f" Loaded run {run_num} (epochs: {len(self.results[f'run_{run_num}']['train_acc'])})")

        return len(self.results) >= 1

    def run_keys(self):
        return [f"run_{n}" for n in range(1, TARGET_RUNS + 1) if f"run_{n}" in self.results]

    def grok_epoch(self, test_acc):
        return np.argmax(np.array(test_acc) > 0.9)

    def generate_grokking_curves(self):
        print("\nGenerating grokking curves comparison...")

```

```

fig, ax = plt.subplots(figsize=(10, 6))

for i, run_num in enumerate(range(1, TARGET_RUNS + 1)):
    key = f"run_{run_num}"
    if key not in self.results:
        continue
    data = self.results[key]
    epochs = range(1, len(data["test_acc"]) + 1)
    grok = self.grok_epoch(data["test_acc"])
    color = self.run_colors[i % len(self.run_colors)]
    ax.plot(epochs, data["test_acc"], label=f"Run {run_num} test acc (grok={grok})",
            color=color, linewidth=2, alpha=0.85)
    ax.plot(epochs, data["train_acc"], color=color, linewidth=1,
            alpha=0.35, linestyle="--")
    if grok > 0:
        ax.axvline(x=grok, color=color, linestyle=":", linewidth=1.5, alpha=0.7)

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)")
ax.set_ylabel("Accuracy")
ax.set_title("p=113 Runs: Grokking Comparison\n(solid = test, dashed = train, dotted = grok epoch)")
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(self.output_dir / "01_grokking_curves.png", dpi=150, bbox_inches='tight')
plt.close(fig)

print(" Saved: 01_grokking_curves.png")

def generate_l2_norm_comparison(self):
    print("Generating L2 Norm comparison...")

    fig, ax = plt.subplots(figsize=(10, 6))

    for i, run_num in enumerate(range(1, TARGET_RUNS + 1)):
        key = f"run_{run_num}"
        if key not in self.results:
            continue
        data = self.results[key]
        epochs = range(1, len(data["l2_norm"]) + 1)
        ax.plot(epochs, data["l2_norm"], label=f"Run {run_num}",
                color=self.run_colors[i % len(self.run_colors)], linewidth=2, alpha=0.85)

    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("L2 Norm")
    ax.set_title("p=113 L2 Norm Comparison")
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(self.output_dir / "02_l2_norm_comparison.png", dpi=150, bbox_inches='tight')
    plt.close(fig)

    print(" Saved: 02_l2_norm_comparison.png")

def generate_run_consistency_report(self):
    print("Generating run consistency report...")

    with PdfPages(self.output_dir / "benchmark_report.pdf") as pdf:

        # Page 1: Grokking epoch per run + mean line
        fig = plt.figure(figsize=(12, 8))
        ax = fig.add_subplot(111)

        grok_epochs = []
        run_labels = []
        for run_num in range(1, TARGET_RUNS + 1):
            key = f"run_{run_num}"
            if key in self.results:
                grok_epochs.append(self.grok_epoch(self.results[key]["test_acc"]))
                run_labels.append(f"Run {run_num}")

        bars = ax.bar(range(len(grok_epochs)), grok_epochs,
                      color=self.run_colors[:len(grok_epochs)],
                      alpha=0.7, edgecolor="black", linewidth=2)

        for bar, epoch in zip(bars, grok_epochs):
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width()/2., height,
                    f'{int(epoch)}', ha='center', va='bottom', fontsize=12, fontweight='bold')

        if len(grok_epochs) >= 2:
            mean_grok = float(np.mean(grok_epochs))
            std_grok = float(np.std(grok_epochs))

```

```

        ax.axhline(y=mean_grok, color="black", linestyle="--", linewidth=1.5,
                    label=f"Mean {mean_grok:.0f} +/- {std_grok:.0f}")
        ax.legend()

    ax.set_ylabel("Grokking Epoch", fontsize=12)
    ax.set_title("p=113 Grokking Epoch Across Seeded Runs", fontsize=14, fontweight='bold')
    ax.set_xticks(range(len(grok_epochs)))
    ax.set_xticklabels(run_labels, rotation=45, ha='right')
    ax.grid(True, alpha=0.3, axis='y')

    pdf.savefig(fig)
    plt.close(fig)

# Page 2: Loss curves
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111)

for i, run_num in enumerate(range(1, TARGET_RUNS + 1)):
    key = f"run_{run_num}"
    if key in self.results:
        epochs = range(1, len(self.results[key]["loss"]) + 1)
        ax.plot(epochs, self.results[key]["loss"], label=f"Run {run_num}",
                color=self.run_colors[i % len(self.run_colors)], linewidth=2, alpha=0.85)

    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Loss (log scale)", fontsize=12)
    ax.set_title("p=113 Loss Curves", fontsize=14, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    pdf.savefig(fig)
    plt.close(fig)

# Page 3: Run consistency statistics
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111)
ax.axis('off')

stats_text = "p=113 RUN CONSISTENCY ANALYSIS (L2-Norm predictor)\n\n"

groks = []
for run_num in range(1, TARGET_RUNS + 1):
    key = f"run_{run_num}"
    if key in self.results:
        groks.append(self.grok_epoch(self.results[key]["test_acc"]))

if groks:
    mean_grok = np.mean(groks)
    std_grok = np.std(groks)
    stats_text += f"Grokking Epochs ({len(groks)} seeded runs):\n"
    for i, grok in enumerate(groks, 1):
        stats_text += f"  Run {i}: {grok} epochs\n"
    stats_text += f"  Mean: {mean_grok:.1f} +/- {std_grok:.1f} epochs\n"
    stats_text += f"  Consistency: {'High' if std_grok < 500 else 'Moderate' if std_grok < 1000 else 'Low'}\n\n"

stats_text += "Setup:\n"
stats_text += "  Task           : (a + b) mod 113\n"
stats_text += "  Optimizer      : AdamW lr=1e-3 wd=1.0 betas=(0.9, 0.98)\n"
stats_text += "  Model          : TransformerFourHead, 4 heads, d_model=128\n"
stats_text += "  Predictor      : L2 Norm only\n\n"

stats_text += "Interpretation:\n"
stats_text += "  - Low std dev: predictor signal consistent across random seeds\n"
stats_text += "  - High std dev: predictor signal sensitive to initialization\n\n"

stats_text += "Next Steps:\n"
stats_text += "  1. Overlay the L2 norm curves on Nanda Figure 7 (Literature/nanda_figures/)\n"
stats_text += "  2. Check whether betas=(0.9, 0.98) shrank the pre-grok weight-norm spike\n"
stats_text += "  3. Validate the L2 Norm predictor against the 3-criteria protocol\n\n"

ax.text(0.1, 0.9, stats_text, transform=ax.transAxes, fontsize=11,
        verticalalignment='top', fontfamily='monospace',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

pdf.savefig(fig)
plt.close(fig)

print(" Saved: benchmark_report.pdf (3 pages)")

def run(self):
    print("\n" + "="*70)
    print("BENCHMARK ANALYSIS: p=113 L2-Norm Results Visualization")
    print("="*70)

```

```

        if not self.load_runs():
            print("\nCannot proceed: no runs found under nanda_l2_p113/runs/")
            return False

        print("\nGenerating visualizations...")
        self.generate_grokking_curves()
        self.generate_l2_norm_comparison()
        self.generate_run_consistency_report()

        print("\n" + "="*70)
        print("ANALYSIS COMPLETE")
        print("="*70)
        print(f"\nVisualizations saved to: {self.output_dir}")
        print("\nGenerated files:")
        for f in ANALYSIS_OUTPUTS:
            print(f" - {f}")
        print("\n" + "="*70 + "\n")

        return True

# =====
# MAIN PIPELINE
# =====

def main():
    print("\n" + "="*70)
    print("GROKING BENCHMARK: nanda_l2_p113 (L2-Norm only, p = 113)")
    print("="*70)
    print("\nThis script will:")
    print(f" 1. Run training ({TARGET_RUNS} runs with independent seeds)")
    print(" 2. Analyze results and generate comparison charts + PDF report")
    print("="*70)

    print("\nResume mode: finished runs are kept, interrupted run folders are")
    print("cleared, and analysis is only redone when something new was produced.")

    # Stage 1: resume up to TARGET_RUNS.
    already_done = prepare_runs_dir()
    if already_done >= TARGET_RUNS:
        print("\n" + "="*70)
        print(f"STAGE 1: all {TARGET_RUNS} runs already complete – nothing to train")
        print("="*70)
    elif already_done:
        print("\n" + "="*70)
        print(f"STAGE 1: {already_done} run(s) complete – resuming from run {already_done + 1}")
        print("="*70)

    newly_run = 0
    for run_num in range(already_done + 1, TARGET_RUNS + 1):
        if not run_training(run_num):
            print("\n" + "="*70)
            print(f"PIPELINE STOPPED at run {run_num}. Re-run this script to resume "
                  f"from here – completed runs will not be repeated.")
            print("="*70)
            sys.exit(1)
        newly_run += 1

    # Stage 2: Analysis – skip if nothing new and the charts already exist.
    print("\n" + "="*70)
    print("STAGE 2: Analysis & Visualization")
    print("="*70)

    if newly_run == 0 and _analysis_outputs_present():
        print("Nothing new since last run and benchmark_analysis/ is already "
              "populated – skipping analysis regeneration.")
    else:
        analyzer = BenchmarkAnalyzer()
        if not analyzer.run():
            print("\n" + "="*70)
            print("PIPELINE WARNING: Analysis failed (training results still saved)")
            print("="*70)
            sys.exit(1)

    print("\n" + "="*70)
    print("nanda_l2_p113 BENCHMARK PIPELINE COMPLETE")
    print("="*70)
    print("\nTraining results saved to:")
    for run_num in range(1, TARGET_RUNS + 1):
        print(f" - nanda_l2_p113/runs/run_{run_num}/")
    print("\nAnalysis saved to nanda_l2_p113/benchmark_analysis/ :)")
    for f in ANALYSIS_OUTPUTS:
        print(f" - {f}")
    print("\n" + "="*70 + "\n")

if __name__ == "__main__":
    main()

```

36. 08_Experiments/nanda_l2_p113/train.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
import re
from predictors.l2_norm import (
    compute_l2_norm,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from data.modular_arithmetic import get_data loaders
from models.transformer_four_head import TransformerFourHead
from measurements import PredictorMeasurements

# =====
# nanda_l2_p113 - L2-Norm-only training run on (a + b) mod 113.
#
# This is a trimmed copy of src/train_four_head.py. It is a fully faithful
# Nanda et al. replication. Four deliberate differences from the main
# experiment in src/:
#
# 1. MODULUS = 113 (Nanda's mainline prime), not 97.
# 2. L2-Norm predictor only - the Dropout multi-rate sweep is gone.
# 3. AdamW betas = (0.9, 0.98) - the value Nanda et al. and Power et al.
#    both use - instead of PyTorch's default (0.9, 0.999). See
#    context.md, Sep 3 2026 session, item 9.2.
# 4. Weight init: TransformerFourHead now draws every matrix (embeddings
#    included) from N(0, 0.8/sqrt(d_model)) - the TransformerLens scheme
#    Nanda uses - instead of PyTorch's defaults. This removes the large
#    early L2 collapse (a weight-decay transient on the oversized N(0,1)
#    embedding table, not a grokking feature). See the model file.
#
# Everything else matches src/train_four_head.py: same architecture
# (TransformerFourHead, 4 heads, d_model=128), same lr=1e-3,
# weight_decay=1.0, same 40000 epochs, same full-batch training on a 30/70
# split, same L2-Norm predictor code and windows.
#
# Each run is saved into nanda_l2_p113/runs/run_&Nt; (a NEW numbered
# folder every time), so several independent seeded runs sit side by side
# and are never overwritten.
# =====

MODULUS = 113

def get_next_run_number(runs_base):
    if not os.path.isdir(runs_base):
        return 1
    existing_numbers = []
    for name in os.listdir(runs_base):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(runs_base, name)):
            existing_numbers.append(int(match.group(1)))
    return max(existing_numbers, default=0) + 1

package_root = os.path.dirname(os.path.abspath(__file__))
runs_base = os.path.join(package_root, "runs")
os.makedirs(runs_base, exist_ok=True)

run_number = get_next_run_number(runs_base)
output_dir = os.path.join(runs_base, f"run_{run_number}")
os.makedirs(output_dir, exist_ok=True)
os.chdir(output_dir)

# Initialize measurements system (L2 Norm + training only)
measurements = PredictorMeasurements(output_dir, model_type="four_head")

print(f"This is Run {run_number}. All outputs will be saved to nanda_l2_p113/runs/run_{run_number}/")

# NEW random seed on every run. torch.seed() picks a fresh, non-deterministic
# seed and returns the exact number it used, so we both use it and record it in
# one step. Grokking is stochastic (grok epoch moves with the seed), so the
# thesis needs several genuinely independent runs, not one run repeated.
seed = torch.seed()
print(f"Seed: {seed}")
np.save("seed.npy", seed)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print(f"Device:", device)
```

```

data_loader = get_data_loaders(MODULUS, batch_size=int(0.3 * MODULUS * MODULUS))
model = TransformerFourHead(vocab_size=MODULUS + 1, d_model=128, num_heads=4).to(device)

# betas=(0.9, 0.98) matches Nanda et al. and Power et al. (PyTorch's AdamW
# default is (0.9, 0.999)). This is one of the three deliberate differences
# from src/train_four_head.py.
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0, betas=(0.9, 0.98))

print("Model: TransformerFourHead (num_heads=4, matches Nanda et al.)")
print(f"Task: (a + b) mod {MODULUS}, vocab_size = {MODULUS + 1}")
print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 40000
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())
    l2_norm_history.append(compute_l2_norm(model))

    print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
          {train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
          {l2_norm_history[-1]:.4f}")

# Save training data
measurements.save_training_data(train_acc_history, test_acc_history, loss_history)

grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

# L2 Norm Predictor: Moving Average Crossover Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model, p=113): Moving Average Crossover")
print("="*60)

epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history, fast_window=50,
                                                                    slow_window=200)

detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)

print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f" MA Crossover epoch: {detection_epoch:.1f}")
    print(f" Grok epoch (test acc > 90%): {grok_epoch}")
    print(f" Lead time: {lead_time:.1f} epochs")
else:
    print(" No MA crossover detected")
    print(f" Grok epoch (test acc > 90%): {grok_epoch}")

print(f"\nL2 Norm Stats:")
print(f" Min: {np.min(l2_norm_history):.4f}")
print(f" Max: {np.max(l2_norm_history):.4f}")

```

```

print(f" Final: {l2_norm_history[-1]:.4f}")

# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model, p=113): MA of Slow MA - Zero-Crossing Trigger")
print("="*60)

fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff, skip_epochs=100)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:
    trigger_lead_time = grok_epoch - trigger_epoch
    print(f" Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
    print(f" Grok epoch (test acc > 90%): {grok_epoch}")
    print(f" Lead time: {trigger_lead_time:.1f} epochs")
else:
    print(" No trigger detected")
    print(f" Grok epoch (test acc > 90%): {grok_epoch}")

# Save all L2 Norm measurements
measurements.save_l2_norm_data(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch
)

# =====
# VISUALIZATION & REPORT GENERATION
# =====
print("\n" + "="*60)
print("VISUALIZATION & REPORT GENERATION (4-head model, p=113)")
print("="*60)

# Generate standalone L2 Norm visualizations
measurements.generate_l2_norm_visualizations(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
    grok_epoch, detection_epoch, trigger_epoch
)
print("[OK] L2 Norm visualizations saved to nanda_l2_p113/runs/run_{}/l2_norm/".format(run_number))

# Generate combined PDF report (training + L2 Norm pages only)
report_path = measurements.generate_combined_report(
    train_acc_history, test_acc_history, loss_history, l2_norm_history,
    epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch
)
print(f"[OK] PDF report saved to {report_path} (3 pages: training + L2 Norm)")

print("\n" + "="*60)
print("RUN {} COMPLETE".format(run_number))
print("="*60)
print(f"Grok epoch: {grok_epoch}")
print(f"L2 Norm MA crossover: {detection_epoch if detection_epoch else 'Not detected'}")
print(f"L2 Norm MA-of-MA trigger: {trigger_epoch if trigger_epoch else 'Not detected'}")
print(f"\nAll measurements saved to nanda_l2_p113/runs/run_{run_number}/")
print(f" - training/ : raw training data")
print(f" - l2_norm/ : L2 norm measurements + visualizations")
print(f" - reports/ : combined PDF report")

```

37. archive/Deprecated_Code/old_benchmarks/run_full_benchmark.py.deprecated

```
#!/usr/bin/env python3
"""
Four-head benchmark pipeline: trains the four-head model several times with
independent seeds, then analyzes and visualizes the results. Single command,
single script.

Stage 1: Four-head baseline training (3 independent runs, different seeds)
Stage 2: Analysis – comparison charts + PDF report across the runs

RESUMABLE: this script can be stopped at any point (Ctrl-C, crash, machine
sleep) and re-run. Finished runs are detected and kept, a half-built run
folder from the interrupted attempt is cleared, and the analysis step is
regenerated only when a new run was actually produced. Nothing already
computed is computed again.

Single-head is no longer part of this pipeline. The four-head model is the
faithful Nanda et al. architecture (d_model=128, 4 attention heads), so the
benchmark stands on that alone. The single-head code has been moved to
archive/single_head/ and can be brought back if a later robustness check
needs it. See context.md for the reasoning.
"""

import re
import shutil
import subprocess
import sys
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
from pathlib import Path

# =====
# STAGE 1: TRAINING (four-head, N seeded runs)
# =====

TARGET_FOUR_HEAD_RUNS = 3

FOUR_HEAD_BASE = Path("runs/four_head")

# A run_N/ folder counts as "finished" only when every one of these is on
# disk. train_four_head.py writes them all right at the end (after all
# epochs), and reports/training_report.pdf is the very last artifact, so
# its presence means the run truly completed. Anything less = the run was
# interrupted (Ctrl-C, crash, machine sleep, ...) and must be redone.
REQUIRED_RUN_FILES = [
    "training/train_acc_history.npy",
    "training/test_acc_history.npy",
    "training/loss_history.npy",
    "l2_norm/l2_norm_history.npy",
    "dropout/dropout_gap_epochs.npy",
    "dropout/dropout_gap_by_rate.npy",
    "dropout/dropout_rates.npy",
    "reports/training_report.pdf",
]

ANALYSIS_OUTPUTS = [
    "01_grokking_curves.png",
    "02_l2_norm_comparison.png",
    "03_dropout_gap_comparison.png",
    "benchmark_report.pdf",
]

def _run_dirs():
    """All run_<N> dirs under runs/four_head/, sorted by number."""
    if not FOUR_HEAD_BASE.is_dir():
        return []
    dirs = [d for d in FOUR_HEAD_BASE.iterdir()
             if d.is_dir() and re.fullmatch(r"run_\d+", d.name)]
    return sorted(dirs, key=lambda d: int(d.name[4:]))

def _run_is_complete(run_dir):
    return all((run_dir / rel).exists() for rel in REQUIRED_RUN_FILES)

def prepare_four_head_dir():
    """Make runs/four_head/ safe to resume into, and return how many runs
    are already fully finished.

    A stopped run leaves a half-built run_N/ folder (usually just seed.npy
```

plus empty sub-folders – training does not checkpoint mid-way, so there is nothing worth keeping there). Such folders are deleted here so that:

- * they are never miscounted as a finished run, and
- * train_four_head.py's own run-numbering picks the right next number instead of skipping over the dead folder.

Finished runs are produced strictly in order and the pipeline aborts on any failure, so after this cleanup the surviving run_N/ folders are a contiguous 1..k and the next run is k+1.

```

"""
complete = 0
for d in _run_dirs():
    if _run_is_complete(d):
        complete += 1
    else:
        print(f"  ✖ clearing incomplete run folder (interrupted / did not finish): {d}")
        shutil.rmtree(d)
return complete

def _analysis_outputs_present():
    return (Path("benchmark_analysis").is_dir()
            and all((Path("benchmark_analysis") / f).exists() for f in ANALYSIS_OUTPUTS))

def run_four_head(run_num):
    """Run one four-head training session, then verify it really finished."""
    print("\n" + "="*70)
    print(f"STAGE 1. {run_num}: Four-Head Baseline Training (Run {run_num})")
    print("="*70)
    print(f"Command: python src/train_four_head.py")
    print(f"Expected output: runs/four_head/run_{run_num}/{training, l2_norm, dropout, reports}")
    print("="*70 + "\n")

    result = subprocess.run([sys.executable, "src/train_four_head.py"], cwd=".")

    if result.returncode != 0:
        print(f"\n  ✖ Four-head run {run_num} FAILED (exit code {result.returncode})")
        return False

    produced = _run_dirs()[-1] if _run_dirs() else None
    if produced is None or not _run_is_complete(produced):
        print(f"\n  ✖ Four-head run {run_num}: process exited cleanly but its output "
              f"folder is incomplete ({produced}). Treating as a failed run.")
        return False

    print(f"\n  ✓ Four-head run {run_num} complete ({produced.name})")
    return True

# =====
# STAGE 2: ANALYSIS & VISUALIZATION
# =====

class BenchmarkAnalyzer:
    def __init__(self):
        self.four_head_base = Path("runs/four_head")
        self.output_dir = Path("benchmark_analysis")
        self.output_dir.mkdir(exist_ok=True)
        self.results = {}
        self.run_colors = ["red", "green", "blue"]

    def load_four_head_runs(self):
        """Load all four-head runs."""
        print("Loading four-head runs...")

        for run_num in [1, 2, 3]:
            run_dir = self.four_head_base / f"run_{run_num}"
            training_dir = run_dir / "training"
            l2_norm_dir = run_dir / "l2_norm"
            dropout_dir = run_dir / "dropout"

            if not training_dir.exists():
                print(f"  ⚠ Run {run_num} not found")
                continue

            self.results[f"four_head_run_{run_num}"] = {
                "train_acc": np.load(training_dir / "train_acc_history.npy"),
                "test_acc": np.load(training_dir / "test_acc_history.npy"),
                "loss": np.load(training_dir / "loss_history.npy"),
                "l2_norm": np.load(l2_norm_dir / "l2_norm_history.npy"),
                "dropout_gap_epochs": np.load(dropout_dir / "dropout_gap_epochs.npy"),
                "dropout_gap_by_rate": np.load(dropout_dir / "dropout_gap_by_rate.npy"),
                "dropout_rates": np.load(dropout_dir / "dropout_rates.npy"),
            }
            print(f"  ✓ Loaded run {run_num} (epochs:

```

```

        {len(self.results[f'four_head_run_{run_num}']['train_acc'])})")

    return len(self.results) >= 1

def run_keys(self):
    """Ordered keys of the four-head runs that actually loaded."""
    return [f"four_head_run_{n}" for n in [1, 2, 3]
            if f"four_head_run_{n}" in self.results]

def grok_epoch(self, test_acc):
    """Find grokking epoch (test acc > 90%)."""
    return np.argmax(np.array(test_acc) > 0.9)

def generate_grokking_curves(self):
    """Overlay grokking curves across the four-head runs."""
    print("\nGenerating grokking curves comparison...")

    fig, ax = plt.subplots(figsize=(10, 6))

    for i, run_num in enumerate([1, 2, 3]):
        key = f"four_head_run_{run_num}"
        if key not in self.results:
            continue
        data = self.results[key]
        epochs = range(1, len(data["test_acc"]) + 1)
        grok = self.grok_epoch(data["test_acc"])
        color = self.run_colors[i]
        ax.plot(epochs, data["test_acc"], label=f"Run {run_num} test acc (grok={grok})",
                color=color, linewidth=2, alpha=0.85)
        ax.plot(epochs, data["train_acc"], color=color, linewidth=1,
                alpha=0.35, linestyle="--")
        if grok > 0:
            ax.axvline(x=grok, color=color, linestyle=":", linewidth=1.5, alpha=0.7)

    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("Accuracy")
    ax.set_title("Four-Head Runs: Grokking Comparison\n(solid = test, dashed = train, dotted = grok epoch)")
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(self.output_dir / "01_grokking_curves.png", dpi=150, bbox_inches='tight')
    plt.close(fig)

    print(" ✓ Saved: 01_grokking_curves.png")

def generate_l2_norm_comparison(self):
    """Overlay L2 Norm curves across the four-head runs."""
    print("Generating L2 Norm comparison...")

    fig, ax = plt.subplots(figsize=(10, 6))

    for i, run_num in enumerate([1, 2, 3]):
        key = f"four_head_run_{run_num}"
        if key not in self.results:
            continue
        data = self.results[key]
        epochs = range(1, len(data["l2_norm"]) + 1)
        ax.plot(epochs, data["l2_norm"], label=f"Run {run_num}",
                color=self.run_colors[i], linewidth=2, alpha=0.85)

    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("L2 Norm")
    ax.set_title("Four-Head L2 Norm Comparison")
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(self.output_dir / "02_l2_norm_comparison.png", dpi=150, bbox_inches='tight')
    plt.close(fig)

    print(" ✓ Saved: 02_l2_norm_comparison.png")

def generate_dropout_comparison(self):
    """Dropout gap: full multi-rate sweep, one panel per four-head run.

    dropout_gap_by_rate is indexed [rate_index, epoch_index], with
    rate_index following dropout_rates.
    """
    print("Generating Dropout gap sweep comparison...")

    keys = self.run_keys()
    n = len(keys)
    fig, axes = plt.subplots(1, n, figsize=(6 * n, 5), squeeze=False)

```

```

for col, key in enumerate(keys):
    data = self.results[key]
    epochs = data["dropout_gap_epochs"]
    rates = data["dropout_rates"]
    gap_by_rate = data["dropout_gap_by_rate"]
    ax = axes[0][col]
    for r_idx, rate in enumerate(rates):
        ax.plot(epochs, gap_by_rate[r_idx], linewidth=1.8, alpha=0.9,
                label=f"p={rate:g}")
    ax.axhline(y=0, color="black", linewidth=0.6, alpha=0.5)
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("Dropout Gap")
    ax.set_title(f"{key.replace('four_head_', '').replace('_', ' ').title()} - rate sweep")
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

fig.suptitle("Four-Head Dropout Gap: Multi-Rate Sweep", fontsize=13)
plt.tight_layout()
plt.savefig(self.output_dir / "03_dropout_gap_comparison.png", dpi=150, bbox_inches='tight')
plt.close(fig)

print(" ✓ Saved: 03_dropout_gap_comparison.png")

def generate_run_consistency_report(self):
    """Generate PDF report with run consistency metrics."""
    print("Generating run consistency report...")

    with PdfPages(self.output_dir / "benchmark_report.pdf") as pdf:

        # Page 1: Grokking epoch per run + mean line
        fig = plt.figure(figsize=(12, 8))
        ax = fig.add_subplot(111)

        grok_epochs = []
        run_labels = []
        for run_num in [1, 2, 3]:
            key = f"four_head_run_{run_num}"
            if key in self.results:
                grok_epochs.append(self.grok_epoch(self.results[key]["test_acc"]))
                run_labels.append(f"4-Head Run {run_num}")

        bars = ax.bar(range(len(grok_epochs)), grok_epochs,
                      color=self.run_colors[:len(grok_epochs)],
                      alpha=0.7, edgecolor="black", linewidth=2)

        for bar, epoch in zip(bars, grok_epochs):
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width()/2., height,
                    f'{int(epoch)}', ha='center', va='bottom', fontsize=12, fontweight='bold')

        if len(grok_epochs) >= 2:
            mean_grok = float(np.mean(grok_epochs))
            std_grok = float(np.std(grok_epochs))
            ax.axhline(y=mean_grok, color="black", linestyle="--", linewidth=1.5,
                      label=f"Mean {mean_grok:.0f} ± {std_grok:.0f}")
            ax.legend()

        ax.set_ylabel("Grokking Epoch", fontsize=12)
        ax.set_title("Four-Head Grokking Epoch Across Seeded Runs", fontsize=14, fontweight='bold')
        ax.set_xticks(range(len(grok_epochs)))
        ax.set_xticklabels(run_labels, rotation=45, ha='right')
        ax.grid(True, alpha=0.3, axis='y')

        pdf.savefig(fig)
        plt.close(fig)

        # Page 2: Loss curves
        fig = plt.figure(figsize=(12, 8))
        ax = fig.add_subplot(111)

        for i, run_num in enumerate([1, 2, 3]):
            key = f"four_head_run_{run_num}"
            if key in self.results:
                epochs = range(1, len(self.results[key]["loss"]) + 1)
                ax.plot(epochs, self.results[key]["loss"], label=f"4-Head Run {run_num}",
                      color=self.run_colors[i], linewidth=2, alpha=0.85)

        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_xlabel("Epoch (log scale)", fontsize=12)
        ax.set_ylabel("Loss (log scale)", fontsize=12)
        ax.set_title("Four-Head Loss Curves", fontsize=14, fontweight='bold')
        ax.legend()
        ax.grid(True, alpha=0.3)
        pdf.savefig(fig)

```

```

plt.close(fig)

# Page 3: Run consistency statistics
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111)
ax.axis('off')

stats_text = "FOUR-HEAD RUN CONSISTENCY ANALYSIS\n\n"

four_head_groks = []
for run_num in [1, 2, 3]:
    key = f"four_head_run_{run_num}"
    if key in self.results:
        four_head_groks.append(self.grok_epoch(self.results[key]["test_acc"]))

if four_head_groks:
    mean_grok = np.mean(four_head_groks)
    std_grok = np.std(four_head_groks)
    stats_text += f"Grokking Epochs ({len(four_head_groks)} seeded runs):\n"
    for i, grok in enumerate(four_head_groks, 1):
        stats_text += f"  Run {i}: {grok} epochs\n"
    stats_text += f"  Mean: {mean_grok:.1f} ± {std_grok:.1f} epochs\n"
    stats_text += f"  Consistency: {'High' if std_grok < 500 else 'Moderate' if std_grok < 1000 else 'Low'}\n\n"

stats_text += "Interpretation:\n"
stats_text += "  • Low std dev: predictor signal consistent across random seeds\n"
stats_text += "  • High std dev: predictor signal sensitive to initialization\n\n"

stats_text += "Next Steps:\n"
stats_text += "  1. Validate L2 Norm & Dropout against the 3-criteria protocol\n"
stats_text += "  2. If both pass: move to the Spectral predictor\n"
stats_text += "  3. Build the baseline benchmark with the validated predictors\n"

ax.text(0.1, 0.9, stats_text, transform=ax.transAxes, fontsize=11,
        verticalalignment='top', fontfamily='monospace',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

pdf.savefig(fig)
plt.close(fig)

print(" ✓ Saved: benchmark_report.pdf (3 pages)")

def run(self):
    """Execute full analysis."""
    print("\n" + "="*70)
    print("BENCHMARK ANALYSIS: Four-Head Results Visualization")
    print("="*70)

    if not self.load_four_head_runs():
        print("\n❌ Cannot proceed: no four-head runs found under runs/four_head/")
        return False

    print("\nGenerating visualizations...")
    self.generate_grokking_curves()
    self.generate_l2_norm_comparison()
    self.generate_dropout_comparison()
    self.generate_run_consistency_report()

    print("\n" + "="*70)
    print("✓ ANALYSIS COMPLETE")
    print("="*70)
    print(f"\nVisualizations saved to: {self.output_dir.absolute()}")
    print("\nGenerated files:")
    print("  - 01_grokking_curves.png")
    print("  - 02_l2_norm_comparison.png")
    print("  - 03_dropout_gap_comparison.png")
    print("  - benchmark_report.pdf")
    print("\n" + "="*70 + "\n")

    return True

# =====
# MAIN PIPELINE
# =====

def main():
    print("\n" + "="*70)
    print("GROKING BENCHMARK: Four-Head Experimental Pipeline")
    print("="*70)
    print("\nThis script will:")
    print(f"  1. Run four-head baseline training ({TARGET_FOUR_HEAD_RUNS} runs with independent seeds)")
    print("  2. Analyze results and generate comparison charts + PDF report")
    print(f"\nTotal training runs: {TARGET_FOUR_HEAD_RUNS} (four-head only)")
    print("Expected time: depends on hardware")

```

```

print("="*70)

print("\nResume mode: finished runs are kept, interrupted run folders are")
print("cleared, and analysis is only redone when something new was produced.")

# Stage 1: Four-head – resume up to TARGET_FOUR_HEAD_RUNS.
already_done = prepare_four_head_dir()
if already_done >= TARGET_FOUR_HEAD_RUNS:
    print("\n" + "="*70)
    print(f"STAGE 1: all {TARGET_FOUR_HEAD_RUNS} four-head runs already complete – nothing to train")
    print("="*70)
elif already_done:
    print("\n" + "="*70)
    print(f"STAGE 1: {already_done} run(s) complete – resuming from run {already_done + 1}")
    print("="*70)

newly_run = 0
for run_num in range(already_done + 1, TARGET_FOUR_HEAD_RUNS + 1):
    if not run_four_head(run_num):
        print("\n" + "="*70)
        print(f"PIPELINE STOPPED at run {run_num}. Re-run this script to resume "
              f"from here – completed runs will not be repeated.")
        print("="*70)
        sys.exit(1)
    newly_run += 1

# Stage 2: Analysis – skip if nothing new and the charts already exist.
print("\n" + "="*70)
print("STAGE 2: Analysis & Visualization")
print("="*70)

master_pdfs_present = (Path("benchmark_analysis/master_plots.pdf").exists()
                       and Path("benchmark_analysis/master_data.pdf").exists())

if newly_run == 0 and _analysis_outputs_present() and master_pdfs_present:
    print("Nothing new since last run and benchmark_analysis/ is already "
          "populated – skipping analysis regeneration.")
else:
    analyzer = BenchmarkAnalyzer()
    if not analyzer.run():
        print("\n" + "="*70)
        print("PIPELINE WARNING: Analysis failed (training results still saved)")
        print("="*70)
        sys.exit(1)

    # Master report: one consolidated plots PDF + one full numeric-dump
    # PDF + CSVs, for handing to another analysis tool.
    print("\nBuilding master report (all plots + full numeric dump)...")
    mr = subprocess.run([sys.executable, "src/generate_master_report.py"], cwd=".")
    if mr.returncode != 0:
        print("PIPELINE WARNING: master report generation failed "
              "(other analysis outputs still saved)")

# Success
print("\n" + "="*70)
print("✓ FOUR-HEAD BENCHMARK PIPELINE COMPLETE")
print("="*70)
print("\nTraining results saved to:")
for run_num in range(1, TARGET_FOUR_HEAD_RUNS + 1):
    print(f" - runs/four_head/run_{run_num}/")
print("\nAnalysis saved to benchmark_analysis/ :")
print(" - 01_grokking_curves.png / 02_l2_norm_comparison.png / 03_dropout_gap_comparison.png")
print(" - benchmark_report.pdf : 3-page consistency report")
print(" - master_plots.pdf : every plot, per run + cross-run")
print(" - master_data.pdf : full literal per-epoch numeric dump")
print(" - data/*.csv : same numbers as CSV")
print("="*70 + "\n")

if __name__ == "__main__":
    main()

```

38. archive/Deprecated_Code/single_head/models/transformer.py

```
from torch import arange, nn
import torch

class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

    def forward(self, x):
        token_vectors = self.token_embedding(x)
        position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors
        query_vector = self.query(combined_vector)
        key_vector = self.key(combined_vector)
        value_vector = self.value(combined_vector)

        scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (query_vector.size(-1) ** 0.5)
        attention_weights = torch.softmax(scores, dim=-1)
        attention_values = combined_vector + self.dropout1(torch.matmul(attention_weights, value_vector))
        mlp_output = attention_values +
            self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))

        logits = self.output_head(mlp_output)

        return logits

if __name__ == "__main__":
    model = Transformer(vocab_size=98, d_model=128)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)
```

39. archive/Deprecated_Code/single_head/plot_results.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os

from predictors.l2_norm import compute_noise_floor, detect_ma_of_ma_zero_crossing

# Set up results directory paths
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
results_base = os.path.join(project_root, "results", "single_head")
training_dir = os.path.join(results_base, "training")
l2_norm_dir = os.path.join(results_base, "l2_norm")

# Load data already computed and saved by train.py – no re-run needed.
epoch_grid = np.load(os.path.join(l2_norm_dir, "epoch_grid.npy"))
slow_ma = np.load(os.path.join(l2_norm_dir, "slow_ma.npy"))
fast_ma_of_slow_ma = np.load(os.path.join(l2_norm_dir, "fast_ma_of_slow_ma.npy"))
diff = np.load(os.path.join(l2_norm_dir, "ma_of_ma_diff.npy"))
train_acc = np.load(os.path.join(training_dir, "train_acc_history.npy"))
test_acc = np.load(os.path.join(training_dir, "test_acc_history.npy"))
num_epochs = len(train_acc)

SKIP_EPOCHS = 100
QUIET_EPOCH_CUTOFF = 90

# Noise floor printed for context only – the trigger itself is a plain zero
# crossing now, not a magnitude threshold.
noise_floor = compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=QUIET_EPOCH_CUTOFF)
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=SKIP_EPOCHS)

# Plot 1: Slow MA vs. Fast MA of Slow MA, with the trigger point marked
fig, ax = plt.subplots(figsize=(12, 7))

ax.plot(epoch_grid, slow_ma, color="purple", linewidth=3.5, label="Slow MA (w=200)")
ax.plot(epoch_grid, fast_ma_of_slow_ma, color="orange", linewidth=3.5, label="Fast MA of Slow MA (w=20)")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax.scatter(trigger_epoch, slow_ma[trigger_idx], color="red", zorder=5, s=150, marker="*",
               label=f"Trigger (epoch {trigger_epoch:.0f})")

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)", fontsize=12)
ax.set_ylabel("L2 Norm", fontsize=12)
ax.set_title("Slow MA vs. Fast MA of Slow MA – Zero-Crossing Trigger", fontsize=13)
ax.legend(loc="upper right", fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(l2_norm_dir, "ma_of_slow_ma_crossover.png"), dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_crossover.png")

# Plot 2: The difference curve itself, with the zero-crossing trigger marked
fig2, ax2 = plt.subplots(figsize=(12, 7))

ax2.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax2.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax2.fill_between(epoch_grid, 0, diff, where=(diff < 0), alpha=0.2, color="red")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax2.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax2.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150, marker="*",
               label=f"Trigger (epoch {trigger_epoch:.0f})")

ax2.set_xscale("log")
ax2.set_xlabel("Epoch (log scale)", fontsize=12)
ax2.set_ylabel("Fast MA of Slow MA – Slow MA", fontsize=12)
ax2.set_title("Difference: Fast MA of Slow MA minus Slow MA – Zero-Crossing Trigger", fontsize=13)
ax2.legend(loc="upper right", fontsize=11)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(l2_norm_dir, "ma_of_slow_ma_diff.png"), dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_diff.png")

# Plot 3: Same difference curve as Plot 2, but on a LINEAR x-axis instead of log.
fig3, ax3 = plt.subplots(figsize=(12, 7))

ax3.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax3.axhline(y=0, color="black", linestyle="--", linewidth=1)
```

```

ax3.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax3.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax3.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax3.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150, marker="*",
                label=f"Trigger (epoch {trigger_epoch:.0f})")

ax3.set_xlabel("Epoch (linear scale)", fontsize=12)
ax3.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax3.set_title("Difference: Fast MA of Slow MA minus Slow MA - Linear Scale", fontsize=13)
ax3.legend(loc="upper right", fontsize=11)
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(l2_norm_dir, "ma_of_slow_ma_diff_linear.png"), dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_diff_linear.png")

# Plot 4: Difference curve (linear x-axis) overlaid on the grokking curve
# (train/test accuracy). Different scales (accuracy 0-1 vs. diff ~0.00-0.02),
# so accuracy uses the left y-axis and the diff uses a twin right y-axis.
grok_epoch = int(np.argmax(test_acc > 0.9))

fig4, ax4 = plt.subplots(figsize=(12, 7))

ax4.plot(range(1, num_epochs + 1), train_acc, color="steelblue", linewidth=2.5, label="Train Accuracy")
ax4.plot(range(1, num_epochs + 1), test_acc, color="seagreen", linewidth=2.5, label="Test Accuracy")
ax4.set_xlabel("Epoch (linear scale)", fontsize=12)
ax4.set_ylabel("Accuracy", fontsize=12)
ax4.set_ylim(-0.05, 1.05)

ax4b = ax4.twinx()
ax4b.plot(epoch_grid, diff, color="darkred", linewidth=3, label="Fast MA of Slow MA - Slow MA", alpha=0.85)
ax4b.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax4b.set_ylabel("MA-of-MA Difference", fontsize=12)

ax4.axvline(x=grok_epoch, color="seagreen", linestyle=":", linewidth=2, alpha=0.7)
if trigger_epoch is not None:
    ax4.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax4b.scatter(trigger_epoch, diff[np.argmin(np.abs(epoch_grid - trigger_epoch))],
                color="red", zorder=5, s=150, marker="*", label=f"Trigger (epoch {trigger_epoch:.0f})")

ax4.set_title(f"MA-of-MA Difference vs. Grokking Curve (Linear Scale) - Grok epoch {grok_epoch}", fontsize=13)
lines4, labels4 = ax4.get_legend_handles_labels()
lines4b, labels4b = ax4b.get_legend_handles_labels()
ax4.legend(lines4 + lines4b, labels4 + labels4b, loc="center right", fontsize=10)
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(l2_norm_dir, "ma_of_ma_diff_vs_grokking_linear.png"), dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_ma_diff_vs_grokking_linear.png")

print(f"\nNoise floor (context only): {noise_floor:.6f}")
if trigger_epoch is not None:
    print(f"Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
else:
    print("No zero-crossing detected")

```

40. archive/Deprecated_Code/single_head/train.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
from predictors.l2_norm import (
    compute_l2_norm,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from data.modular_arithmetic import get_data_loaders
from models.transformer import Transformer
from predictors.dropout import compute_dropout_gap_multi_rate
from unified_measurements import PredictorMeasurements

# Set up results directory paths
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
results_base = os.path.join(project_root, "results", "single_head")

# Initialize unified measurements
measurements = PredictorMeasurements(results_base, model_type="single_head")

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_data_loaders(97, batch_size=int(0.3 * 97 * 97))
model = Transformer(vocab_size=98, d_model=128).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 10000
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []
dropout_gap_epochs = []
dropout_gap_history = []
dropout_train_acc_history = []
dropout_eval_acc_history = []
dropout_rates = [0.1, 0.3, 0.5, 0.7, 0.9]
dropout_gap_history_by_rate = {rate: [] for rate in dropout_rates}

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())
    l2_norm_history.append(compute_l2_norm(model))

# Dropout Gap Predictor: now computed at every epoch (previously throttled)
```

```

# to every 100th epoch, because it needs two full passes over the test set
# - dropout ON, then dropout OFF - which is costly). Jonathan has since
# asked for full per-epoch resolution here as well, so this block now
# runs unconditionally. This will make each epoch noticeably slower than
# before, since two extra passes over the test set now happen every time.
# Now tracking multi-rate dropout gaps across five rates: [0.1, 0.3, 0.5, 0.7, 0.9]
results = compute_dropout_gap_multi_rate(
    model, data_loader[1], dropout_rates
)

for rate in dropout_rates:
    dropout_gap_history_by_rate[rate].append(
        results[rate]["dropout_gap"]
    )

# Maintain backward compatibility with single-rate record (rate=0.9)
dropout_gap = results[0.9]["dropout_gap"]
dropout_train_acc = results[0.9]["train_accuracy"]
dropout_eval_acc = results[0.9]["eval_accuracy"]
dropout_gap_epochs.append(epoch + 1)
dropout_gap_history.append(dropout_gap)
dropout_train_acc_history.append(dropout_train_acc)
dropout_eval_acc_history.append(dropout_eval_acc)

# compute_dropout_gap() leaves the model in eval() mode with p reset to
# 0.0. Training must resume in train() mode, so we restore it here before
# the next epoch's training pass begins.
model.train()

print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
      {train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
      {l2_norm_history[-1]:.4f}, Dropout Gap: {dropout_gap:.4f}")

# Save all measurements using unified system
measurements.save_training_data(train_acc_history, test_acc_history, loss_history)

grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

# L2 Norm Predictor: Moving Average Crossover Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR: Moving Average Crossover")
print("="*60)

# Compute fast and slow moving averages on a log-epoch-uniform grid
epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history, fast_window=50,
    slow_window=200)

# Detect crossover point: where fast MA crosses above slow MA
detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)

# Report results
print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f"  MA Crossover epoch: {detection_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {lead_time:.1f} epochs")
else:
    print("  No MA crossover detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

# Debug stats
print(f"\nL2 Norm Stats:")
print(f"  Min: {np.min(l2_norm_history):.4f}")
print(f"  Max: {np.max(l2_norm_history):.4f}")
print(f"  Final: {l2_norm_history[-1]:.4f}")

# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR: MA of Slow MA - Zero-Crossing Trigger")
print("="*60)

# Apply second MA on top of slow_ma
fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

# Noise floor calculation
noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

# Trigger: first zero-crossing after skip_epochs
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff, skip_epochs=100)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:

```

```

        trigger_lead_time = grok_epoch - trigger_epoch
        print(f" Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
        print(f" Grok epoch (test acc > 90%): {grok_epoch}")
        print(f" Lead time: {trigger_lead_time:.1f} epochs")
    else:
        print(" No trigger detected")
        print(f" Grok epoch (test acc > 90%): {grok_epoch}")

# Save all L2 Norm measurements
measurements.save_l2_norm_data(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch
)

# Dropout Predictor: gap tracked at every epoch during training
print("\n" + "="*60)
print("DROPOUT PREDICTOR: Gap tracked every epoch")
print("="*60)
for rate in dropout_rates:
    print(f"\nFinal Dropout Gap Check (rate={rate}, epoch {dropout_gap_epochs[-1]}):")
    print(f" Gap: {dropout_gap_history_by_rate[rate][-1]:.4f}")

# Save all Dropout measurements
measurements.save_dropout_data(
    dropout_gap_epochs, dropout_gap_history, dropout_gap_history_by_rate,
    dropout_train_acc_history, dropout_eval_acc_history, dropout_rates
)

# =====
# VISUALIZATION & REPORT GENERATION
# =====
print("\n" + "="*60)
print("VISUALIZATION & REPORT GENERATION")
print("="*60)

# Generate standalone L2 Norm visualizations
measurements.generate_l2_norm_visualizations(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
    grok_epoch, detection_epoch, trigger_epoch
)
print("[OK] L2 Norm visualizations saved to results/single_head/l2_norm/")

# Generate standalone Dropout visualizations
measurements.generate_dropout_visualizations(
    dropout_gap_epochs, dropout_gap_history_by_rate,
    dropout_rates, test_acc_history, grok_epoch
)
print("[OK] Dropout visualizations saved to results/single_head/dropout/")

# Generate combined PDF report
report_path = measurements.generate_combined_report(
    train_acc_history, test_acc_history, loss_history, l2_norm_history,
    dropout_gap_epochs, dropout_gap_history_by_rate, dropout_rates,
    epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch
)
print(f"[OK] PDF report saved to {report_path} (4 pages: all measurements)")

print("\n" + "="*60)
print("TRAINING COMPLETE")
print("="*60)
print(f"Grok epoch: {grok_epoch}")
print(f"L2 Norm MA crossover: {detection_epoch if detection_epoch else 'Not detected'}")
print(f"L2 Norm MA-of-MA trigger: {trigger_epoch if trigger_epoch else 'Not detected'}")
print(f"\nAll measurements saved to results/single_head/")
print(f" - training/ : raw training data")
print(f" - l2_norm/ : L2 norm measurements + visualizations")
print(f" - dropout/ : dropout measurements + visualizations")
print(f" - reports/ : combined PDF report")

```

41. archive/Deprecated_Code/src_p97_mainstream_old/data/modular_arithmetic.py

```
from torch import long, tensor
from torch.utils.data import Dataset, DataLoader, random_split

def generate_pairs(number):
    pairs = []
    for i in range(number):
        for j in range(number):
            pairs.append((i, j, (i + j) % number))
    return pairs

def get_dataloaders(number, batch_size):
    modular_arithmetic_dataset = ModularArithmeticDataset(number)
    train_size = int(0.3 * len(modular_arithmetic_dataset))
    test_size = len(modular_arithmetic_dataset) - train_size

    train_dataset, test_dataset = random_split(modular_arithmetic_dataset, [train_size, test_size])
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_dataloader = DataLoader(test_dataset, batch_size=batch_size)

    return train_dataloader, test_dataloader

class ModularArithmeticDataset(Dataset):
    def __init__(self, number):
        self.pairs = generate_pairs(number)

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        return (self.get_tensor(self.pairs[idx]), self.pairs[idx][2])

    def get_tensor(self, item):
        sequence = [item[0], item[1], 97]
        return tensor(sequence)

if __name__ == "__main__":
    print(tensor([5, 3, 8, 97]))
```

42. archive/Deprecated_Code/src_p97_mainstream_old/models/model_shadow_with_layernorm.py

```
from torch import arange, nn
import torch

# SHADOW - LayerNorm added, original had no norm
USE_FINAL_LN = False

class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

        self.ln1 = nn.LayerNorm(d_model) # SHADOW - LayerNorm added, original had no norm
        self.ln2 = nn.LayerNorm(d_model) # SHADOW - LayerNorm added, original had no norm
        self.ln_final = nn.LayerNorm(d_model) # SHADOW - LayerNorm added, original had no norm

    def forward(self, x):
        token_vectors = self.token_embedding(x)
        position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors
        query_vector = self.query(combined_vector)
        key_vector = self.key(combined_vector)
        value_vector = self.value(combined_vector)

        scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (query_vector.size(-1) ** 0.5)
        attention_weights = torch.softmax(scores, dim=-1)
        attention_values = combined_vector + self.dropout1(torch.matmul(attention_weights, value_vector))
        attention_values = self.ln1(attention_values) # SHADOW - LayerNorm added, original had no norm
        mlp_output = attention_values + self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))
        mlp_output = self.ln2(mlp_output) # SHADOW - LayerNorm added, original had no norm

        if USE_FINAL_LN:
            mlp_output = self.ln_final(mlp_output) # SHADOW - LayerNorm added, original had no norm

        logits = self.output_head(mlp_output)

        return logits

if __name__ == "__main__":
    model = Transformer(vocab_size=98, d_model=128)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

    # Quick 100 epoch sanity check - should not crash
    # Should reach >90% test acc and ideally break the 99.5446% plateau
    pass
```

43. archive/Deprecated_Code/src_p97_mainstream_old/models/transformer_four_head.py

```
from torch import arange, nn
import torch

# =====
# 4-HEAD VARIANT - matches Nanda et al.'s actual architecture
# (d_model=128, 4 attention heads, head dimension = 128/4 = 32 each).
#
# The original src/models/transformer.py is UNCHANGED and still uses
# single-head attention (one Query/Key/Value block covering the full
# d_model=128 at once). That file is left exactly as it is - this is a
# separate, new model class, following the same "shadow copy" pattern
# already used in this project for the LayerNorm side-experiment
# (see src/models/model_shadow_with_layernorm.py).
#
# Everything else is kept identical to the original design, so that the
# ONLY architectural difference between the two models is the number of
# attention heads: same token + position embedding, same MLP (4x
# expansion + ReLU), no LayerNorm, no Linear biases, same dropout1/
# dropout2 hooks (used by predictors/dropout.py's stress-test), same
# output head.
# =====

class TransformerFourHead(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads=4):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

    def split_into_heads(self, x):
        # x: (batch_size, seq_len, d_model) -> (batch_size, num_heads, seq_len, head_dim)
        # This is the step that turns one big d_model-wide attention into
        # num_heads smaller, independent attention computations running
        # side by side.
        batch_size, seq_len, d_model = x.shape
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.permute(0, 2, 1, 3)

    def combine_heads(self, x):
        # x: (batch_size, num_heads, seq_len, head_dim) -> (batch_size, seq_len, d_model)
        # Reverses split_into_heads - puts all heads' outputs back side by
        # side into one d_model-wide vector, exactly as Nanda et al.
        # describe combining the 4 heads' outputs before the MLP.
        batch_size, num_heads, seq_len, head_dim = x.shape
        x = x.permute(0, 2, 1, 3)
        return x.reshape(batch_size, seq_len, num_heads * head_dim)

    def forward(self, x):
        token_vectors = self.token_embedding(x)
        position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors

        query_vector = self.split_into_heads(self.query(combined_vector))
        key_vector = self.split_into_heads(self.key(combined_vector))
        value_vector = self.split_into_heads(self.value(combined_vector))

        # Scaling uses head_dim (32), not the full d_model (128) - each
        # head does its own scaled dot-product attention independently.
        scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (self.head_dim ** 0.5)
        attention_weights = torch.softmax(scores, dim=-1)
        attention_output = torch.matmul(attention_weights, value_vector)
        attention_output = self.combine_heads(attention_output)

        attention_values = combined_vector + self.dropout1(attention_output)
        mlp_output = attention_values +
            self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))
        logits = self.output_head(mlp_output)
```

```
        return logits

if __name__ == "__main__":
    model = TransformerFourHead(vocab_size=98, d_model=128, num_heads=4)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)
```

44. archive/Deprecated_Code/src_p97_mainstream_old/plot_results_four_head.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os
import re

from predictors.l2_norm import compute_noise_floor, detect_ma_of_ma_zero_crossing, detect_ma_crossover

# =====
# 4-HEAD VARIANT of plot_results.py – now MULTI-RUN aware.
#
# train_four_head.py saves every run into its own numbered subfolder,
# results/four_head/run_1/, results/four_head/run_2/, and so on (see that
# file's RUN NUMBERING comment for why – grokking is stochastic, so a
# real comparison needs several independent runs kept side by side, not
# one run's numbers overwritten by the next).
#
# This script does two things:
# 1. For every run <N> folder it finds, regenerates that run's own 8
#    plots (same as before), saved inside that run's own folder.
# 2. Builds 4 NEW comparison plots at the results/four_head/ level,
#    overlaying every run's grokking curve / loss curve / L2 norm
#    curve / Dropout Gap curve together, so all previous runs can be
#    read on one chart instead of opening each run's folder one by one.
# =====

project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
four_head_dir = os.path.join(project_root, "runs", "four_head")

SKIP_EPOCHS = 100
QUIET_EPOCH_CUTOFF = 90

LEGACY_FILENAMES = [
    "train_acc_history.npy", "test_acc_history.npy", "loss_history.npy",
    "l2_norm_history.npy", "dropout_gap_epochs.npy", "dropout_gap_history.npy",
    "dropout_train_acc_history.npy", "dropout_eval_acc_history.npy",
    "epoch_grid.npy", "fast_ma.npy", "slow_ma.npy", "fast_ma_of_slow_ma.npy",
    "ma_of_ma_diff.npy", "training_report.pdf",
    "ma_of_slow_ma_crossover.png", "ma_of_slow_ma_diff.png",
    "ma_of_slow_ma_diff_linear.png", "ma_of_ma_diff_vs_grokking_linear.png",
    "grokking_curve.png", "loss_curve.png", "l2_norm_curve.png", "dropout_gap_curve.png",
]

# Distinct colors cycled across runs, so Run 1 / Run 2 / Run 3 ... always
# look different from each other on the comparison plots.
RUN_COLORS = ["steelblue", "darkorange", "seagreen", "crimson", "purple",
              "goldenrod", "teal", "indianred", "slategray", "mediumvioletred"]

def migrate_legacy_flat_run(base_dir):
    """Same migration as train_four_head.py – kept here too so this
    script can be re-run on its own and still pick up an old flat run
    even if train_four_head.py has not been re-run since the update."""
    run_1_dir = os.path.join(base_dir, "run_1")
    if os.path.isdir(run_1_dir):
        return
    legacy_present = any(
        os.path.isfile(os.path.join(base_dir, name)) for name in LEGACY_FILENAMES
    )
    if not legacy_present:
        return
    os.makedirs(run_1_dir, exist_ok=True)
    for name in LEGACY_FILENAMES:
        src = os.path.join(base_dir, name)
        if os.path.isfile(src):
            os.rename(src, os.path.join(run_1_dir, name))
    print("[MIGRATION] Found results saved directly in results/four_head/ by an earlier "
          "version of these scripts – moved them into results/four_head/run_1/ so they "
          "are counted as Run 1.")

def discover_run_dirs(base_dir):
    """Finds every run <N> folder inside base_dir, sorted by run number
    (not alphabetically, so run_2 comes before run_10)."""
    if not os.path.isdir(base_dir):
        return []
    found = []
    for name in os.listdir(base_dir):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(base_dir, name)):
            found.append((int(match.group(1)), os.path.join(base_dir, name)))
    found.sort(key=lambda pair: pair[0])
    return found
```

```

def load_run_data(run_dir):
    # unified_measurements.PredictorMeasurements writes into per-predictor
    # subfolders: training/, l2_norm/, dropout/. Each entry below is
    # (key, relative_path).
    required_files = [
        ("train_acc", "training/train_acc_history.npy"), ("test_acc", "training/test_acc_history.npy"),
        ("loss", "training/loss_history.npy"), ("l2_norm", "l2_norm/l2_norm_history.npy"),
        ("epoch_grid", "l2_norm/epoch_grid.npy"), ("fast_ma", "l2_norm/fast_ma.npy"),
        ("slow_ma", "l2_norm/slow_ma.npy"), ("fast_ma_of_slow_ma", "l2_norm/fast_ma_of_slow_ma.npy"),
        ("ma_of_ma_diff", "l2_norm/ma_of_ma_diff.npy"),
        ("dropout_gap_epochs", "dropout/dropout_gap_epochs.npy"),
        ("dropout_gap_by_rate", "dropout/dropout_gap_by_rate.npy"),
        ("dropout_rates", "dropout/dropout_rates.npy"),
    ]

    for _, rel in required_files:
        if not os.path.exists(os.path.join(run_dir, rel)):
            return None

    data = {}
    for key, rel in required_files:
        data[key] = np.load(os.path.join(run_dir, rel))
    data["num_epochs"] = len(data["train_acc"])
    return data

def plot_single_run(run_number, run_dir, data):
    """Recreates this run's own 8 plots, saved inside results/four_head/run_&N&gt;/ itself."""
    epoch_grid = data["epoch_grid"]
    fast_ma = data["fast_ma"]
    slow_ma = data["slow_ma"]
    fast_ma_of_slow_ma = data["fast_ma_of_slow_ma"]
    diff = data["ma_of_ma_diff"]
    train_acc = data["train_acc"]
    test_acc = data["test_acc"]
    loss_history = data["loss"]
    l2_norm_history = data["l2_norm"]
    dropout_gap_epochs = data["dropout_gap_epochs"]
    dropout_gap_by_rate = data["dropout_gap_by_rate"] # shape [rate, epoch]
    dropout_rates = data["dropout_rates"]
    num_epochs = data["num_epochs"]
    epochs_axis = range(1, num_epochs + 1)

    noise_floor = compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=QUIET_EPOCH_CUTOFF)
    trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=SKIP_EPOCHS)
    detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=SKIP_EPOCHS)
    grokked = bool((test_acc > 0.9).any())
    grok_epoch = int(np.argmax(test_acc > 0.9)) if grokked else None

    def save(fig, filename):
        path = os.path.join(run_dir, filename)
        fig.savefig(path, dpi=150, bbox_inches="tight")
        plt.close(fig)
        print(f"[OK] Run {run_number}: plot saved to results/four_head/run_{run_number}/{filename}")

    # Plot 1: Slow MA vs. Fast MA of Slow MA
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epoch_grid, slow_ma, color="purple", linewidth=3.5, label="Slow MA (w=200)")
    ax.plot(epoch_grid, fast_ma_of_slow_ma, color="orange", linewidth=3.5, label="Fast MA of Slow MA (w=20)")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
        ax.scatter(trigger_epoch, slow_ma[trigger_idx], color="red", zorder=5, s=150, marker="*",
            label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("L2 Norm", fontsize=12)
    ax.set_title(f"4-head model, Run {run_number}: Slow MA vs. Fast MA of Slow MA", fontsize=13)
    ax.legend(loc="upper right", fontsize=11)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig, "ma_of_slow_ma_crossover.png")

    # Plot 2: Difference curve, log-x
    fig2, ax2 = plt.subplots(figsize=(12, 7))
    ax2.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
    ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
    ax2.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
    ax2.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax2.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
        ax2.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150, marker="*",
            label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax2.set_xscale("log")
    ax2.set_xlabel("Epoch (log scale)", fontsize=12)

```

```

ax2.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax2.set_title(f"4-head model, Run {run_number}: Difference - Fast MA of Slow MA minus Slow MA", fontsize=13)
ax2.legend(loc="upper right", fontsize=11)
ax2.grid(True, alpha=0.3)
plt.tight_layout()
save(fig2, "ma_of_slow_ma_diff.png")

# Plot 3: Same difference curve, linear x-axis
fig3, ax3 = plt.subplots(figsize=(12, 7))
ax3.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax3.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax3.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax3.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")
if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax3.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax3.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150, marker="*",
               label=f"Trigger (epoch {trigger_epoch:.0f})")
ax3.set_xlabel("Epoch (linear scale)", fontsize=12)
ax3.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax3.set_title(f"4-head model, Run {run_number}: Difference - Linear Scale", fontsize=13)
ax3.legend(loc="upper right", fontsize=11)
ax3.grid(True, alpha=0.3)
plt.tight_layout()
save(fig3, "ma_of_slow_ma_diff_linear.png")

# Plot 4: Difference curve overlaid on the grokking curve
fig4, ax4 = plt.subplots(figsize=(12, 7))
ax4.plot(range(1, num_epochs + 1), train_acc, color="steelblue", linewidth=2.5, label="Train Accuracy")
ax4.plot(range(1, num_epochs + 1), test_acc, color="seagreen", linewidth=2.5, label="Test Accuracy")
ax4.set_xlabel("Epoch (linear scale)", fontsize=12)
ax4.set_ylabel("Accuracy", fontsize=12)
ax4.set_ylim(-0.05, 1.05)
ax4b = ax4.twinx()
ax4b.plot(epoch_grid, diff, color="darkred", linewidth=3, label="Fast MA of Slow MA - Slow MA", alpha=0.85)
ax4b.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax4b.set_ylabel("MA-of-MA Difference", fontsize=12)
if grokked:
    ax4.axvline(x=grok_epoch, color="seagreen", linestyle=":", linewidth=2, alpha=0.7)
if trigger_epoch is not None:
    ax4.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax4b.scatter(trigger_epoch, diff[np.argmin(np.abs(epoch_grid - trigger_epoch))],
               color="red", zorder=5, s=150, marker="*", label=f"Trigger (epoch {trigger_epoch:.0f})")
title_suffix = f"Grok epoch {grok_epoch}" if grokked else "did not grok in this run"
ax4.set_title(f"4-head model, Run {run_number}: MA-of-MA Difference vs. Grokking Curve - {title_suffix}",
             fontsize=13)
lines4, labels4 = ax4.get_legend_handles_labels()
lines4b, labels4b = ax4b.get_legend_handles_labels()
ax4.legend(lines4 + lines4b, labels4 + labels4b, loc="center right", fontsize=10)
ax4.grid(True, alpha=0.3)
plt.tight_layout()
save(fig4, "ma_of_ma_diff_vs_grokking_linear.png")

# Plot 5: Grokking curve
fig5, ax5 = plt.subplots(figsize=(12, 7))
ax5.plot(epochs_axis, train_acc, label="Train Accuracy", color="steelblue", linewidth=2)
ax5.plot(epochs_axis, test_acc, label="Test Accuracy", color="seagreen", linewidth=2)
ax5.set_xscale("log")
ax5.set_xlabel("Epoch (log scale)")
ax5.set_ylabel("Accuracy")
ax5.set_title(f"4-head model, Run {run_number}: Grokking Curve - Train vs. Test Accuracy")
ax5.legend(loc="center right")
ax5.grid(True, alpha=0.3)
plt.tight_layout()
save(fig5, "grokking_curve.png")

# Plot 6: Loss curve
fig6, ax6 = plt.subplots(figsize=(12, 7))
ax6.plot(epochs_axis, loss_history, color="darkorange", linewidth=2)
ax6.set_xscale("log")
ax6.set_xlabel("Epoch (log scale)")
ax6.set_ylabel("Loss")
ax6.set_title(f"4-head model, Run {run_number}: Training Loss")
ax6.grid(True, alpha=0.3)
plt.tight_layout()
save(fig6, "loss_curve.png")

# Plot 7: L2 Norm curve, with MA Crossover marked
fig7, ax7 = plt.subplots(figsize=(12, 7))
ax7.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
if detection_epoch is not None:
    ax7.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
               label=f"MA Crossover (epoch {detection_epoch:.0f})")
ax7.set_xscale("log")
ax7.set_xlabel("Epoch (log scale)")
ax7.set_ylabel("L2 Norm")

```

```

ax7.set_title(f"4-head model, Run {run_number}: L2 Norm of Model Weights")
ax7.legend(loc="upper right")
ax7.grid(True, alpha=0.3)
plt.tight_layout()
save(fig7, "l2_norm_curve.png")

# Plot 8: Dropout Gap – full multi-rate sweep
fig8, ax8 = plt.subplots(figsize=(12, 7))
for r_idx, rate in enumerate(dropout_rates):
    ax8.plot(dropout_gap_epochs, dropout_gap_by_rate[r_idx], linewidth=2,
             label=f"p={rate:g}")
ax8.axhline(y=0, color="black", linewidth=0.6, alpha=0.5)
ax8.set_xscale("log")
ax8.set_xlabel("Epoch (log scale)")
ax8.set_ylabel("Dropout Gap")
ax8.set_title(f"4-head model, Run {run_number}: Dropout Gap – Multi-Rate Sweep")
ax8.legend()
ax8.grid(True, alpha=0.3)
plt.tight_layout()
save(fig8, "dropout_gap_curve.png")

final_gap_by_rate = {float(rate): float(dropout_gap_by_rate[r_idx][-1])
                     for r_idx, rate in enumerate(dropout_rates)}

return {
    "run_number": run_number, "num_epochs": num_epochs, "grokked": grokked,
    "grok_epoch": grok_epoch, "final_test_acc": float(test_acc[-1]),
    "final_l2_norm": float(l2_norm_history[-1]),
    "final_dropout_gap_by_rate": final_gap_by_rate,
}

def plot_comparison(all_runs_data, out_dir):
    """Builds 4 plots overlaying every discovered run together, saved
    directly in results/four_head/ (not inside any single run's folder),
    so all previous runs can be read on one chart."""

    if not all_runs_data:
        print("[SKIP] No complete runs to compare – comparison plots not generated.")
        return

    def color_for(i):
        return RUN_COLORS[i % len(RUN_COLORS)]

    # Comparison Plot A: Test Accuracy (grokking curve) across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
        color = color_for(i)
        ax.plot(epochs_axis, data["test_acc"], color=color, linewidth=2.2,
              label=f"Run {run_number} ({data['num_epochs']} epochs)")
        if (data["test_acc"] > 0.9).any():
            grok_epoch = int(np.argmax(data["test_acc"] > 0.9))
            ax.axvline(x=grok_epoch, color=color, linestyle=":", linewidth=1.5, alpha=0.6)
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Test Accuracy", fontsize=12)
    ax.set_ylim(-0.05, 1.05)
    ax.set_title("4-head model: Test Accuracy Across All Runs", fontsize=13)
    ax.legend(loc="center right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "comparison_grokking_curve.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to results/four_head/comparison_grokking_curve.png")

    # Comparison Plot B: Loss across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
        ax.plot(epochs_axis, data["loss"], color=color_for(i), linewidth=2,
              label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Loss", fontsize=12)
    ax.set_title("4-head model: Training Loss Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "comparison_loss_curve.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to results/four_head/comparison_loss_curve.png")

    # Comparison Plot C: L2 Norm across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)

```

```

        ax.plot(epochs_axis, data["l2_norm"], color=color_for(i), linewidth=2,
                label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("L2 Norm", fontsize=12)
    ax.set_title("4-head model: L2 Norm of Model Weights Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "comparison_l2_norm_curve.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to results/four_head/comparison_l2_norm_curve.png")

# Comparison Plot D: Dropout Gap sweep – one panel per run, all rates
n = len(all_runs_data)
fig, axes = plt.subplots(1, n, figsize=(6 * n, 5), squeeze=False)
for i, (run_number, data) in enumerate(all_runs_data):
    ax = axes[0][i]
    rates = data["dropout_rates"]
    gap_by_rate = data["dropout_gap_by_rate"]
    for r_idx, rate in enumerate(rates):
        ax.plot(data["dropout_gap_epochs"], gap_by_rate[r_idx], linewidth=1.8,
                alpha=0.9, label=f"p={rate:g}")
    ax.axhline(y=0, color="black", linewidth=0.6, alpha=0.5)
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=11)
    ax.set_ylabel("Dropout Gap", fontsize=11)
    ax.set_title(f"Run {run_number} ({data['num_epochs']} epochs)", fontsize=12)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)
fig.suptitle("4-head model: Dropout Gap Multi-Rate Sweep Across All Runs", fontsize=13)
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "comparison_dropout_gap_curve.png"), dpi=150, bbox_inches="tight")
plt.close(fig)
print("[OK] Comparison plot saved to results/four_head/comparison_dropout_gap_curve.png")

migrate_legacy_flat_run(four_head_dir)
run_dirs = discover_run_dirs(four_head_dir)

if not run_dirs:
    print(f"No runs found under results/four_head/. Run train_four_head.py at least once first.")
else:
    print(f"Found {len(run_dirs)} run(s): {' '.join(f'Run {n}' for n, _ in run_dirs)}\n")

    all_runs_data = []
    summaries = []
    for run_number, run_dir in run_dirs:
        data = load_run_data(run_dir)
        if data is None:
            print(f"Skipping Run {run_number} (incomplete or empty)")
            continue
        all_runs_data.append((run_number, data))
        summaries.append(plot_single_run(run_number, run_dir, data))

    plot_comparison(all_runs_data, four_head_dir)

    print("\n" + "=" * 60)
    print("SUMMARY ACROSS ALL RUNS")
    print("=" * 60)
    for s in summaries:
        grok_text = f"grokked at epoch {s['grok_epoch']}" if s["grokked"] else "did NOT grok"
        gap_text = ", ".join(f"p{rate:g}={gap:+.4f}"
                             for rate, gap in sorted(s["final_dropout_gap_by_rate"].items()))
        print(f"Run {s['run_number']} ({s['num_epochs']} epochs): {grok_text}, "
              f"final test acc = {s['final_test_acc']:.4f}, "
              f"final L2 norm = {s['final_l2_norm']:.4f}")
        print(f"final dropout gap sweep: {gap_text}")

    print(f"\n[OK] Per-run plots saved inside each results/four_head/run_&N& folder.")
    print(f"[OK] 4 comparison plots saved directly in results/four_head/:")
    print(" - comparison_grokking_curve.png")
    print(" - comparison_loss_curve.png")
    print(" - comparison_l2_norm_curve.png")
    print(" - comparison_dropout_gap_curve.png")

```

45. archive/Deprecated_Code/src_p97_mainstream_old/plot_shadow_layernorm.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os

# Look for files in project root (parent of src/), same pattern as plot_results.py
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Known plateau value from the original (no-LayerNorm) model, recorded in
# context.md (Aug 7, 2026 plateau-investigation session): 6557/6587 correct.
ORIGINAL_PLATEAU_ACC = 0.995446

shadow_train_acc = np.load("shadow_ln_train_acc_history.npy")
shadow_test_acc = np.load("shadow_ln_test_acc_history.npy")
num_epochs = len(shadow_train_acc)
epochs_axis = range(1, num_epochs + 1)

fig, ax = plt.subplots(figsize=(12, 7))

ax.plot(epochs_axis, shadow_train_acc, color="steelblue", linewidth=2.5, label="Shadow (LayerNorm) Train Accuracy")
ax.plot(epochs_axis, shadow_test_acc, color="seagreen", linewidth=2.5, label="Shadow (LayerNorm) Test Accuracy")

# Reference line: the original no-LayerNorm model's known plateau value.
ax.axhline(y=ORIGINAL_PLATEAU_ACC, color="darkred", linestyle="--", linewidth=2,
           label=f"Original plateau (no LayerNorm): {ORIGINAL_PLATEAU_ACC:.4%}")

# If the original run's test_acc_history.npy is present in the project root,
# overlay it too, so both curves can be compared directly on one graph.
# This is optional – the shadow curve above still plots fine without it.
try:
    original_test_acc = np.load("test_acc_history.npy")
    original_epochs_axis = range(1, len(original_test_acc) + 1)
    ax.plot(original_epochs_axis, original_test_acc, color="darkorange", linewidth=2,
            linestyle=":", label="Original (no LayerNorm) Test Accuracy", alpha=0.85)
except FileNotFoundError:
    print("Note: test_acc_history.npy (original run) not found – skipping overlay, "
          "shadow curve plotted alone.")

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)", fontsize=12)
ax.set_ylabel("Accuracy", fontsize=12)
ax.set_ylim(-0.05, 1.05)
ax.set_title("Shadow Model (with LayerNorm) – Grokking Curve vs. Original Plateau", fontsize=13)
ax.legend(loc="center right", fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("shadow_ln_grokking_curve.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to shadow_ln_grokking_curve.png")

final_shadow_test_acc = shadow_test_acc[-1]
print(f"\nFinal shadow (LayerNorm) test accuracy: {final_shadow_test_acc:.6f}")
print(f"Original (no LayerNorm) plateau: {ORIGINAL_PLATEAU_ACC:.6f}")
if final_shadow_test_acc > ORIGINAL_PLATEAU_ACC:
    print("Shadow model beat the original plateau.")
elif final_shadow_test_acc == ORIGINAL_PLATEAU_ACC:
    print("Shadow model matched the original plateau exactly.")
else:
    print("Shadow model did not reach the original plateau.")
```

46. archive/Deprecated_Code/src_p97_mainstream_old/predictors/dropout.py

```
import torch

def compute_accuracy(model, data_loader):
    device = next(model.parameters()).device
    total_correct = 0
    total_samples = 0
    for x, y in data_loader:
        x, y = x.to(device), y.to(device)
        with torch.no_grad():
            logit = model.forward(x)
            equal_sign_logit = logit[:, 2, :]
            predicted = equal_sign_logit.argmax(dim=1)
            total_correct += (predicted == y).sum().item()
            total_samples += len(y)
    return total_correct / total_samples if total_samples > 0 else 0.0

# NOTE: the old single-rate compute_dropout_gap(model, data_loader, dropout_rate)
# has been removed. The Dropout predictor is now a full multi-rate sweep only -
# no hardcoded p=0.9 anywhere. Use compute_dropout_gap_multi_rate.

def compute_dropout_gap_multi_rate(model, data_loader, dropout_rates):
    # Step 1: START (inputs: model, data_loader, dropout_rates)

    # Step 2: Setup Evaluation Mode
    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0

    # Step 3: Compute Clean Accuracy (execute ONLY ONCE)
    clean_accuracy = compute_accuracy(model, data_loader)

    # Step 4: Initialize Results Storage
    results = {}

    # Step 5: FOR each rate in dropout_rates
    for dropout_rate in dropout_rates:
        # --- Switch to train mode with dropout active ---
        model.train()
        model.dropout1.p = dropout_rate
        model.dropout2.p = dropout_rate

        # Compute train accuracy (with dropout)
        train_accuracy = compute_accuracy(model, data_loader)

        # Compute dropout gap (train_accuracy - clean_accuracy)
        dropout_gap = train_accuracy - clean_accuracy

        # Store results
        results[dropout_rate] = {
            "train_accuracy": train_accuracy,
            "eval_accuracy": clean_accuracy,      # == clean_accuracy (dropout=0, eval mode)
            "dropout_gap": dropout_gap,
            "clean_accuracy": clean_accuracy
        }

    # Step 6: Loop Decision - "More rates?" (handled by for-loop)

    # Step 7: Final Cleanup - restore model to clean eval state
    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0

    # Step 8: RETURN results - END
    return results
```

47. archive/Deprecated_Code/src_p97_mainstream_old/predictors/l2_norm.py

```
import numpy as np
import torch
from scipy.ndimage import uniform_filter1d

def compute_l2_norm(model):
    all_params = torch.cat([p.flatten() for p in model.parameters()])
    l2_norm = torch.norm(all_params).item()
    return l2_norm

def compute_l2_norm_rate_of_decline(l2_norm_history):
    return np.diff(l2_norm_history) * -1

def detect_l2_norm_drop(rate_of_decline, threshold=0.05, skip_epochs=200):
    for i, rate in enumerate(rate_of_decline):
        if i < skip_epochs: # Skip early epochs
            continue
        if rate > threshold:
            return i
    return None

def compute_acceleration(rate_of_decline):
    """
    Compute the second derivative of L2 norm (acceleration).
    acceleration[i] = rate[i] - rate[i-1]
    Positive = decay rate is increasing (speeding up)
    Negative = decay rate is decreasing (slowing down)
    Sign change = inflection point (change in decay character)
    """
    acceleration = np.diff(rate_of_decline)
    return acceleration

def detect_inflection(acceleration, skip_epochs=200):
    """
    Detect inflection point where acceleration changes sign.
    This is where the decay pattern changes character – exactly what happens
    before/during the grok transition.
    Returns the epoch index where the sign flip occurs, or None.
    """
    if len(acceleration) <= skip_epochs:
        return None

    # Check from skip_epochs onwards for sign changes
    for i in range(skip_epochs, len(acceleration)):
        # acceleration[i-1] * acceleration[i] < 0 means opposite signs (sign flip)
        if i > 0 and acceleration[i-1] * acceleration[i] < 0:
            return i

    return None

def apply_moving_average(data, window_size=50):
    """
    Apply moving average smoothing to reduce noise.
    window_size: number of epochs to average over (default 50).
    Returns smoothed array (same length as input, padded at edges).
    """
    return uniform_filter1d(np.array(data), size=window_size, mode='nearest')

def resample_to_log_uniform_grid(data, num_points=None):
    """
    Resample data onto a grid uniformly spaced in log10(epoch) – i.e. the
    SAME spacing the data has when viewed on a log-x plot.
    Without this, a fixed window_size (in raw epoch index) covers a huge
    visual chunk of the plot near epoch 1 and a tiny sliver near epoch N,
    which is why smoothing looked over-smoothed early and scribbly late.
    Returns (epoch_grid, resampled_data) where epoch_grid is uniform in
    log10-space (non-uniform in linear epoch terms).
    """
    data = np.array(data)
    num_epochs = len(data)
    epochs = np.arange(1, num_epochs + 1)
    log_epochs = np.log10(epochs)

    if num_points is None:
        num_points = num_epochs

    log_grid = np.linspace(log_epochs[0], log_epochs[-1], num_points)
    resampled = np.interp(log_grid, log_epochs, data)
    epoch_grid = 10 ** log_grid
    return epoch_grid, resampled
```

```

def compute_fast_slow_moving_averages(l2_norm_history, fast_window=50, slow_window=200, num_points=None):
    """
    Compute two moving averages on a LOG-EPOCH-UNIFORM grid, so the window
    covers equal VISUAL width everywhere on a log-x plot (fixes both the
    over-smoothing near epoch 1 and the scribbling near epoch N).

    Steps:
    1. Log-transform L2 norm values (handles the exponential decay in y).
    2. Resample onto a grid uniform in log10(epoch) (handles the log-x axis).
    3. Apply the moving average on that resampled grid.
    4. Convert back to linear L2 norm space.

    fast_window / slow_window are now measured in grid points, which (since
    the grid is log-uniform) corresponds to a constant *proportional* width
    in epochs everywhere along the curve.

    Returns (epoch_grid, fast_ma, slow_ma) – always plot fast_ma/slow_ma
    against epoch_grid, not against range(1, num_epochs+1).
    """
    l2_norm_history = np.array(l2_norm_history)
    log_l2_norm = np.log(l2_norm_history)

    epoch_grid, log_l2_norm_resampled = resample_to_log_uniform_grid(log_l2_norm, num_points=num_points)

    fast_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=fast_window)
    slow_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=slow_window)

    fast_ma = np.exp(fast_ma_log)
    slow_ma = np.exp(slow_ma_log)

    return epoch_grid, fast_ma, slow_ma


def detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100):
    """
    Detect where fast MA crosses above slow MA, searching along the
    log-uniform grid. skip_epochs is a real epoch value (not a grid index) –
    crossovers before this epoch are ignored (initialization noise).
    Returns the real epoch (float) of the crossover, or None.
    """
    if len(fast_ma) != len(slow_ma):
        return None

    for i in range(len(fast_ma) - 1):
        if epoch_grid[i] <= skip_epochs:
            continue
        # Previous: fast <= slow, Current: fast > slow (crossover from below)
        if fast_ma[i] <= slow_ma[i] and fast_ma[i + 1] > slow_ma[i + 1]:
            return epoch_grid[i + 1]

    return None


def compute_ma_of_slow_ma(slow_ma, fast_window=20):
    """
    Apply a second, faster-window moving average on top of the already-smoothed
    slow_ma. Since slow_ma has already filtered out raw L2 norm noise, this
    reveals slow_ma's OWN turning points cleanly – the difference between the
    two tracks how sharply slow_ma is currently bending.
    Returns (fast_ma_of_slow_ma, diff) where diff = fast_ma_of_slow_ma - slow_ma.
    """
    fast_ma_of_slow_ma = apply_moving_average(slow_ma, window_size=fast_window)
    diff = fast_ma_of_slow_ma - slow_ma
    return fast_ma_of_slow_ma, diff


def compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=90):
    """
    Estimate the normal noise level of the MA-of-MA difference from the quiet
    early-training region (before any real dynamics begin), so later spikes
    can be judged against what "flat" actually looks like for this run.
    """
    quiet_mask = epoch_grid < quiet_epoch_cutoff
    return np.std(diff[quiet_mask])


def detect_ma_of_ma_trigger(epoch_grid, diff, noise_floor, threshold_multiplier=10, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff climbs past
    threshold_multiplier x noise_floor – the rising edge of a real signal.
    Causal: only needs the noise floor (known early) and points seen so far,
    unlike picking the curve's peak, which requires seeing the whole future.
    Returns the real epoch (float) of the trigger, or None.
    """
    threshold = threshold_multiplier * noise_floor
    for i in range(len(diff)):

```

```

        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] > threshold:
            return epoch_grid[i]
    return None

def detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff crosses from
    positive to negative (the first "green -> red" zero crossing).
    Confirmed on two separate training runs to land in a consistent,
    unambiguous spot even though the crossing's magnitude is weak.
    Returns the real epoch (float) of the crossing, or None.
    """
    for i in range(len(diff) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] >= 0 and diff[i + 1] < 0:
            return epoch_grid[i + 1]
    return None

```

48. archive/Deprecated_Code/src_p97_mainstream_old/train_four_head.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
import re
from predictors.l2_norm import (
    compute_l2_norm,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from data.modular_arithmetic import get_dataloaders
from models.transformer_four_head import TransformerFourHead
from predictors.dropout import compute_dropout_gap_multi_rate
from unified_measurements import PredictorMeasurements

# =====
# 4-HEAD VARIANT of train.py.
#
# This script is a separate copy of the main training loop – it does NOT
# modify train.py or any of its saved outputs. The only real difference
# from train.py is the model: TransformerFourHead(num_heads=4) instead of
# the original single-head Transformer. Same task (p=97), same optimiser
# (AdamW, lr=1e-3, weight_decay=1.0), same full-batch training, same 30/70
# split, same L2 Norm and Dropout Gap predictor code (reused as-is from
# src/predictors/ – nothing predictor-specific needed to change, since
# both predictors work on any model that exposes .parameters() and
# .dropout1/.dropout2, which this model does too).
#
# Outputs are saved under results/four_head/run_&Nt;/ (created if it does not
# exist yet), NOT in the project root, so they never overwrite train.py's
# own train_acc_history.npy / l2_norm_history.npy / dropout_gap_history.npy
# / etc., which belong to the original single-head run.
#
# RUN NUMBERING: every time this script is run, it saves into a NEW
# numbered subfolder (run_1, run_2, run_3, ...) instead of overwriting the
# previous run's results. This is needed because grokking is stochastic –
# the grok epoch moves with the random seed – so a real comparison needs
# several independent runs kept side by side, not one run's numbers
# overwritten by the next. plot_results_four_head.py reads every run_&Nt;;
# folder it finds and plots them together for comparison.
# =====

LEGACY_FILENAMES = [
    "train_acc_history.npy", "test_acc_history.npy", "loss_history.npy",
    "l2_norm_history.npy", "dropout_gap_epochs.npy", "dropout_gap_history.npy",
    "dropout_train_acc_history.npy", "dropout_eval_acc_history.npy",
    "epoch_grid.npy", "fast_ma.npy", "slow_ma.npy", "fast_ma_of_slow_ma.npy",
    "ma_of_ma_diff.npy", "training_report.pdf",
    "ma_of_slow_ma_crossover.png", "ma_of_slow_ma_diff.png",
    "ma_of_slow_ma_diff_linear.png", "ma_of_ma_diff_vs_grokking_linear.png",
    "grokking_curve.png", "loss_curve.png", "l2_norm_curve.png", "dropout_gap_curve.png",
]

def migrate_legacy_flat_run(four_head_dir):
    """
    Earlier versions of this script saved directly into results/four_head/
    instead of results/four_head/run_&Nt;/. If such files are found here, and
    run_1/ does not exist yet, move them into run_1/ so they are counted
    as the first run instead of silently sitting outside the new
    numbering scheme (and instead of being silently overwritten by this
    run, which is what would have happened before this change).
    """
    run_1_dir = os.path.join(four_head_dir, "run_1")
    if os.path.isdir(run_1_dir):
        return
    legacy_present = any(
        os.path.isfile(os.path.join(four_head_dir, name)) for name in LEGACY_FILENAMES
    )
    if not legacy_present:
        return
    os.makedirs(run_1_dir, exist_ok=True)
    for name in LEGACY_FILENAMES:
        src = os.path.join(four_head_dir, name)
        if os.path.isfile(src):
            os.rename(src, os.path.join(run_1_dir, name))
    print("[MIGRATION] Found results saved directly in results/four_head/ by an earlier "
          "version of this script – moved them into results/four_head/run_1/ so they are "
          "counted as Run 1 instead of being overwritten.")
def get_next_run_number(four_head_dir):
```

```

    if not os.path.isdir(four_head_dir):
        return 1
    existing_numbers = []
    for name in os.listdir(four_head_dir):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(four_head_dir, name)):
            existing_numbers.append(int(match.group(1)))
    return max(existing_numbers, default=0) + 1

project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

four_head_dir = os.path.join(project_root, "runs", "four_head")
migrate_legacy_flat_run(four_head_dir)
run_number = get_next_run_number(four_head_dir)

output_dir = os.path.join(four_head_dir, f"run_{run_number}")
os.makedirs(output_dir, exist_ok=True)
os.chdir(output_dir)

# Initialize unified measurements system
measurements = PredictorMeasurements(output_dir, model_type="four_head")

print(f"This is Run {run_number}. All outputs will be saved to runs/four_head/run_{run_number}/")

# NEW random seed on every run, instead of a fixed one. torch.seed() picks
# a fresh, non-deterministic seed itself and also returns the exact number
# it used, so we can both use it and record it in the same step. This is
# needed because grokking is stochastic (grok epoch moves with the seed -
# see the L2 Norm predictor's 3-run history) and the thesis needs several
# genuinely independent 4-head runs, not the same run repeated.
seed = torch.seed()
print(f"Seed: {seed}")
np.save("seed.npy", seed)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_data_loaders(97, batch_size=int(0.3 * 97 * 97))
model = TransformerFourHead(vocab_size=98, d_model=128, num_heads=4).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Model: TransformerFourHead (num_heads=4, matches Nanda et al.)")
print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 40000
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []
dropout_gap_epochs = []
dropout_rates = [0.1, 0.3, 0.5, 0.7, 0.9]
dropout_gap_history_by_rate = {rate: [] for rate in dropout_rates}
dropout_train_acc_by_rate = {rate: [] for rate in dropout_rates}
dropout_eval_acc_by_rate = {rate: [] for rate in dropout_rates}

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

```

```

train_acc_history.append(total_correct / total_samples)
test_acc_history.append(test_total_correct / test_total_samples)
loss_history.append(loss.item())
l2_norm_history.append(compute_l2_norm(model))

# Dropout Gap Predictor: full multi-rate sweep across all 5 rates
# (0.1, 0.3, 0.5, 0.7, 0.9). There is no single "primary" rate – every
# rate is stored, and the analysis works on the whole sweep.
results = compute_dropout_gap_multi_rate(model, data_loader[1], dropout_rates)

dropout_gap_epochs.append(epoch + 1)
for rate in dropout_rates:
    dropout_gap_history_by_rate[rate].append(results[rate]["dropout_gap"])
    dropout_train_acc_by_rate[rate].append(results[rate]["train_accuracy"])
    dropout_eval_acc_by_rate[rate].append(results[rate]["eval_accuracy"])

# compute_dropout_gap_multi_rate() leaves the model in eval() mode with p
# reset to 0.0. Training must resume in train() mode, so we restore it here
# before the next epoch's training pass begins.
model.train()

gap_summary = ", ".join(f"p{rate}={results[rate]['dropout_gap']:.3f}" for rate in dropout_rates)
print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
      {train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
      {l2_norm_history[-1]:.4f}, Dropout Gap sweep: {gap_summary}")

# Save all measurements using unified system
measurements.save_training_data(train_acc_history, test_acc_history, loss_history)

grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

# L2 Norm Predictor: Moving Average Crossover Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model): Moving Average Crossover")
print("="*60)

epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history, fast_window=50,
    slow_window=200)

detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)

print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f"  MA Crossover epoch: {detection_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {lead_time:.1f} epochs")
else:
    print("  No MA crossover detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

print(f"\nL2 Norm Stats:")
print(f"  Min: {np.min(l2_norm_history):.4f}")
print(f"  Max: {np.max(l2_norm_history):.4f}")
print(f"  Final: {l2_norm_history[-1]:.4f}")

# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model): MA of Slow MA – Zero-Crossing Trigger")
print("="*60)

fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff, skip_epochs=100)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:
    trigger_lead_time = grok_epoch - trigger_epoch
    print(f"  Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {trigger_lead_time:.1f} epochs")
else:
    print("  No trigger detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

# Save all L2 Norm measurements
measurements.save_l2_norm_data(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch
)

# Dropout Predictor: gap tracked at every epoch during training

```

```

print("\n" + "="*60)
print("DROPOUT PREDICTOR (4-head model): Gap tracked every epoch")
print("="*60)
print(f"\nFinal Dropout Gap sweep (epoch {dropout_gap_epochs[-1]}):")
for rate in dropout_rates:
    print(f"    rate={rate}: gap={dropout_gap_history_by_rate[rate][-1]:+.4f}  "
          f"(train p={rate}: {dropout_train_acc_by_rate[rate][-1]:.4f}, "
          f"clean p=0.0: {dropout_eval_acc_by_rate[rate][-1]:.4f})")

# Save all Dropout measurements – full sweep, no single-rate record
measurements.save_dropout_data(
    dropout_gap_epochs, dropout_gap_history_by_rate,
    dropout_train_acc_by_rate, dropout_eval_acc_by_rate, dropout_rates
)

# =====
# VISUALIZATION & REPORT GENERATION
# =====
print("\n" + "="*60)
print("VISUALIZATION & REPORT GENERATION (4-head model)")
print("="*60)

# Generate standalone L2 Norm visualizations
measurements.generate_l2_norm_visualizations(
    l2_norm_history, epoch_grid, fast_ma, slow_ma,
    fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
    grok_epoch, detection_epoch, trigger_epoch
)
print("[OK] L2 Norm visualizations saved to runs/four_head/run_{}/l2_norm/".format(run_number))

# Generate standalone Dropout visualizations
measurements.generate_dropout_visualizations(
    dropout_gap_epochs, dropout_gap_history_by_rate,
    dropout_rates, test_acc_history, grok_epoch
)
print("[OK] Dropout visualizations saved to runs/four_head/run_{}/dropout/".format(run_number))

# Generate combined PDF report
report_path = measurements.generate_combined_report(
    train_acc_history, test_acc_history, loss_history, l2_norm_history,
    dropout_gap_epochs, dropout_gap_history_by_rate, dropout_rates,
    epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch
)
print(f"[OK] PDF report saved to {report_path} (4 pages: all measurements)")

print("\n" + "="*60)
print("RUN {} COMPLETE".format(run_number))
print("="*60)
print(f"Grok epoch: {grok_epoch}")
print(f"L2 Norm MA crossover: {detection_epoch if detection_epoch else 'Not detected'}")
print(f"L2 Norm MA-of-MA trigger: {trigger_epoch if trigger_epoch else 'Not detected'}")
print(f"\nAll measurements saved to runs/four_head/run_{run_number}/")
print(f"  - training/ : raw training data")
print(f"  - l2_norm/ : L2 norm measurements + visualizations")
print(f"  - dropout/ : dropout measurements + visualizations")
print(f"  - reports/ : combined PDF report")

```

49. archive/Deprecated_Code/src_p97_mainstream_old/train_shadow_layernorm.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os

from data.modular_arithmetic import get_dataloaders
from models.model_shadow_with_layernorm import Transformer

# Save results to project root (same pattern as train.py)
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Fixed seed, so this run can later be repeated with the same starting
# weights and the same train/test split. NOTE: train.py itself does not
# currently set any seed. If you want a true apples-to-apples comparison
# against the original model, add this same line to train.py yourself
# before that run too (temporarily) - this script does not modify train.py.
torch.manual_seed(1337)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_dataloaders(97, batch_size=int(0.3 * 97 * 97))
model = Transformer(vocab_size=98, d_model=128).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 10000
train_acc_history = []
test_acc_history = []
loss_history = []

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())

    if (epoch + 1) % 100 == 0 or epoch == 0:
        print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, "
              f"Train Accuracy: {train_acc_history[-1]:.4f}, "
              f"Test Accuracy: {test_acc_history[-1]:.4f}")

# Saved under a "shadow_ln_" prefix on purpose, so these files never
# overwrite train.py's own train_acc_history.npy / test_acc_history.npy /
# loss_history.npy, which the L2 Norm and Dropout predictors depend on.
np.save("shadow_ln_train_acc_history.npy", train_acc_history)
np.save("shadow_ln_test_acc_history.npy", test_acc_history)
np.save("shadow_ln_loss_history.npy", loss_history)

final_test_acc = test_acc_history[-1]
test_acc_array = np.array(test_acc_history)
grok_epoch = int(np.argmax(test_acc_array > 0.9)) if (test_acc_array > 0.9).any() else None
print("\n" + "=" * 60)
```

```
print("SHADOW MODEL (LayerNorm) RESULT")
print("=" * 60)
print(f"Final Test Accuracy: {final_test_acc:.4f}")
print(f"Grok epoch (test acc > 90%): {grok_epoch}")
print("Compare this Final Test Accuracy against the original model's known")
print("plateau value: 0.995446 (99.5446%) - see context.md, Aug 7 session.")
```

50. archive/Deprecated_Code/src_p97_mainstream_old/unified_measurements.py

```
"""
Unified measurement collection for L2 Norm and Dropout predictors.
Ensures consistent, complete measurements across single-head and four-head models.
"""

import numpy as np
import os
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

class PredictorMeasurements:
    """Collects and saves all measurements for both predictors."""

    def __init__(self, output_dir, model_type="single_head"):
        self.output_dir = output_dir
        self.model_type = model_type
        self.create_subdirs()

    def create_subdirs(self):
        """Create subdirectories for each predictor."""
        self.l2_norm_dir = os.path.join(self.output_dir, "l2_norm")
        self.dropout_dir = os.path.join(self.output_dir, "dropout")
        self.reports_dir = os.path.join(self.output_dir, "reports")
        self.training_dir = os.path.join(self.output_dir, "training")

        for dir_path in [self.l2_norm_dir, self.dropout_dir, self.reports_dir, self.training_dir]:
            os.makedirs(dir_path, exist_ok=True)

    def save_training_data(self, train_acc, test_acc, loss):
        """Save core training metrics."""
        np.save(os.path.join(self.training_dir, "train_acc_history.npy"), train_acc)
        np.save(os.path.join(self.training_dir, "test_acc_history.npy"), test_acc)
        np.save(os.path.join(self.training_dir, "loss_history.npy"), loss)

# ===== L2 NORM MEASUREMENTS =====

def save_l2_norm_data(self, l2_norm_history, epoch_grid, fast_ma, slow_ma,
                      fast_ma_of_slow_ma, ma_of_ma_diff, detection_epoch=None):
    """Save all L2 Norm measurements."""

    # Raw L2 norm
    np.save(os.path.join(self.l2_norm_dir, "l2_norm_history.npy"), l2_norm_history)

    # Smoothed L2 norm (simple low-pass filter)
    l2_norm_smoothed = self._smooth_signal(l2_norm_history, window=50)
    np.save(os.path.join(self.l2_norm_dir, "l2_norm_smoothed.npy"), l2_norm_smoothed)

    # Moving averages
    np.save(os.path.join(self.l2_norm_dir, "epoch_grid.npy"), epoch_grid)
    np.save(os.path.join(self.l2_norm_dir, "fast_ma.npy"), fast_ma)
    np.save(os.path.join(self.l2_norm_dir, "slow_ma.npy"), slow_ma)
    np.save(os.path.join(self.l2_norm_dir, "fast_ma_of_slow_ma.npy"), fast_ma_of_slow_ma)
    np.save(os.path.join(self.l2_norm_dir, "ma_of_ma_diff.npy"), ma_of_ma_diff)

    # Acceleration derivatives
    accel_raw = np.diff(l2_norm_history)
    accel_smoothed = self._smooth_signal(accel_raw, window=50)
    accel_double_smoothed = self._smooth_signal(accel_smoothed, window=50)

    np.save(os.path.join(self.l2_norm_dir, "acceleration_raw.npy"), accel_raw)
    np.save(os.path.join(self.l2_norm_dir, "acceleration_smoothed.npy"), accel_smoothed)
    np.save(os.path.join(self.l2_norm_dir, "acceleration_double_smoothed.npy"), accel_double_smoothed)

    if detection_epoch is not None:
        np.save(os.path.join(self.l2_norm_dir, "detection_epoch.npy"), np.array([detection_epoch]))

# ===== DROPOUT MEASUREMENTS =====

def save_dropout_data(self, dropout_gap_epochs, dropout_gap_history_by_rate,
                     dropout_train_acc_by_rate, dropout_eval_acc_by_rate, dropout_rates):
    """Save all Dropout measurements – full multi-rate sweep only.

    There is no single-rate (p=0.9) record any more. Every array is
    indexed [rate_index, epoch_index], with rate_index following the
    order of dropout_rates.npy.
    """

    np.save(os.path.join(self.dropout_dir, "dropout_gap_epochs.npy"), dropout_gap_epochs)
    np.save(os.path.join(self.dropout_dir, "dropout_rates.npy"), np.array(dropout_rates))

    gap_by_rate = np.array([dropout_gap_history_by_rate[r] for r in dropout_rates])
```

```

train_acc_by_rate = np.array([dropout_train_acc_by_rate[r] for r in dropout_rates])
eval_acc_by_rate = np.array([dropout_eval_acc_by_rate[r] for r in dropout_rates])

np.save(os.path.join(self.dropout_dir, "dropout_gap_by_rate.npy"), gap_by_rate)
np.save(os.path.join(self.dropout_dir, "dropout_train_acc_by_rate.npy"), train_acc_by_rate)
np.save(os.path.join(self.dropout_dir, "dropout_eval_acc_by_rate.npy"), eval_acc_by_rate)

# ===== VISUALIZATION GENERATION =====

def generate_l2_norm_visualizations(self, l2_norm_history, epoch_grid, fast_ma, slow_ma,
                                   fast_ma_of_slow_ma, ma_of_ma_diff, test_acc_history,
                                   grok_epoch, detection_epoch=None, trigger_epoch=None):
    """Generate standalone L2 Norm visualization graphs."""

    num_epochs = len(l2_norm_history)
    epochs_axis = range(1, num_epochs + 1)

    # 1. L2 Norm curve
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2)
    if detection_epoch is not None:
        ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                  label=f"MA Crossover (epoch {detection_epoch:.0f})")
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch (epoch {grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("L2 Norm")
    ax.set_title("L2 Norm of Model Weights")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.l2_norm_dir, "l2_norm_curve.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)

    # 2. MA crossover detection
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epoch_grid, fast_ma, label="Fast MA (window=50)", linewidth=2, color="blue")
    ax.plot(epoch_grid, slow_ma, label="Slow MA (window=200)", linewidth=2, color="orange")
    if detection_epoch is not None:
        ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                  label=f"Crossover (epoch {detection_epoch:.0f})")
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch (epoch {grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("L2 Norm")
    ax.set_title("Moving Average Crossover Detection")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_slow_ma_crossover.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)

    # 3. MA-of-MA differential
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epoch_grid, ma_of_ma_diff, color="darkred", linewidth=2, label="MA-of-MA Differential")
    ax.axhline(y=0, color="black", linestyle="--", linewidth=0.5, alpha=0.5)
    if trigger_epoch is not None:
        ax.axvline(x=trigger_epoch, color="red", linestyle="---", linewidth=2,
                  label=f"Zero-crossing (epoch {trigger_epoch:.0f})")
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch (epoch {grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("Differential")
    ax.set_title("MA-of-MA Differential: Zero-Crossing Trigger")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_ma_diff.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)

    # 4. MA-of-MA differential vs grokking (linear scale)
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.plot(epoch_grid, ma_of_ma_diff, color="darkred", linewidth=2, label="MA-of-MA Differential")
    ax.axhline(y=0, color="black", linestyle="--", linewidth=0.5, alpha=0.5)
    if trigger_epoch is not None:
        ax.axvline(x=trigger_epoch, color="red", linestyle="---", linewidth=2,
                  label=f"Zero-crossing (epoch {trigger_epoch:.0f})")
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch (epoch {grok_epoch})")
    ax.set_xscale("linear")
    ax.set_xlabel("Epoch (linear scale)")
    ax.set_ylabel("Differential")
    ax.set_title("MA-of-MA Differential: Linear Scale")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.l2_norm_dir, "ma_of_ma_diff_linear.png"), dpi=150, bbox_inches='tight')

```

```

plt.close(fig)

def generate_dropout_visualizations(self, dropout_gap_epochs, dropout_gap_history_by_rate,
                                   dropout_rates, test_acc_history, grok_epoch):
    """Generate standalone Dropout visualization graphs."""

    # Multi-rate sweep: all 5 rates on one plot
    fig, ax = plt.subplots(figsize=(12, 7))
    for rate in dropout_rates:
        ax.plot(
            dropout_gap_epochs,
            dropout_gap_history_by_rate[rate],
            linewidth=2,
            label=f"rate={rate}"
        )
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
              label=f"Grok epoch ({grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("Dropout Gap")
    ax.set_title("Dropout Gap: Multi-Rate Sweep (p=0.1, 0.3, 0.5, 0.7, 0.9)")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(self.dropout_dir, "dropout_gap_curve.png"), dpi=150, bbox_inches='tight')
    plt.close(fig)

def generate_combined_report(self, train_acc, test_acc, loss, l2_norm_history,
                             dropout_gap_epochs, dropout_gap_history_by_rate, dropout_rates,
                             epoch_grid, fast_ma, slow_ma, grok_epoch, detection_epoch=None):
    """Generate combined PDF report with all measurements."""

    num_epochs = len(train_acc)
    epochs_axis = range(1, num_epochs + 1)

    with PdfPages(os.path.join(self.reports_dir, "training_report.pdf")) as pdf:

        # Page 1: Grokking curve
        fig, ax = plt.subplots(figsize=(12, 7))
        ax.plot(epochs_axis, train_acc, label="Train Accuracy", color="steelblue", linewidth=2)
        ax.plot(epochs_axis, test_acc, label="Test Accuracy", color="seagreen", linewidth=2)
        ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
                  label=f"Grok epoch ({grok_epoch})")
        ax.set_xscale("log")
        ax.set_xlabel("Epoch (log scale)")
        ax.set_ylabel("Accuracy")
        ax.set_title("Grokking Curve: Train vs. Test Accuracy")
        ax.legend()
        ax.grid(True, alpha=0.3)
        pdf.savefig(fig)
        plt.close(fig)

        # Page 2: Loss curve
        fig, ax = plt.subplots(figsize=(12, 7))
        ax.plot(epochs_axis, loss, color="darkorange", linewidth=2)
        ax.set_xscale("log")
        ax.set_xlabel("Epoch (log scale)")
        ax.set_ylabel("Loss")
        ax.set_title("Training Loss")
        ax.grid(True, alpha=0.3)
        pdf.savefig(fig)
        plt.close(fig)

        # Page 3: L2 Norm with MA crossover
        fig, ax = plt.subplots(figsize=(12, 7))
        ax.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
        ax.plot(epoch_grid, fast_ma, label="Fast MA", linewidth=2, color="blue", alpha=0.7)
        ax.plot(epoch_grid, slow_ma, label="Slow MA", linewidth=2, color="orange", alpha=0.7)
        if detection_epoch is not None:
            ax.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                      label=f"Crossover ({detection_epoch:.0f})")
        ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
                  label=f"Grok ({grok_epoch})")
        ax.set_xscale("log")
        ax.set_xlabel("Epoch (log scale)")
        ax.set_ylabel("L2 Norm")
        ax.set_title("L2 Norm & Moving Averages")
        ax.legend()
        ax.grid(True, alpha=0.3)
        pdf.savefig(fig)
        plt.close(fig)

        # Page 4: Dropout Gap (multi-rate)
        fig, ax = plt.subplots(figsize=(12, 7))
        for rate in dropout_rates:
            ax.plot(
                dropout_gap_epochs,

```

```

        dropout_gap_history_by_rate[rate],
        linewidth=2,
        label=f"rate={rate}"
    )
    ax.axvline(x=grok_epoch, color="green", linestyle=":", linewidth=2,
               label=f"Grok ({grok_epoch})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)")
    ax.set_ylabel("Dropout Gap")
    ax.set_title("Dropout Gap: Multi-Rate Sweep")
    ax.legend()
    ax.grid(True, alpha=0.3)
    pdf.savefig(fig)
    plt.close(fig)

    return os.path.join(self.reports_dir, "training_report.pdf")

# ===== UTILITIES =====

@staticmethod
def _smooth_signal(signal, window=50):
    """Apply simple moving average smoothing."""
    if window > len(signal):
        window = len(signal)
    return np.convolve(signal, np.ones(window)/window, mode='same')

```